

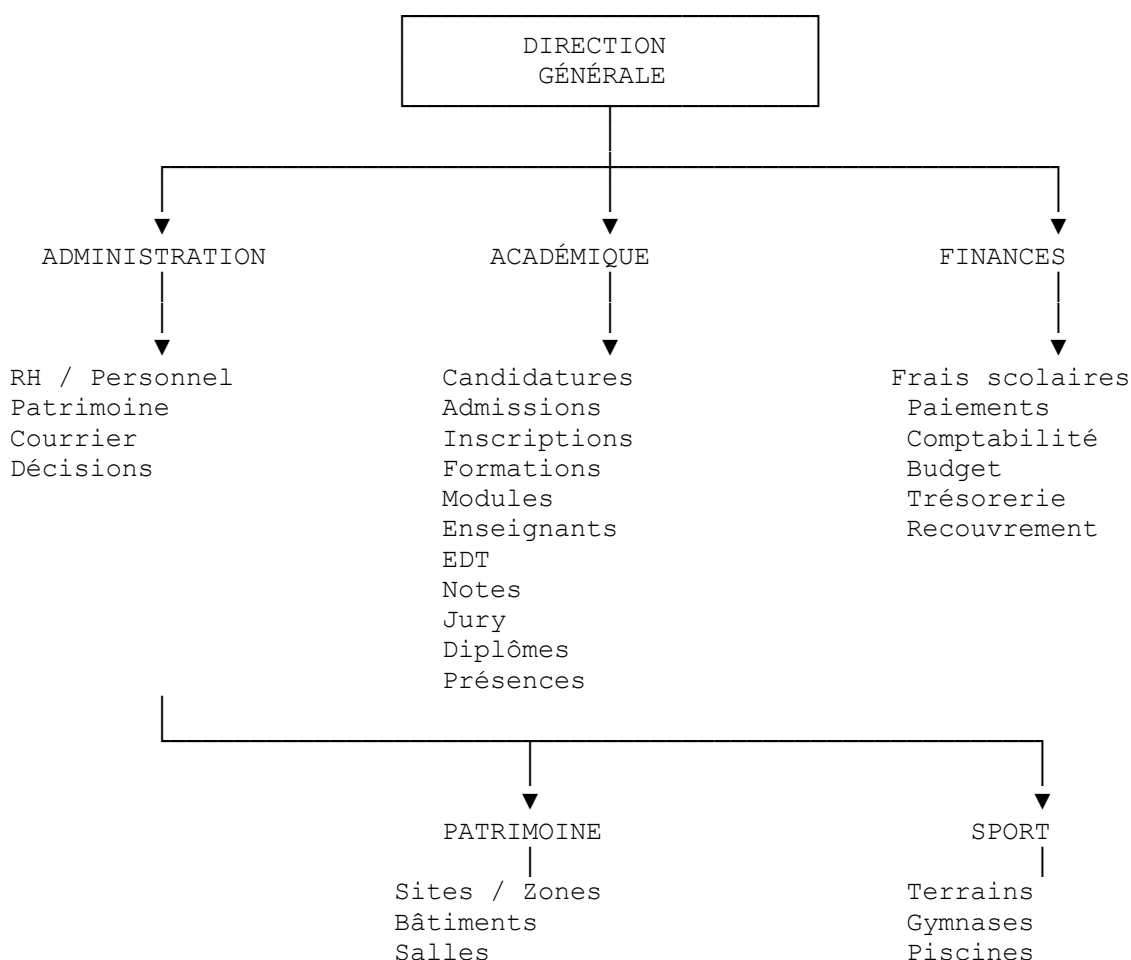
Oui. Pour concevoir une **application de gestion INJS Marcory réellement exploitable de bout en bout**, il faut éviter de penser uniquement « application scolaire ». L'INJS doit être modélisé comme un **établissement public d'enseignement supérieur spécialisé dans les métiers du sport, de la jeunesse et des loisirs**, avec des activités académiques, sportives, administratives, financières, logistiques et immobilières interconnectées.

L'INJS est implanté à **Marcory, terminus SOTRA**, et sa mission officielle comprend notamment la formation initiale et continue de cadres dans les domaines du sport, de la jeunesse et des loisirs. Le passage au système LMD a été engagé depuis plusieurs années ; des formations de type Professorat de collège sont notamment organisées sur trois ans et sanctionnées par une licence professionnelle.

Je te propose ci-dessous **le modèle fonctionnel de référence** que je retiendrais pour ton application `app-injs-lmd-2026`.

1. Vision globale du système INJS

L'application doit être organisée autour de **10 grands domaines fonctionnels** :



L'idée fondamentale est que **tous ces domaines doivent partager un référentiel unique.**

Par exemple :

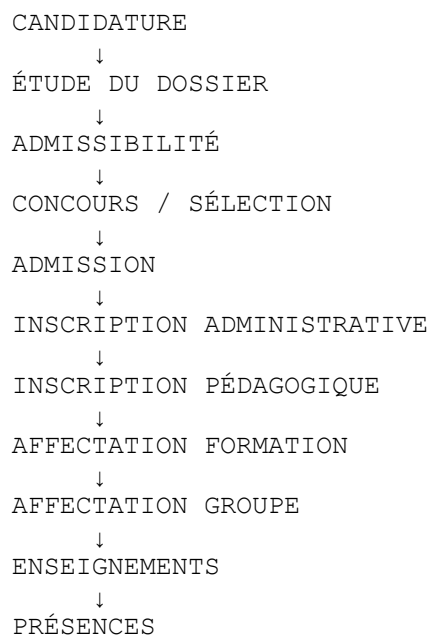
Une salle n'est pas simplement une ligne dans une table `Salle`.

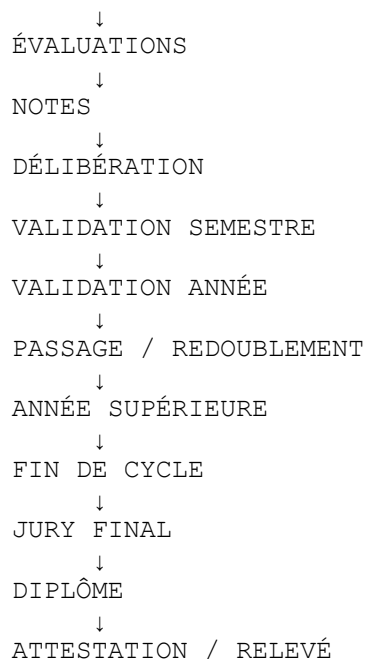
Elle doit pouvoir être reliée à :

- un site ;
- une zone ;
- un bâtiment ;
- un type d'espace ;
- une capacité ;
- une surface ;
- des équipements ;
- des formations autorisées ;
- des activités ;
- des créneaux EDT ;
- des examens ;
- des enseignants ;
- des groupes d'étudiants ;
- des réservations.

2. Le cycle académique complet

Le parcours étudiant doit être modélisé comme une chaîne :





C'est le cœur de ton application.

3. Étape 1 — Gestion des candidatures

L'application doit commencer par le **candidat**, et non par l'étudiant.

C'est une distinction extrêmement importante.

3.1 Candidat

Un candidat possède notamment :

- numéro de candidature ;
- identité ;
- nom ;
- prénoms ;
- sexe ;
- date de naissance ;
- lieu de naissance ;
- nationalité ;
- pièce d'identité ;
- téléphone ;
- email ;
- adresse ;
- photo ;
- personne à contacter ;
- diplôme antérieur ;
- établissement d'origine ;

- année d’obtention ;
 - série ;
 - spécialité ;
 - formation demandée.
-

4. Dossier de candidature

Le candidat sélectionne :

Année académique
↓
Campagne de recrutement
↓
Formation
↓
Option / parcours
↓
Centre de composition

Le système crée alors un **dossier de candidature**.

Exemple :

CAND-2026-000154

Formation :
Professorat de Collège

Option :
Éducation Physique et Sportive

Session :
2026

Statut :
DOSSIER COMPLET

5. Contrôle du dossier

Le système vérifie :

- pièces obligatoires ;
- âge ;
- diplôme ;
- conformité de la pièce d’identité ;
- photo ;
- certificat médical si nécessaire ;
- paiement éventuel ;
- conditions particulières de la formation.

Chaque pièce doit avoir son propre état :

NON FOURNIE
REÇUE
EN VÉRIFICATION
VALIDÉE
REJETÉE
À REMPLACER

6. Concours / sélection

Le système doit gérer plusieurs types de sélection.

Exemple

Concours

- Épreuves écrites
- Épreuves physiques
- Épreuves pratiques
- Épreuve orale
- Visite médicale
- Résultat final

Pour chaque épreuve :

- matière ;
- date ;
- heure ;
- centre ;
- bâtiment ;
- salle ;
- terrain ;
- gymnase ;
- piscine ;
- piste ;
- surveillants ;
- candidats convoqués ;
- numéro de table ;
- note ;
- absence ;
- fraude ;
- observation.

Les convocations réelles de l'INJS montrent justement une organisation par **cycle de formation, numéro de table, centre, salle, dates et horaires d'épreuves**.

7. Résultats du concours

Le moteur calcule :

Notes épreuves
↓
Coefficients
↓
Moyenne concours
↓
Classement
↓
Seuil d'admission
↓
Admis / Liste d'attente / Ajourné

Puis :

CANDIDAT
↓
ADMIS
↓
ÉTUDIANT

Ne jamais transformer automatiquement tous les candidats en étudiants.

C'est une règle importante d'architecture.

8. Inscription administrative

L'étudiant admis passe ensuite par l'inscription administrative.

Le système contrôle :

- admission ;
- identité ;
- pièces ;
- paiement ;
- assurance éventuelle ;
- situation administrative ;
- année académique.

L'étudiant reçoit alors un :

MAT matricule étudiant

Exemple :

INJS-LMD-2026-00125

9. Inscription pédagogique

Elle constitue une seconde étape.

L'étudiant est inscrit :

Année académique



Formation



Parcours



Niveau



Semestre



UE / ECUE / Modules



Groupe

10. Le référentiel LMD

Le système doit impérativement distinguer :

Formation

Exemple :

Licence Éducation et Motricité

Parcours

Exemple :

Éducation et Motricité

Niveau

L1

L2

L3

M1

M2

Semestre

S1

S2

S3

S4

S5

S6

...

Module / UE

Exemple :

Anatomie
Physiologie
Pédagogie
Psychologie
Biomécanique
Athlétisme
Natation
Football
Basketball
...

Catégorie d'enseignement

Dans ton modèle INJS :

CM
TD
TP
STAGE
VH

11. Maquette pédagogique

La maquette est le **contrat pédagogique de la formation**.

Elle doit définir :

Formation
↓
Parcours
↓
Niveau
↓
Semestre
↓
Module
↓
Crédits
↓
Coefficient
↓
Volume horaire
├── CM
├── TD
├── TP
└── Stage

Exemple :

Module	CM	TD	TP	Stage	VH	Crédits
Module A	15	15	0	0	30	3
Module B	10	10	20	0	40	4
Module C	0	0	30	0	30	3

12. Affectation des enseignants

L'enseignant ne doit pas être directement attaché à une salle ou à un module.

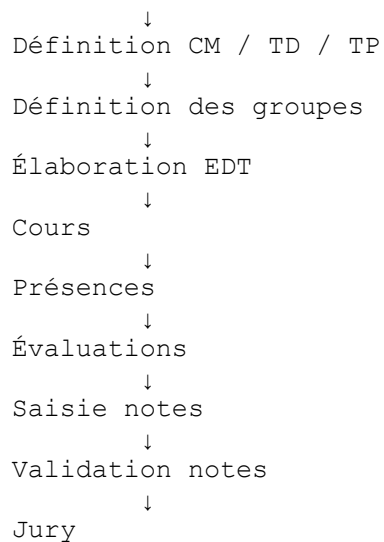
Il faut une chaîne :

ENSEIGNANT
↓
ANNÉE ACADÉMIQUE
↓
DÉPARTEMENT / STRUCTURE
↓
FORMATION
↓
PARCOURS
↓
NIVEAU
↓
SEMESTRE
↓
MODULE
↓
TYPE D'ENSEIGNEMENT
↓
GROUPE

13. Parcours de l'enseignant

Le parcours fonctionnel doit être :

Création fiche enseignant
↓
Validation
↓
Affectation administrative
↓
Affectation académique
↓
Affectation aux formations
↓
Affectation aux modules



14. Fiche enseignant

Elle doit contenir :

- matricule ;
 - nom ;
 - prénoms ;
 - grade ;
 - statut ;
 - spécialité ;
 - diplôme ;
 - département ;
 - téléphone ;
 - email ;
 - disponibilité ;
 - charge horaire ;
 - formations ;
 - modules ;
 - groupes ;
 - cours ;
 - évaluations.
-

15. Charge horaire enseignant

Le système doit automatiquement calculer :

Charge prévue
vs
Charge affectée
vs

Charge réalisée

Exemple :

Enseignant X

CM : 30 h

TD : 45 h

TP : 60 h

Stage : 20 h

TOTAL = 155 h

Le système doit détecter :

- surcharge ;
- sous-charge ;
- conflit ;
- double affectation.

16. Gestion des emplois du temps

Le moteur EDT doit travailler avec **4 contraintes simultanément** :

Ressource humaine

Enseignant

Ressource étudiante

Groupe

Ressource spatiale

Salle / Terrain / Gymnase / Piscine / Laboratoire...

Ressource temporelle

Jour + heure

17. Exemple de cours

Cours ID : CR-2026-000451

Formation :

Licence Éducation et Motricité

Niveau :

L2

Semestre :
S3

Module :
Athlétisme

Type :
TP

Enseignant :
Dr X

Groupe :
L2-EM-G2

Jour :
Mardi

08h00-10h00

Espace :
Piste d'athlétisme

Capacité :
60

Effectif :
42

18. Gestion intelligente des espaces

C'est une partie particulièrement importante pour l'INJS.

Il ne faut pas modéliser uniquement les salles.

Je recommande :

SITE
↓
ZONE
↓
BÂTIMENT
↓
NIVEAU
↓
ESPACE

Avec :

ESPACE
├─ Salle de cours
├─ Salle TD
├─ Salle TP
└─ Amphithéâtre

☐	Laboratoire
☐	Terrain
☐	Gymnase
☐	Piscine
☐	Piste
☐	Stade
☐	Salle spécialisée
☐	Bureau
☐	Salle de réunion
☐	Autre

19. Caractéristiques d'un espace

Chaque espace doit avoir :

- code ;
- nom ;
- type ;
- site ;
- zone ;
- bâtiment ;
- étage ;
- capacité ;
- superficie ;
- statut ;
- équipements ;
- accessibilité ;
- disponibilité ;
- spécialités autorisées.

Exemple :

PISTE-ATH-01

Type :
Piste d'athlétisme

Capacité :
120

Surface :
...

Équipements :
Chronométrage
Vestiaires
Éclairage
...

20. Le système doit connaître les incompatibilités

Par exemple :

Cours de natation
↓
Piscine obligatoire

et non :

Salle 204

Autre exemple :

TP biomécanique
↓
Laboratoire biomécanique

Autre :

Football
↓
Terrain approprié

21. Génération automatique des EDT

Le moteur peut fonctionner ainsi :

Maquettes
+
Enseignants
+
Groupes
+
Disponibilités
+
Espaces
+
Contraintes
+
Volumes horaires
↓
MOTEUR EDT
↓
Emploi du temps

Il doit rechercher :

CONFLIT ENSEIGNANT = 0
CONFLIT GROUPE = 0

CONFLIT ESPACE = 0
SURCAPACITÉ = 0
VH NON COUVERT = 0

22. Présences

Chaque séance doit produire une présence.

EDT
↓
Séance
↓
Cours
↓
Présence

Méthodes :

- QR Code ;
- saisie enseignant ;
- saisie manuelle autorisée ;
- correction ;
- justification ;
- absence ;
- retard.

Le système peut ensuite calculer :

Présences
Absences
Retards
Taux présence
Taux absence

23. Évaluations

Une évaluation doit être rattachée à :

Formation
Niveau
Semestre
Module
Groupe
Enseignant
Type évaluation
Date

Exemple :

CC
CT

24. Gestion des notes

Le processus doit être :

```
ENSEIGNANT
  ↓
SAISIE
  ↓
BROUILLON
  ↓
CONTRÔLE
  ↓
SOUMISSION
  ↓
VALIDATION PÉDAGOGIQUE
  ↓
JURY
  ↓
VERROUILLAGE
```

Après verrouillage :

l'enseignant ne doit plus pouvoir modifier directement la note.

Toute correction doit passer par une procédure de rectification tracée.

25. Calcul académique

Le moteur doit calculer :

```
Note module
  ↓
UE
  ↓
Semestre
  ↓
Année
  ↓
Diplôme
```

Il doit gérer :

- coefficients ;
- crédits ;

- moyenne ;
- compensation ;
- validation ;
- dette académique ;
- rattrapage ;
- redoublement ;
- passage ;
- jury.

26. Jury

Le jury doit avoir son propre module.

```
Préparation jury
↓
Contrôle données
↓
Calcul résultats
↓
Proposition
↓
Délibération
↓
Décision
↓
PV
↓
Verrouillage
```

Le système génère :

- PV de délibération ;
- liste des admis ;
- liste des ajournés ;
- relevés ;
- décisions ;
- statistiques.

27. Diplômation

Lorsqu'un étudiant satisfait toutes les conditions :

```
Étudiant
↓
Fin de cycle
↓
Contrôle crédits
↓
```

Contrôle notes
↓
Contrôle stages
↓
Contrôle obligations
↓
Jury final
↓
Diplômé

Le système génère :

- attestation ;
- relevé de notes ;
- diplôme ;
- numéro de diplôme ;
- registre des diplômés ;
- vérification de diplôme.

28. Gestion des frais scolaires

Le module financier étudiant doit être connecté au dossier académique.

Étudiant
↓
Formation
↓
Année académique
↓
Tarification
↓
Frais
↓
Échéancier
↓
Païement
↓
Quittance

29. Paramétrage des frais

Le système doit permettre de définir :

Formation
Niveau
Nationalité / catégorie éventuelle
Type de frais
Montant
Échéance

Types :

- inscription ;
- scolarité ;
- formation ;
- examen ;
- dossier ;
- carte ;
- document ;
- autres frais autorisés.

30. Échéancier

Exemple :

Scolarité annuelle : 300 000 FCFA

Tranche 1 : 100 000

Tranche 2 : 100 000

Tranche 3 : 100 000

Le système calcule :

Total dû

Total payé

Reste à payer

Retard

31. Paiements

Ton architecture doit pouvoir intégrer ultérieurement :

- paiement en caisse ;
- virement ;
- carte bancaire ;
- Orange Money ;
- MTN Money ;
- Moov Money.

Chaque paiement doit produire :

Transaction

↓

Référence

↓

Montant

↓

Date

↓
Mode
↓
Statut
↓
Quittance

32. Direction financière et comptable

Il faut distinguer :

Finance étudiant

Facturation
Paielements
Restes à payer
Recouvrement

Comptabilité

Recettes
Dépenses
Engagements
Ordres
Pièces
Budget
Trésorerie

33. Cycle financier

BUDGET
↓
ENGAGEMENT
↓
LIQUIDATION
↓
ORDONNANCEMENT
↓
PAIEMENT
↓
COMPTABILISATION
↓
RAPPROCHEMENT

Pour les recettes :

Facturation
↓
Encaissement
↓
Quittance
↓

Comptabilisation



Rapprochement bancaire

34. Direction financière — tableaux de bord

La direction doit pouvoir voir :

Recettes du jour
Recettes du mois
Recettes annuelles
Scolarité encaissée
Scolarité restante
Impayés
Dépenses
Budget consommé
Trésorerie

35. Administration générale

L'administration doit gérer :

Personnel

Agents
Enseignants
Vacataires
Contractuels
Administratifs

Courrier

Courrier entrant
Courrier sortant
Références
Destinataires
Pièces jointes

Décisions

Décisions
Notes de service
Arrêtés
Missions
Autorisations

Réunions

Conseils
Commissions
Jury

36. Ressources humaines

Le système RH doit connaître :

Agent
↓
Fonction
↓
Direction
↓
Service
↓
Grade
↓
Statut
↓
Affectation

Et pour les enseignants :

Enseignant
↓
Compétences
↓
Spécialités
↓
Formations
↓
Modules
↓
Charge horaire

37. Patrimoine

Le patrimoine doit être indépendant du module académique.

Patrimoine

- Immobilier
 - Sites
 - Zones
 - Bâtiments
 - Espaces
- Mobilier
- Matériel informatique
- Matériel sportif
- Véhicules
- Équipements

38. Maintenance

Un terrain, une piscine ou un laboratoire n'est pas simplement « disponible ».

Il peut être :

OPÉRATIONNEL
PARTIELLEMENT DISPONIBLE
EN MAINTENANCE
HORS SERVICE
RÉSERVÉ

Exemple :

Piscine
↓
Maintenance
↓
Du 05/09/2026 au 12/09/2026
↓
EDT concernés
↓
Cours à reprogrammer

39. Réservation des espaces

Il faut prévoir un moteur de réservation.

Demande
↓
Vérification disponibilité
↓
Contrôle conflit
↓
Validation
↓
Réservation

Cela permet de gérer :

- cours ;
 - examens ;
 - concours ;
 - stages ;
 - compétitions ;
 - réunions ;
 - manifestations ;
 - activités extérieures.
-

40. Architecture fonctionnelle finale

Je structurerais le menu principal de ton application comme ceci :

TABLEAU DE BORD

1. ACADÉMIQUE

- Années académiques
- Formations
- Parcours
- Maquettes
- Modules
- UE / ECUE
- Groupes
- Étudiants
- Candidatures
- Admissions
- Inscriptions
- Présences
- Évaluations
- Notes
- Jurys
- Diplômes

2. ENSEIGNANTS

- Enseignants
- Affectations
- Modules enseignés
- Groupes
- Charges horaires
- Disponibilités
- Évaluations

3. EMPLOIS DU TEMPS

- Génération automatique
- EDT formations
- EDT groupes
- EDT enseignants
- EDT espaces
- Examens
- Conflits

4. CAMPUS / PATRIMOINE

- Sites
- Zones
- Bâtiments
- Étages
- Salles
- Amphithéâtres
- Laboratoires
- Terrains
- Gymnases
- Piscines
- Pistes
- Équipements
- Maintenance
- Réservations

5. FINANCES

- Tarification
- Frais scolaires
- Échéanciers
- Paiements
- Quittances
- Impayés
- Recouvrement
- Budget
- Dépenses
- Comptabilité
- Trésorerie

6. ADMINISTRATION

- Personnel
- RH
- Services
- Courrier
- Décisions
- Réunions
- PV
- Archives

7. RÉFÉRENTIELS

- Formations
- Modules
- Catégories
- Grades
- Vagues
- Sites
- Zones
- Bâtiments
- Salles
- Types de secrétariat

8. DOCUMENTS

- Convocations
- Certificats
- Attestations
- Relevés
- PV
- Diplômes

9. RAPPORTS / STATISTIQUES

10. PARAMÈTRES

41. Architecture technique Django + Flutter

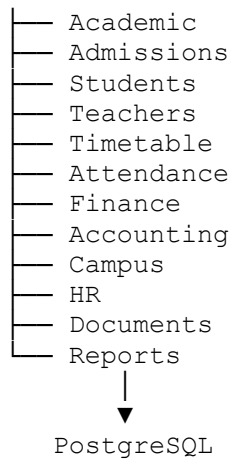
Pour ton projet, je recommande de séparer clairement :

Flutter

↓ HTTPS / REST
▼

Django REST Framework

- Authentication
- Permissions



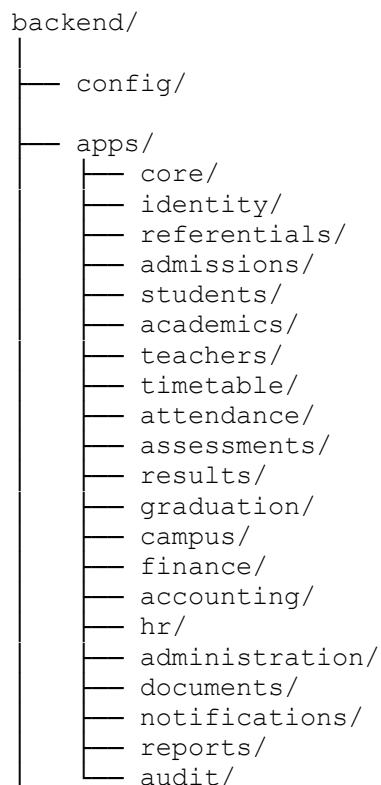
Avec éventuellement :

Redis
Celery
Object Storage
PDF generation
Notification service
Audit service

42. Structure Django recommandée

Je déconseille une énorme application Django unique.

Je partirais plutôt sur :



43. Flutter

Flutter doit avoir des espaces différents selon les utilisateurs.

Étudiant

Accueil
Mes informations
Ma formation
Mes inscriptions
Mon EDT
Mes cours
Mes présences
Mes notes
Mes paiements
Mes documents
Mes notifications

Enseignant

Accueil
Mes formations
Mes modules
Mes groupes
Mon EDT
Mes séances
Présences
Évaluations
Saisie des notes
Mes charges

Administration académique

Candidatures
Admissions
Inscriptions
Étudiants
Formations
Maquettes
Groupes
EDT
Notes
Jurys
Diplômes

Finance

Dashboard financier
Facturation
Paiements
Impayés
Recouvrement
Budget
Dépenses

Comptabilité

Direction

Dashboard stratégique
Effectifs
Admissions
Réussite
Formations
Charges enseignants
Occupation espaces
Finances
Statistiques

44. Le principe architectural le plus important

Il faut éviter de créer des données isolées.

Par exemple :

Formation

doit être la même formation utilisée par :

Candidature
Admission
Étudiant
Inscription
Maquette
Module
Enseignant
Groupe
EDT
Notes
Jury
Diplôme
Finance
Statistiques

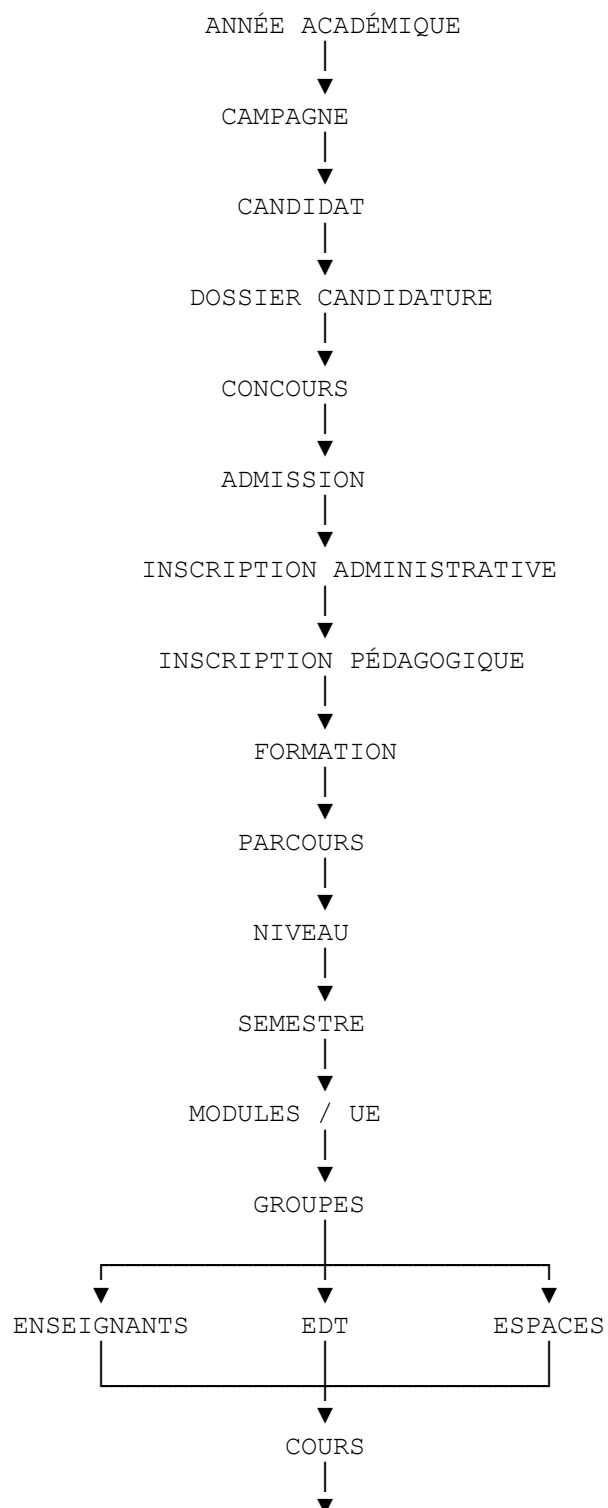
Même chose pour :

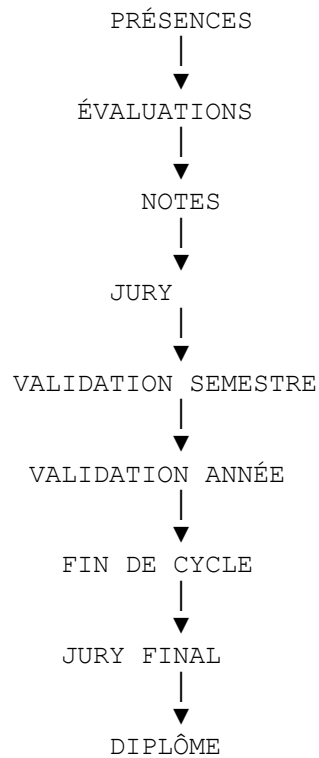
Étudiant
Enseignant
Module
Salle
Année académique
Formation

C'est ce qui permettra à ton système de devenir un **véritable SI académique INJS**, plutôt qu'une juxtaposition de modules.

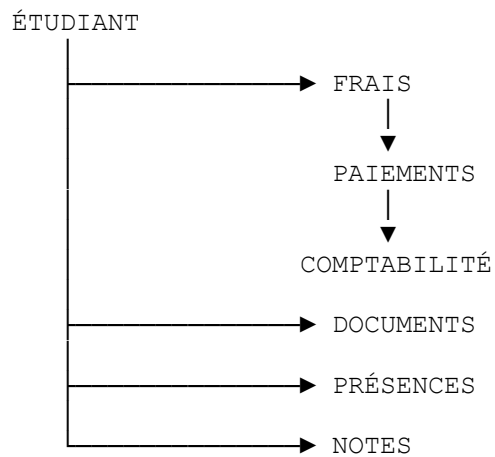
45. Le parcours complet dans une seule chaîne

Le fonctionnement final doit pouvoir être représenté ainsi :





Et parallèlement :



46. Ce que je recommande pour

app-injs-lmd-2026

Compte tenu de l'application INJS-LMD que tu es actuellement en train de construire, **je ne commencerais surtout pas par coder tous ces modules en même temps.**

Je construirais le système selon cet ordre :

PHASE 1 — Socle

Authentication

RBAC / Permissions
Utilisateurs
Rôles
Année académique
Référentiels
Audit

PHASE 2 — Référentiel académique

Formations
Parcours
Niveaux
Semestres
Modules
Catégories
Maquettes

PHASE 3 — Étudiants

Candidatures
Concours
Admissions
Inscriptions
Étudiants
Groupes

PHASE 4 — Enseignants

Enseignants
Affectations
Modules
Groupes
Charges horaires

PHASE 5 — EDT / Campus

Sites
Zones
Bâtiments
Salles
Terrains
Gymnases
Piscines
Laboratoires
Disponibilités
EDT
Réservations

PHASE 6 — Pédagogie

Cours
Séances
Présences
Évaluations
Notes
Jury

PHASE 7 — Diplômation

Validation
PV
Relevés
Attestations
Diplômes
Registre diplômés

PHASE 8 — Finance

Tarifs
Frais
Échéanciers
Palements
Quittances
Impayés
Recouvrement

PHASE 9 — Finance/Comptabilité

Budget
Engagement
Dépenses
Recettes
Trésorerie
Comptabilité
Rapprochements

PHASE 10 — Administration

RH
Personnel
Courrier
Décisions
Réunions
Archives
Patrimoine
Maintenance

Point important sur les données INJS réelles

Il faut distinguer **le fonctionnement cible de l'application** des informations publiques réellement vérifiables sur l'INJS. Les sources publiques confirment notamment son statut, son implantation à Marcory, ses missions de formation et l'existence de formations/cycles liés au système LMD. En revanche, une liste exhaustive et à jour de **toutes les salles, bâtiments, laboratoires, terrains, piscines, capacités, surfaces, tarifs internes, procédures comptables et règles pédagogiques** ne peut pas être déduite sérieusement de quelques sources publiques : ces éléments doivent être alimentés par les référentiels/documents officiels de l'INJS.

Pour ton application, je recommande donc une architecture qui ne “code” jamais en dur ces informations. Elles doivent être des données administrables dans Référentiels.

La prochaine étape logique est de transformer cette architecture fonctionnelle en **un véritable cahier des charges technique de app-injs-lmd-2026**, avec **toutes les entités Django, leurs relations, les workflows, statuts, rôles/permissions N0–N4, API REST, écrans Flutter, règles métier, contraintes EDT, modèles de données et ordre exact d’implémentation**. C’est ce document qui pourra ensuite servir de **spécification maître à Cline/Ollama** pour construire le système sans casser l’existant.

Parfait.

La prochaine étape sera de produire le **cahier des charges technique maître de app-injs-lmd-2026**, en partant de l’architecture ci-dessus et en la transformant en spécification directement exploitable par **Django + DRF + Flutter + PostgreSQL**, puis par **Cline/Ollama**.

Je le structurerai notamment en :

1. **Architecture générale**
2. **Cartographie exhaustive des modules**
3. **Modèle de données Django** — entités, champs, relations, contraintes
4. **Référentiels INJS**
5. **Workflow candidature → admission → inscription → diplôme**
6. **Workflow enseignant → affectation → cours → notes**
7. **Maquettes LMD et volumes horaires**
8. **Moteur EDT et gestion des espaces**
9. **Présences et QR Code**
10. **Évaluations, notes et jurys**
11. **Diplômation et documents officiels**
12. **Frais scolaires, paiements et recouvrement**
13. **Finance et comptabilité**
14. **Administration, RH et patrimoine**
15. **Rôles et permissions N0–N4**
16. **API REST**
17. **Interfaces Flutter Web/Mobile**
18. **Notifications et traçabilité**
19. **Audit et sécurité**
20. **Tableaux de bord et statistiques**
21. **Plan de migration de l’application actuelle**
22. **Plan d’implémentation par lots pour Cline**
23. **Tests et critères d’acceptation**
24. **Règles strictes empêchant Cline de casser l’existant**

L’objectif sera d’obtenir **une spécification suffisamment précise pour que chaque module puisse être développé, testé et validé indépendamment**, sans devoir réécrire l’application à chaque nouvelle fonctionnalité.

Très bien. Nous avons maintenant la **base fonctionnelle de référence**.

Pour la suite, je te conseille de procéder dans cet ordre :

Étape 1 → Architecture fonctionnelle détaillée

→ modules, sous-modules, workflows et interactions.

Étape 2 → Modèle de données complet

→ modèles Django, champs, relations, contraintes et règles métier.

Étape 3 → RBAC N0–N4

→ qui peut consulter, créer, modifier, valider, verrouiller ou supprimer chaque donnée.

Étape 4 → API Django REST Framework

→ endpoints, paramètres, réponses, filtres, pagination et permissions.

Étape 5 → Interfaces Flutter

→ écrans, navigation, tableaux de bord et parcours par profil.

Étape 6 → Plan d'implémentation Cline/Ollama

→ petits prompts successifs, avec vérification après chaque lot afin de protéger l'application existante.

Je recommande de commencer par **l'Étape 1 : la cartographie fonctionnelle exhaustive de l'INJS-LMD**, avant de toucher au code.

Parfait. Nous allons donc commencer par **l'Étape 1 : la cartographie fonctionnelle exhaustive de l'application INJS-LMD**.

Je vais la construire comme une **architecture de référence**, avec :

- modules ;
- sous-modules ;
- fonctionnalités ;
- acteurs concernés ;
- workflows ;
- dépendances entre modules ;
- données entrantes/sortantes ;
- règles métier ;
- liens avec les autres domaines.

Ensuite, cette cartographie pourra servir directement de base au **modèle Django**, aux **API DRF**, aux **écrans Flutter** et aux **prompts Cline/Ollama**.

Très bien. On peut passer à la construction proprement dite.

Je vais considérer désormais cette cartographie comme le **socle fonctionnel de référence** du projet app-injs-lmd-2026, puis descendre progressivement jusqu'au modèle technique Django/Flutter.

Ordre de travail recommandé :

Référentiels → Académique → Étudiants → Enseignants → EDT/Espaces → Présences → Évaluations/Notes → Jurys/Diplômes → Finances → Administration → Reporting → Paramètres/Sécurité

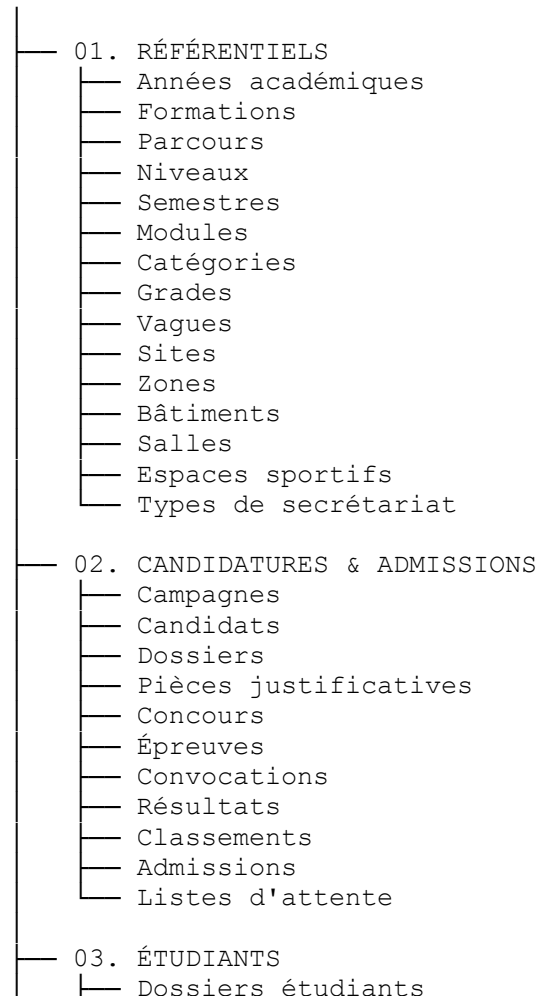
La prochaine production utile est la **cartographie détaillée de chaque module et sous-module**, avec les acteurs, données, statuts et workflows associés.

Nous pouvons donc démarrer avec le **Lot 1 — Cartographie fonctionnelle détaillée INJS-LMD**.

Je te propose de le traiter comme un **référentiel maître**, qui servira ensuite à générer le modèle de données Django et les prompts Cline.

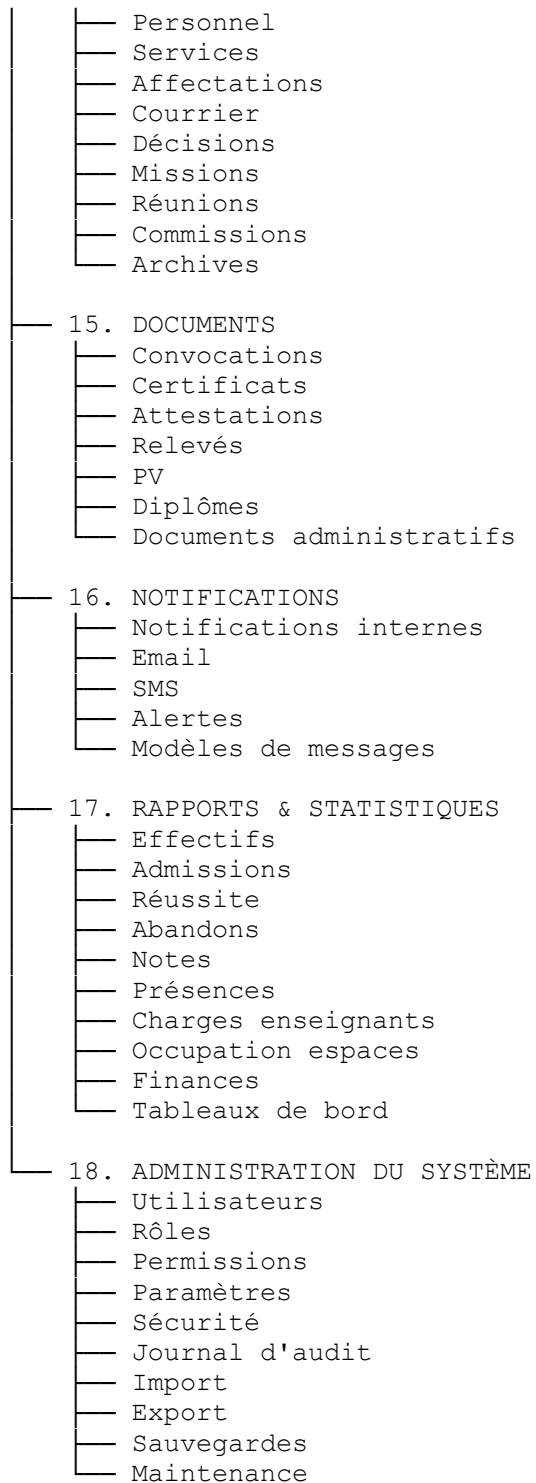
Lot 1 — Architecture fonctionnelle

INJS-LMD



	Matricules
	Inscriptions administratives
	Inscriptions pédagogiques
	Groupes
	Changements de formation
	Transferts
	Réinscriptions
	Abandons
	Situations académiques
04.	FORMATIONS & PÉDAGOGIE
	Maquettes
	UE / ECUE / Modules
	Crédits
	Coefficients
	CM
	TD
	TP
	Stages
	Volumes horaires
05.	ENSEIGNANTS
	Fiches enseignants
	Grades
	Spécialités
	Affectations
	Modules enseignés
	Groupes
	Disponibilités
	Charges horaires
	Services réalisés
06.	EMPLOIS DU TEMPS
	Contraintes
	Disponibilités
	Génération automatique
	EDT formations
	EDT groupes
	EDT enseignants
	EDT espaces
	Examens
	Réservations
	Gestion des conflits
07.	CAMPUS & PATRIMOINE
	Sites
	Zones
	Bâtiments
	Étages
	Salles
	Amphithéâtres
	Laboratoires
	Terrains
	Gymnases
	Piscines
	Pistes
	Équipements
	Maintenance
	Réservations
08.	PRÉSENCES

	— Séances
	— QR Code
	— Présence enseignant
	— Saisie manuelle
	— Retards
	— Absences
	— Justifications
	— Statistiques
—	09. ÉVALUATIONS & NOTES
	— Types d'évaluation
	— Épreuves
	— Calendrier
	— Saisie
	— Contrôle
	— Validation
	— Corrections
	— Rattrapages
	— Historique
—	10. JURYS
	— Composition
	— Sessions
	— Candidats au jury
	— Calculs
	— Délibérations
	— Décisions
	— PV
	— Verrouillage
—	11. DIPLÔMES
	— Contrôle fin de cycle
	— Diplômes
	— Attestations
	— Relevés
	— Numérotation
	— Registre
	— Vérification
—	12. FINANCES ÉTUDIANTS
	— Tarification
	— Frais
	— Facturation
	— Échéanciers
	— Paiements
	— Quittances
	— Impayés
	— Recouvrement
—	13. FINANCES & COMPTABILITÉ
	— Budget
	— Recettes
	— Dépenses
	— Engagements
	— Liquidations
	— Ordonnancement
	— Paiements
	— Trésorerie
	— Rapprochement
—	14. ADMINISTRATION



Principe directeur

Le système doit être construit autour de quelques **objets maîtres** :

**Année académique → Formation → Parcours → Niveau → Semestre → Module →
Groupe → Étudiant → Enseignant → Séance → Espace → Évaluation → Note → Jury
→ Diplôme.**

Les finances et l'administration viennent ensuite se rattacher à ces objets, plutôt que de créer des données parallèles.

Prochaine étape : descendre dans le **module 01 — Référentiels**, et définir précisément chaque référentiel, ses champs, ses relations, ses règles de gestion et son utilisation par les autres modules.

Parfait. Nous allons donc attaquer **01 — RÉFÉRENTIELS**, qui constitue le socle de tout app-injs-lmd-2026.

L'objectif est de définir les référentiels **avant** les écrans et avant les modèles Django, afin d'éviter les incohérences entre formations, étudiants, enseignants, EDT, espaces et finances.

01 — RÉFÉRENTIELS

RÉFÉRENTIELS

- 01. Années académiques
- 02. Formations
- 03. Parcours
- 04. Niveaux
- 05. Semestres
- 06. Modules
- 07. Catégories
- 08. Grades
- 09. Vagues
- 10. Sites
- 11. Zones
- 12. Bâtiments
- 13. Espaces
- 14. Types d'espaces
- 15. Types de secrétariat
- 16. Types d'enseignement
- 17. Types d'évaluation
- 18. Types de documents
- 19. Paramètres académiques

Relation fondamentale

ANNÉE ACADÉMIQUE



FORMATION



PARCOURS

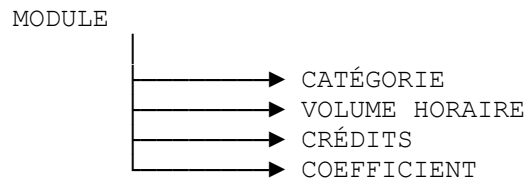


NIVEAU

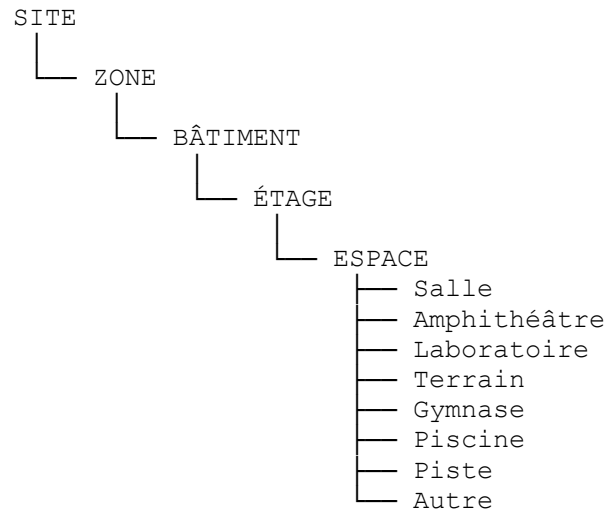


SEMESTRE





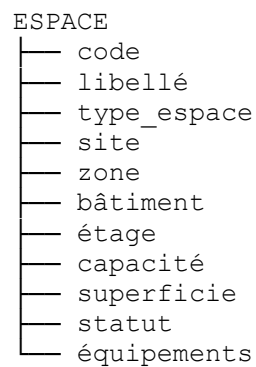
Pour le campus :



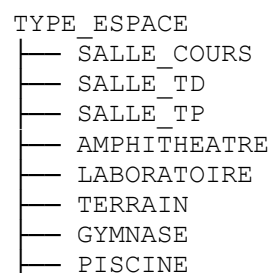
Règle importante

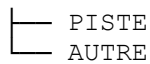
Je recommande de ne **pas** créer directement une table Django différente pour chaque type d'infrastructure (Piscine, Gymnase, Terrain, etc.).

Il vaut mieux avoir un référentiel commun :



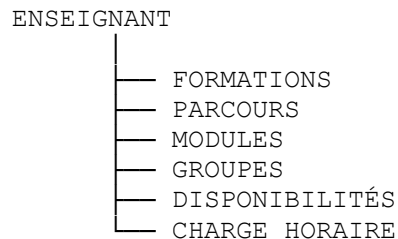
avec :



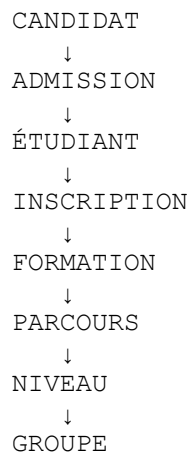


Cela permettra au moteur EDT de rechercher **n'importe quel espace compatible avec une activité**, sans multiplier les règles dans le code.

Même principe pour les enseignants



Et pour les étudiants :



Cette structure nous permettra ensuite de construire le **modèle relationnel Django sans doublons et sans dépendances circulaires inutiles**.

Étape suivante : définir précisément les **19 référentiels**, avec pour chacun **champs, types de données, clés, contraintes, relations, statuts et règles métier**.

Parfait. Je poursuis avec la **spécification opérationnelle des 19 référentiels**. L'objectif est qu'elle puisse ensuite être traduite presque directement en modèles Django/DRF.

01 — RÉFÉRENTIELS : SPÉCIFICATION MAÎTRE

1. Année académique

Exemple :

2026-2027

Champs principaux :

Champ	Type	Règle
code	string	unique
libelle	string	obligatoire
date_debut	date	obligatoire
date_fin	date	obligatoire
statut	enum	BROUILLON / OUVERTE / CLOTUREE
est_active	bool	une seule année active
description	text	facultatif

Une année clôturée ne doit plus permettre de modifier les résultats académiques définitifs.

2. Formation

La formation est un objet central.

Exemples de données à gérer :

Licence
Master
Formation professionnelle
Formation continue
Concours / cycle spécifique

Champs :

code
libelle
description
type_formation
niveau_diplome
duree
credits_total
statut
date_ouverture
date_fermeture

Relations :

Formation

- Parcours
 - Maquettes
 - Candidatures
 - Étudiants
 - Enseignants
 - Groupes
-

3. Parcours

Un parcours précise la spécialisation d'une formation.

Formation

↓

Parcours

Champs :

code
libelle
formation
description
duree
credits_total
statut

Contrainte :

Un parcours appartient à une seule formation.

4. Niveau

Référentiel :

L1
L2
L3
M1
M2

Mais le système doit permettre d'ajouter d'autres niveaux/cycles sans modification du code.

Champs :

code
libelle
ordre
cycle
credits_cibles
statut

5. Semestre

S1
S2
S3
S4
S5
S6
...

Champs :

code
libelle
ordre
niveau
duree
credits_cibles
statut

6. Module

C'est l'un des objets les plus importants.

Champs :

code
reference
libelle
description
type_module
credits
coefficient
statut

Relations :

Module
├── Catégories d'enseignement
├── Maquette
├── Enseignants
├── Groupes
├── Évaluations
└── Notes

7. Catégorie d'enseignement

Pour l'INJS, le référentiel doit notamment supporter :

CM = Cours magistral
TD = Travaux dirigés

TP = Travaux pratiques
STAGE = Stage

Et conserver VH comme **volume horaire calculé**, plutôt que comme une catégorie pédagogique, sauf si les documents officiels de l'INJS imposent explicitement ce traitement.

Exemple :

CM = 20 h
TD = 15 h
TP = 25 h

VH = 60 h

8. Grade

Le grade doit rester indépendant de la formation.

Il peut être utilisé notamment pour :

Étudiants
Enseignants
Personnel

Mais attention :

Grade étudiant et grade enseignant ne doivent pas être confondus fonctionnellement, même si un référentiel générique peut exister.

Pour les étudiants, par exemple, il peut représenter une catégorie/situation définie par l'établissement.

9. Vague

Très utile pour organiser les promotions/cohortes.

Exemple :

VAGUE 2026
VAGUE 2027

Relations :

Vague
├── Année académique
├── Formation
├── Étudiants
└── Admission

10. Site

Le site représente un emplacement géographique ou administratif.

```
SITE
├── code
├── nom
├── adresse
├── description
├── latitude
├── longitude
└── statut
```

Cela permettra à terme de gérer plusieurs sites si l'INJS en dispose ou en crée.

11. Zone

Hierarchie :

```
SITE
↓
ZONE
```

Exemple :

```
Site  INJS
├── Zone administrative
├── Zone pédagogique
├── Zone sportive
├── Zone résidentielle
└── Zone technique
```

Champs :

```
code
nom
site
description
statut
```

12. Bâtiment

```
ZONE
↓
BÂTIMENT
```

Champs :

```
code
nom
zone
nombre_etages
surface_totale
fonction_principale
statut
```

13. Espace

C'est le référentiel permettant de gérer toutes les infrastructures.

ESPACE

Champs recommandés :

```
code
nom
type_espace
site
zone
batiment
niveau
surface
capacite
statut
description
```

14. Type d'espace

Référentiel extensible :

```
SALLE_COURS
SALLE_TD
SALLE_TP
AMPHITHEATRE
LABORATOIRE
BUREAU
SALLE_REUNION
GYMNASE
TERRAIN
PISCINE
PISTE_ATHLETISME
STADE
ESPACE_POLYVALENT
AUTRE
```

Il faudra également prévoir des caractéristiques spécialisées :

```
Éclairage
Vestiaires
Gradins
Matériel audiovisuel
```

Matériel informatique
Matériel sportif
Lignes de terrain
Chronométrage
etc.

15. Type de secrétariat

Le référentiel doit permettre de rattacher un secrétariat à une structure.

Exemple :

Secrétariat académique
Secrétariat pédagogique
Secrétariat de département
Secrétariat de direction
Secrétariat financier
Secrétariat administratif

Champs :

code
libelle
description
niveau
statut

16. Type d'enseignement

À distinguer de la catégorie d'enseignement.

Exemple :

Présentiel
Distanciel
Hybride
Terrain
Laboratoire
Pratique sportive
Stage

Cette distinction sera importante pour l'EDT.

17. Type d'évaluation

Référentiel extensible :

CC
CT
TP
EXAMEN
RATTRAPAGE
ORAL
PROJET
STAGE

Chaque type peut avoir :

coefficient
poids
durée
mode
note_max

18. Type de document

Le système doit centraliser les documents.

Exemple :

PIECE_IDENTITE
DIPLOME_ANTERIEUR
CERTIFICAT
CONVOCATION
ATTESTATION
RELEVÉ_NOTES
PV_JURY
DIPLOME
QUITTANCE
CERTIFICAT_SCOLARITE

Chaque type peut définir :

obligatoire
signature_requise
cachet_requis
numérotation
duree_validite

19. Paramètres académiques

Ce référentiel doit éviter de mettre des règles métier directement dans le code.

Exemples :

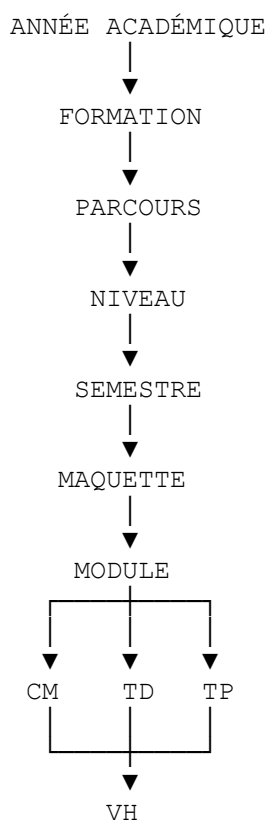
Nombre de crédits Licence
Nombre de crédits Master
Note minimale
Seuil de validation

Règles de compensation
Nombre de tentatives
Seuil de présence
Règles de rattrapage

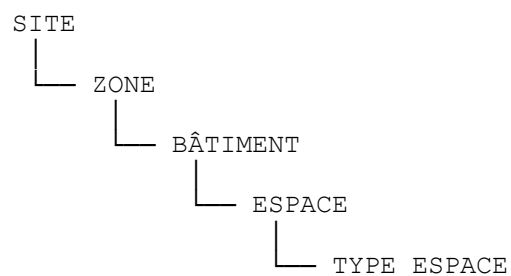
Ainsi, si une règle académique change, l'administration peut modifier le paramètre sans modifier Django.

Architecture relationnelle

Le socle peut finalement être représenté ainsi :



Et pour le campus :



Règles d'architecture à imposer à Django

Je recommande fortement les règles suivantes pour `app-injs-lmd-2026` :

R1 — Aucun doublon de référentiel

Une formation ne doit exister qu'une seule fois.

R2 — Soft delete

Éviter les suppressions physiques des données académiques.

Utiliser :

```
is_active
deleted_at
```

ou un mécanisme équivalent.

R3 — Historisation

Les modifications importantes doivent être historisées.

Exemple :

```
Formation
↓
Modification
↓
Utilisateur
↓
Date
↓
Ancienne valeur
↓
Nouvelle valeur
```

R4 — Année académique obligatoire

Toute donnée académique opérationnelle doit être rattachée à une année académique lorsqu'elle est temporelle.

R5 — Référentiels administrables

Les valeurs ne doivent pas être codées en dur dans Flutter.

R6 — Identifiants internes invisibles

L'interface utilisateur doit afficher :

```
FORMATION : EM-LIC
```

plutôt que des IDs techniques comme :

```
formation_id = 37
```

R7 — Code métier unique

Chaque objet important doit posséder un code exploitable :

```
FORM-EM-LIC  
MOD-ANA-001  
BAT-001  
SAL-204
```

R8 — API découplée

Flutter ne doit jamais accéder directement à PostgreSQL.

```
Flutter  
↓  
HTTPS  
↓  
DRF  
↓  
Django  
↓  
PostgreSQL
```

Et surtout : ne pas encore coder

À ce stade, **ne donne pas encore à Cline un prompt lui demandant de créer tous ces modèles.**

Nous devons d'abord définir les modules métier suivants et leurs relations :

02 — Candidatures & Admissions → 03 — Étudiants → 04 — Formations/Pédagogie → 05 — Enseignants → 06 — EDT → 07 — Campus → 08 — Présences → 09 — Évaluations/Notes → 10 — Jurys → 11 — Diplômes → 12/13 — Finances → 14 — Administration.

Une fois cette cartographie terminée, nous pourrons construire **le modèle relationnel global**, puis seulement après les modèles Django et les API.

Parfait. Nous continuons donc méthodiquement.

Prochaine étape : Module 02 — Candidatures & Admissions, en détaillant le parcours complet :

```
Campagne → Candidat → Dossier → Pièces → Épreuves/Concours → Convocation →  
Évaluation → Classement → Admission → Liste d'attente → Inscription
```

Nous définirons pour chaque étape :

- les acteurs ;
- les données ;
- les statuts ;
- les règles métier ;
- les validations ;
- les documents générés ;
- les notifications ;
- les relations Django ;
- les contrôles anti-doublons ;
- les traces d'audit.

Puis nous enchaînerons avec **Module 03 — Étudiants**, afin de garantir une continuité parfaite entre admission et inscription.

Parfait. Nous continuons donc méthodiquement.

Prochaine étape : Module 02 — Candidatures & Admissions, en détaillant le parcours complet :

Campagne → Candidat → Dossier → Pièces → Épreuves/Concours → Convocation → Évaluation → Classement → Admission → Liste d'attente → Inscription

Nous définirons pour chaque étape :

- les acteurs ;
- les données ;
- les statuts ;
- les règles métier ;
- les validations ;
- les documents générés ;
- les notifications ;
- les relations Django ;
- les contrôles anti-doublons ;
- les traces d'audit.

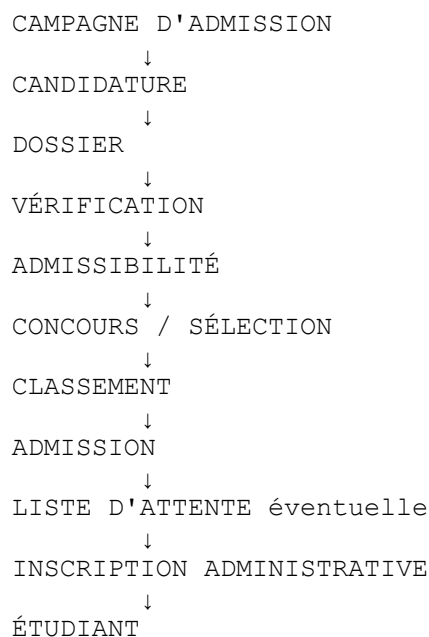
Puis nous enchaînerons avec **Module 03 — Étudiants**, afin de garantir une continuité parfaite entre admission et inscription.

Parfait. Nous poursuivons avec le **Module 02 — CANDIDATURES & ADMISSIONS**, en gardant comme référence l'architecture fonctionnelle de `app-injs-lmd-2026`.

MODULE 02 — CANDIDATURES & ADMISSIONS

1. Objectif du module

Ce module gère tout le processus entre **l'ouverture d'une campagne de recrutement/admission** et la transformation d'un candidat admis en **étudiant officiellement inscrit**.



Le module ne doit donc **pas créer directement un étudiant** lorsqu'une personne commence une candidature.

2. Acteurs

Acteur	Principales responsabilités
Candidat	Créer et suivre sa candidature
Secrétariat	Contrôler les dossiers
Agent admissions	Gérer campagnes et admissions
Commission	Examiner les candidatures
Responsable concours	Organiser les épreuves

Acteur	Principales responsabilités
Surveillant	Gérer les sessions d'épreuves
Enseignant/examineur	Évaluer certaines épreuves
Direction	Valider les décisions
Administrateur	Paramétrage et supervision
Système	Calculs, classements, notifications, documents

3. Sous-modules

Le module 02 doit être composé au minimum de :

02 – CANDIDATURES & ADMISSIONS

- 02.01 Campagnes d'admission
- 02.02 Candidats
- 02.03 Candidatures
- 02.04 Choix de formation
- 02.05 Dossiers
- 02.06 Pièces justificatives
- 02.07 Vérification des dossiers
- 02.08 Critères d'éligibilité
- 02.09 Concours / sélection
- 02.10 Épreuves
- 02.11 Centres d'examen
- 02.12 Salles d'examen
- 02.13 Sessions
- 02.14 Convocations
- 02.15 Surveillance
- 02.16 Évaluation des candidats
- 02.17 Résultats
- 02.18 Classement
- 02.19 Admission
- 02.20 Liste d'attente
- 02.21 Désistements
- 02.22 Recours / réclamations
- 02.23 Conversion candidat → étudiant
- 02.24 Documents
- 02.25 Notifications
- 02.26 Statistiques

4. Campagne d'admission

Une campagne représente une période officielle durant laquelle l'INJS accepte des candidatures.

Exemple

Campagne : Admission INJS 2026-2027
Année académique : 2026-2027
Date ouverture : configurable
Date clôture : configurable
Statut : OUVERTE

Informations

code
libellé
année académique
type de recrutement
date ouverture
date clôture
heure limite
formations ouvertes
nombre de places
mode de sélection
frais de candidature
statut
conditions générales
description

Statuts

BROUILLON
PLANIFIÉE
OUVERTE
SUSPENDUE
CLÔTURÉE
ANNULÉE
ARCHIVÉE

Une campagne clôturée ne doit plus accepter de nouvelles candidatures.

5. Candidat

Le **candidat** est une personne qui souhaite intégrer l'INJS.

Il est distinct de l'étudiant.

Identité

Numéro candidat
Nom
Prénoms
Date de naissance
Lieu de naissance
Sexe
Nationalité
Téléphone

Email
Adresse
Photo

Identifiants administratifs

Selon les besoins réglementaires :

CNI
Passeport
Carte consulaire
Autre pièce d'identité

Il faut éviter de créer plusieurs fiches pour la même personne.

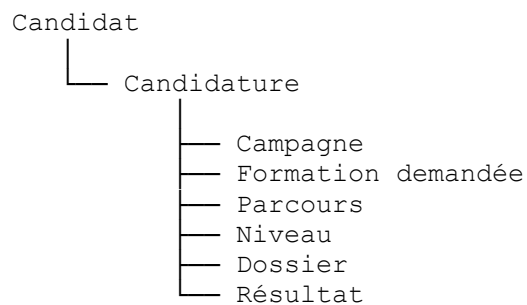
Le système doit rechercher les doublons sur des combinaisons configurables :

Nom + prénom + date de naissance
Email
Téléphone
Numéro de pièce

6. Candidature

Une personne peut avoir une ou plusieurs candidatures selon les règles de la campagne.

Structure :



Numéro de candidature

Le système génère automatiquement un identifiant métier :

INJS-2026-000001
INJS-2026-000002
INJS-2026-000003

Le format doit être configurable.

7. Choix de formation

Pour certaines campagnes, le candidat pourra sélectionner plusieurs choix.

Exemple :

Choix 1 → Formation A
Choix 2 → Formation B
Choix 3 → Formation C

Chaque choix doit comporter :

ordre
formation
parcours
niveau
priorité
statut

Mais le nombre maximum de choix doit être paramétrable par campagne.

8. Dossier de candidature

Le dossier rassemble toutes les informations et pièces nécessaires.

CANDIDATURE

- Informations personnelles
- Formation demandée
- Diplômes
- Expériences
- Documents
- Résultats
- Décision

États du dossier

BROUILLON
EN_COURS
SOUMIS
À_VÉRIFIER
INCOMPLET
COMPLET
IRRECEVABLE
ADMISSIBLE
NON_ADMISSIBLE

9. Pièces justificatives

Les pièces doivent être pilotées par le référentiel **Types de documents**.

Exemples possibles :

Pièce d'identité
Acte de naissance
Diplôme
Attestation de réussite
Relevé de notes
Photo d'identité
Certificat médical
Attestation professionnelle
Quittance
Autre document

Le système doit permettre de définir :

Document obligatoire
Document facultatif
Document conditionnel
Format accepté
Taille maximale
Date d'expiration

10. Contrôle du dossier

Le contrôleur doit pouvoir examiner chaque pièce.

Exemple :

Pièce	Statut
CNI	Conforme
Diplôme	Conforme
Relevé	À corriger
Photo	Conforme

Statuts :

NON_FOURNI
FOURNI
EN_EXAMEN
CONFORME
NON_CONFORME
À_REEMPLACER
EXPIRÉ

Le système conserve :

agent contrôleur
date
heure
ancienne décision
nouvelle décision
commentaire

11. Admissibilité

L'admissibilité doit être **configurable**, et non codée en dur.

Exemple :

Condition 1 : diplôme requis
Condition 2 : âge autorisé
Condition 3 : dossier complet
Condition 4 : pièce d'identité valide
Condition 5 : aptitude physique si applicable

Chaque critère peut être :

OBLIGATOIRE
OPTIONNEL
ÉLIMINATOIRE

Résultat :

ADMISSIBLE
NON_ADMISSIBLE
ADMISSIBILITÉ_SOUS_RÉSERVE

12. Concours / sélection

Le système doit pouvoir gérer plusieurs modes :

CONCOURS
SÉLECTION SUR DOSSIER
ENTRETIEN
TEST
CONCOURS + TEST
CONCOURS + ÉPREUVES PHYSIQUES
AUTRE

Il ne faut donc pas construire l'application autour d'un seul type de concours.

13. Épreuves

Une campagne peut comporter plusieurs épreuves.

Exemple générique :

Épreuve 1 – Écrit
Épreuve 2 – Oral
Épreuve 3 – Pratique
Épreuve 4 – Épreuve physique

Chaque épreuve possède :

code
libellé
type
coefficient
note maximale
durée
date
heure
espace
formation concernée
obligatoire
éliminatoire

14. Centres d'examen

Un centre peut être :

Site
Zone
Bâtiment
Espace

Le système doit réutiliser le **référentiel Campus & Patrimoine**.

Ainsi, on ne crée pas une nouvelle table indépendante pour les salles d'examen.

SITE
↓
ZONE
↓
BÂTIMENT
↓
ESPACE

15. Sessions d'épreuves

Une session représente une occurrence précise d'une épreuve.

Épreuve
↓

Session

↓

Date + Heure + Espace

Exemple :

Épreuve : Écrit

Date : configurable

Heure : 08:00

Salle : espace configuré

Capacité : 80

Candidats affectés : 72

Le système doit empêcher :

- dépassement de capacité ;
- double affectation d'un candidat ;
- conflit de salle ;
- conflit de surveillant ;
- conflit d'horaire.

16. Convocation

Pour chaque candidat concerné, le système peut générer automatiquement :

Convocation au concours

avec :

Nom

Prénoms

Numéro candidat

Formation

Centre

Bâtiment

Salle

Date

Heure

Épreuve

Consignes

QR code éventuel

Le document doit pouvoir être généré en PDF.

17. Présence au concours

À l'entrée :

Candidat
↓
Identification
↓
Convocation
↓
Contrôle
↓
Présent / Absent

Possibilités :

QR Code
Recherche numéro candidat
Liste papier
Validation manuelle

Statuts :

PRÉSENT
ABSENT
RETARD
EXCLU
NON_AUTORISÉ

18. Évaluation

Les résultats des épreuves doivent être séparés de la décision finale d'admission.

Candidat
↓
Épreuve
↓
Note
↓
Calcul
↓
Résultat concours
↓
Classement
↓
Admission

Une note doit comporter notamment :

candidat
épreuve
note
coefficient
évaluateur
date de saisie
statut
commentaire

19. Verrouillage des résultats

Cycle recommandé :

SAISIE
↓
BROUILLON
↓
SOUMIS
↓
CONTRÔLÉ
↓
VALIDÉ
↓
VERROUILLÉ

Après verrouillage :

l'enseignant ou l'agent ne peut plus modifier directement la note.

Toute correction doit générer :

ancienne valeur
nouvelle valeur
motif
auteur
date
validation

C'est essentiel pour la traçabilité.

20. Classement

Le classement doit être calculé automatiquement.

Exemple :

Note finale =
 $\sum(\text{note} \times \text{coefficient}) / \sum(\text{coefficients})$

Mais les règles doivent être paramétrables.

Le système doit gérer :

- classement décroissant ;
- seuil minimal ;
- épreuve éliminatoire ;
- nombre de places ;
- quotas éventuels ;

- égalités ;
 - départage ;
 - candidats ex æquo ;
 - candidats éliminés.
-

21. Liste principale et liste d'attente

Après classement :

1 → ADMIS
2 → ADMIS
3 → ADMIS
...
N → ADMIS

N+1 → LISTE D'ATTENTE
N+2 → LISTE D'ATTENTE
...

Une place libérée peut déclencher une proposition automatique au candidat suivant selon les règles définies.

22. Décision d'admission

La décision doit être formalisée.

Statuts :

ADMIS
ADMIS_SOUS_RÉSERVE
LISTE_ATTENTE
NON_ADMIS
ÉLIMINÉ
DÉSISTÉ

Informations :

candidat
campagne
formation
rang
score
décision
date
autorité décisionnelle
motif éventuel

23. Conversion Candidat → Étudiant

C'est une étape critique.

```
CANDIDAT
↓
ADMIS
↓
CONFIRMATION
↓
INSCRIPTION ADMINISTRATIVE
↓
DOSSIER ÉTUDIANT
↓
NUMÉRO ÉTUDIANT
```

Ne jamais transformer un candidat en étudiant simplement parce qu'il a été classé.

Il faut tenir compte de la confirmation et/ou de l'inscription effective selon les règles de l'INJS.

24. Numéro étudiant

Lors de l'inscription :

```
Candidat
↓
Étudiant
↓
Matricule étudiant
```

Le matricule doit être généré selon une règle configurable.

Exemple :

INJS26-00001

Le numéro de candidature reste néanmoins conservé dans l'historique.

25. Recours et réclamations

Le module doit prévoir :

```
Réclamation
Recours
Demande de correction
Contestation de note
```

Contestation d'admissibilité

Workflow :

Dépôt
↓
Enregistrement
↓
Instruction
↓
Décision
↓
Notification
↓
Clôture

Avec traçabilité complète.

26. Documents générés

Le module doit pouvoir produire automatiquement :

- ✓ Accusé de réception candidature
- ✓ Reçu de candidature
- ✓ Convocation
- ✓ Liste des candidats
- ✓ Liste des présents
- ✓ Feuille d'émargement
- ✓ Procès-verbal d'épreuve
- ✓ Résultats
- ✓ Liste des admis
- ✓ Liste d'attente
- ✓ Notification d'admission
- ✓ Notification de non-admission
- ✓ Attestation éventuelle

Les modèles doivent être configurables.

27. Notifications

Canaux possibles :

Notification application
Email
SMS

Événements :

Candidature enregistrée
Dossier incomplet
Pièce rejetée
Dossier valide
Convocation disponible
Modification de convocation
Résultats publiés
Admission
Liste d'attente
Non-admission
Recours accepté/refusé
Inscription ouverte

28. Tableau de bord Admissions

Le responsable doit avoir immédiatement :

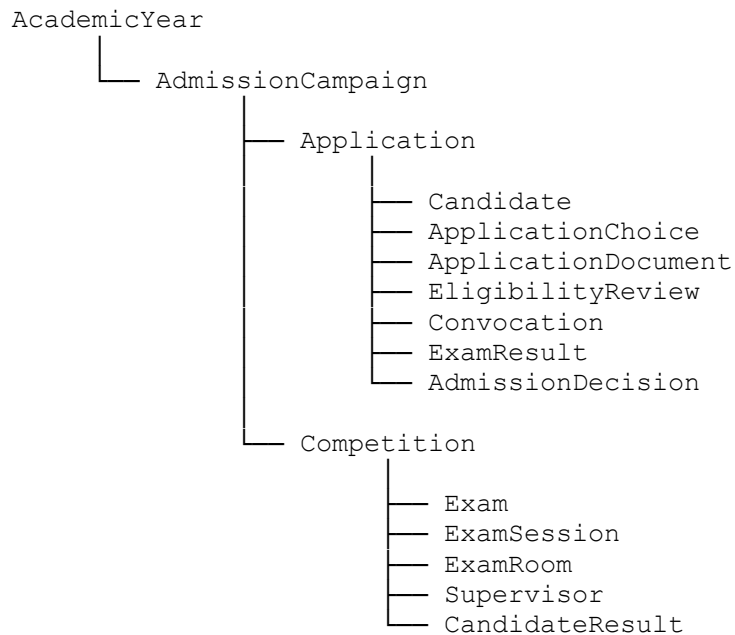
Campagnes actives
↓
Nombre de candidats
↓
Dossiers reçus
↓
Dossiers complets
↓
Dossiers incomplets
↓
Admissibles
↓
Candidats convoqués
↓
Présents
↓
Absents
↓
Résultats
↓
Admis
↓
Liste d'attente
↓
Inscrits

Indicateurs

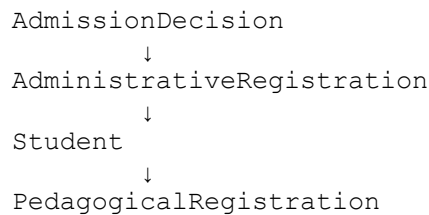
Nombre de candidatures
Taux de dossiers complets
Taux d'admissibilité
Taux de présence
Taux de réussite
Nombre d'admis
Nombre d'inscrits
Places disponibles
Places occupées

29. Modèle relationnel principal

La structure Django devra tendre vers :



Et :



30. Règles métier essentielles

Le système doit notamment empêcher :

Règle 1

Une candidature ne peut être soumise sans les informations obligatoires.

Règle 2

Une pièce obligatoire manquante empêche la validation du dossier.

Règle 3

Un candidat ne peut pas être affecté deux fois à la même session.

Règle 4

Une salle ne peut pas dépasser sa capacité.

Règle 5

Un surveillant ne peut pas être affecté à deux sessions simultanées.

Règle 6

Une note validée ne peut pas être modifiée sans procédure de correction.

Règle 7

Le classement doit être reproductible à partir des notes et règles enregistrées.

Règle 8

Une admission doit être liée à une campagne et à une formation.

Règle 9

Un candidat admis ne devient étudiant qu'au moment défini par le workflow d'inscription.

Règle 10

Toutes les décisions sensibles doivent être auditées.

31. Permissions

Le module devra respecter les niveaux de droits déjà prévus dans le projet :

N0 → Consultation très limitée

N1 → Consultation

N2 → Saisie / traitement

N3 → Validation

N4 → Administration / décision

Exemple :

Fonction	N0	N1	N2	N3	N4
Voir campagne	✓	✓	✓	✓	✓
Créer candidature	✓	✓	✓	✓	✓

Fonction	N0	N1	N2	N3	N4
Contrôler dossier	—	—	✓	✓	✓
Valider admissibilité	—	—	—	✓	✓
Saisir résultats	—	—	✓	✓	✓
Verrouiller résultats	—	—	—	✓	✓
Modifier règles concours	—	—	—	—	✓
Publier admission	—	—	—	✓	✓

Les droits réels devront naturellement être alignés avec le **Manuel utilisateur backend-INJS v3** et la matrice de permissions retenue.

32. Architecture API prévue

Les futures routes DRF pourront suivre cette logique :

```

/api/admissions/campaigns/
/api/admissions/candidates/
/api/admissions/applications/
/api/admissions/applications/{id}/documents/
/api/admissions/applications/{id}/eligibility/
/api/admissions/competitions/
/api/admissions/exams/
/api/admissions/sessions/
/api/admissions/convocations/
/api/admissions/results/
/api/admissions/rankings/
/api/admissions/decisions/
/api/admissions/waitlist/
/api/admissions/appeals/

```

Les identifiants techniques Django restent internes.

Les imports/exports et interfaces doivent utiliser les **codes et libellés métier**.

33. Architecture Flutter

Le menu pourrait devenir :

CANDIDATURES & ADMISSIONS

—	Tableau de bord
—	Campagnes
—	Candidats
—	Candidatures
—	Dossiers
—	Admissibilité
—	Concours
—	— Épreuves
—	— Sessions
—	— Centres
—	— Salles
—	— Surveillants
—	Convocations
—	Résultats
—	Classements
—	Admissions
—	Liste d'attente
—	Recours
—	Documents
—	Statistiques

34. Point particulièrement important pour

app-injs-lmd-2026

Le module 02 doit être **entièrement connecté aux référentiels du Module 01.**

Il ne faut surtout pas avoir :

Formation dans Admissions
 Formation dans Étudiants
 Formation dans Maquette
 Formation dans EDT

comme quatre référentiels indépendants.

Il doit exister **une seule Formation de référence**, utilisée partout :

Référentiels	
—	Admissions
—	Étudiants
—	Pédagogie
—	Enseignants
—	EDT
—	Notes
—	Jurys
—	Diplômes
—	Statistiques

C'est cette architecture qui permettra ensuite à Cline de développer les modules progressivement sans créer de doublons ni casser les relations.

Suite logique

Nous pouvons maintenant passer au **MODULE 03 — ÉTUDIANTS & SCOLARITÉ**, avec notamment :

dossier étudiant → matricule → inscription administrative → inscription pédagogique → formation/parcours/niveau → groupes → changement de formation → transfert → réinscription → redoublement → passage en année supérieure → abandon → suspension → radiation → historique académique.

Parfait. Nous passons au **MODULE 03 — ÉTUDIANTS & SCOLARITÉ**, qui constitue le cœur du système LMD après l'admission.

MODULE 03 — ÉTUDIANTS & SCOLARITÉ

1. Objectif

Ce module prend le relais du Module 02 dès qu'un candidat est effectivement admis et inscrit.

CANDIDAT
↓
ADMIS
↓
INSCRIPTION ADMINISTRATIVE
↓
ÉTUDIANT
↓
INSCRIPTION PÉDAGOGIQUE
↓
FORMATION / PARCOURS / NIVEAU
↓
GROUPE
↓
ENSEIGNEMENTS
↓
ÉVALUATIONS
↓
RÉSULTATS

Il constitue la **fiche académique centrale de chaque étudiant**.

2. Sous-modules

03 — ÉTUDIANTS & SCOLARITÉ

- 03.01 Étudiants
- 03.02 Dossiers étudiants
- 03.03 Matricules

—	03.04 Informations personnelles
—	03.05 Contacts
—	03.06 Responsables / personnes à contacter
—	03.07 Historique académique
—	03.08 Inscriptions administratives
—	03.09 Inscriptions pédagogiques
—	03.10 Formations
—	03.11 Parcours
—	03.12 Niveaux
—	03.13 Semestres
—	03.14 Groupes
—	03.15 Affectations aux groupes
—	03.16 Réinscriptions
—	03.17 Passage / progression
—	03.18 Redoublement
—	03.19 Suspension
—	03.20 Transfert
—	03.21 Changement de formation
—	03.22 Abandon
—	03.23 Radiation
—	03.24 Diplômes / fin de cycle
—	03.25 Documents étudiants
—	03.26 Attestations
—	03.27 Cartes étudiants
—	03.28 Statistiques

3. Fiche étudiant

Chaque étudiant dispose d'une fiche unique.

Identité

Matricule
Nom
Prénoms
Date de naissance
Lieu de naissance
Sexe
Nationalité
Photo

Coordonnées

Téléphone
Email
Adresse
Ville
Pays

Informations administratives

Date de création du dossier
Date première inscription
Origine de l'admission
Numéro de candidature

4. Statut étudiant

Le statut ne doit pas être simplement « actif/inactif ».

Prévoir notamment :

ADMIS
INSCRIT
ACTIF
EN_ATTENTE_INSCRIPTION
RÉINSCRIPTION
REDOUBLANT
EN_STAGE
SUSPENDU
EN_TRANSFERT
ABANDON
RADIÉ
DIPLÔMÉ
DÉCÉDÉ

Les valeurs exactes doivent être configurables dans les référentiels.

5. Matricule

Le matricule est **unique et permanent**.

Exemple :

INJS26-00001

Un changement de :

- formation ;
- parcours ;
- niveau ;
- groupe ;
- année académique

ne doit jamais modifier le matricule.

Le numéro de candidature reste également conservé dans l'historique.

6. Dossier étudiant

Le dossier doit regrouper :

Identité
↓
Admission
↓
Inscriptions
↓
Formation
↓
Parcours
↓
Niveaux
↓
Semestres
↓
Modules
↓
Notes
↓
Stages
↓
Présences
↓
Décisions de jury
↓
Diplôme

7. Inscription administrative

L'inscription administrative confirme officiellement l'inscription de l'étudiant pour une **année académique donnée**.

Structure :

Étudiant
+
Année académique
+
Formation
+
Statut
+
Situation financière

Exemple :

2026-2027
↓
Étudiant INJS26-00001
↓
Formation X
↓
Inscription administrative
↓
Validée

8. Statuts de l'inscription administrative

BROUILLON
EN_ATTENTE
DOSSIER_INCOMPLET
EN_ATTENTE_PAIEMENT
PARTIELLEMENT_PAYÉE
PAYÉE
VALIDÉE
ANNULÉE
SUSPENDUE

La validation définitive peut dépendre de plusieurs conditions.

9. Inscription pédagogique

Elle indique **ce que l'étudiant suit réellement**.

Étudiant
↓
Année académique
↓
Formation
↓
Parcours
↓
Niveau
↓
Semestre
↓
Modules / UE / ECUE
↓
Groupe

C'est cette inscription qui alimentera notamment :

- l'EDT ;
 - les listes de groupes ;
 - les présences ;
 - les évaluations ;
 - les notes ;
 - les jurys.
-

10. Formation / parcours / niveau

Le système doit reprendre intégralement les référentiels du Module 01.

FORMATION
↓
PARCOURS
↓
NIVEAU
↓
SEMESTRE
↓
MODULE

Il ne faut pas recopier ces données dans le module étudiants.

11. Groupes

Un étudiant peut être affecté à un groupe pédagogique.

Exemple :

Formation
↓
Licence
↓
Niveau
↓
L1
↓
Groupe L1-A

Un groupe possède :

Code
Libellé
Formation
Parcours
Niveau
Année académique
Effectif maximal
Responsable
Statut

12. Affectation étudiant → groupe

L'affectation doit être historisée.

Étudiant
↓
Groupe
↓
Date début

Date fin
Motif
Auteur

Ainsi, si un étudiant passe de :

Groupe A → Groupe B

l'ancien groupe reste dans l'historique.

13. Progression académique

Le système doit calculer automatiquement la situation de l'étudiant.

```
Semestre 1
  ↓
Validation
  ↓
Semestre 2
  ↓
Validation
  ↓
Année validée
  ↓
Passage
```

Situations possibles :

```
ADMIS
AJOURNÉ
ADMIS_AVEC_DETTEs
REDOUBLEMENT
PASSAGE_CONDITIONNEL
```

Les règles exactes doivent être paramétrables conformément aux règles académiques de l'INJS.

14. Redoublement

Un étudiant qui redouble ne doit pas être recréé.

```
Étudiant INJS26-00001
  ↓
Année 2026-2027
  ↓
L1
  ↓
Résultat insuffisant
  ↓
```

REDOUBLEMENT



Année 2027-2028



L1

Même matricule.

15. Dette académique

Le système doit pouvoir identifier les modules non validés.

Exemple :

L2



Module A ✓

Module B ✓

Module C ✗

Module D ✓

L'étudiant peut avoir :

Modules validés

Modules non validés

Crédits obtenus

Crédits restant à obtenir

Cette information sera indispensable pour le passage et le jury.

16. Changement de formation

Prévoir un workflow spécifique :

Demande



Étude



Validation



Nouvelle formation



Nouveau parcours éventuel



Nouvelle inscription pédagogique

Mais l'historique de l'ancienne formation reste conservé.

17. Transfert

Le système doit distinguer :

Transfert interne

Formation A → Formation B

Transfert externe

INJS → autre établissement

ou éventuellement :

autre établissement → INJS

avec conservation des justificatifs et décisions.

18. Suspension

Un étudiant peut temporairement interrompre son parcours.

ACTIF

↓

DEMANDE DE SUSPENSION

↓

VALIDATION

↓

SUSPENDU

↓

REPRISE

↓

ACTIF

Il faut enregistrer :

motif

date début

date fin prévue

date reprise

décision

autorité

document justificatif

19. Abandon

L'abandon doit être différent de la radiation.

ACTIF
↓
ABSENCE / ABANDON
↓
CONSTAT
↓
DÉCISION
↓
ABANDON

Il doit être possible de réactiver le dossier si les règles le permettent.

20. Radiation

La radiation est une décision administrative.

Elle doit obligatoirement être traçable :

motif
date
décision
autorité
document
commentaire

Une radiation ne doit pas supprimer physiquement les données historiques.

21. Historique académique

C'est une fonctionnalité majeure.

Pour chaque étudiant :

2026-2027
Formation X
L1
S1
S2
Résultats
Décision

2027-2028
Formation X
L2
S3
S4
Résultats

Décision

2028-2029
Formation X
L3
S5
S6
Résultats
Décision

2029
Jury final
Diplôme

22. Carte étudiant

Le système peut générer une carte avec :

Logo INJS
Photo
Nom
Prénoms
Matricule
Formation
Parcours
Niveau
Année académique
QR Code

Le QR code ne doit pas exposer directement les identifiants internes de la base.

Il doit pointer vers un identifiant/token public contrôlé.

23. Documents étudiants

Le dossier peut contenir :

Pièce d'identité
Acte de naissance
Diplômes antérieurs
Relevés
Certificats
Attestations
Documents administratifs
Décisions
Justificatifs

Avec :

type
date

version
statut
auteur
date d'ajout

24. Attestations

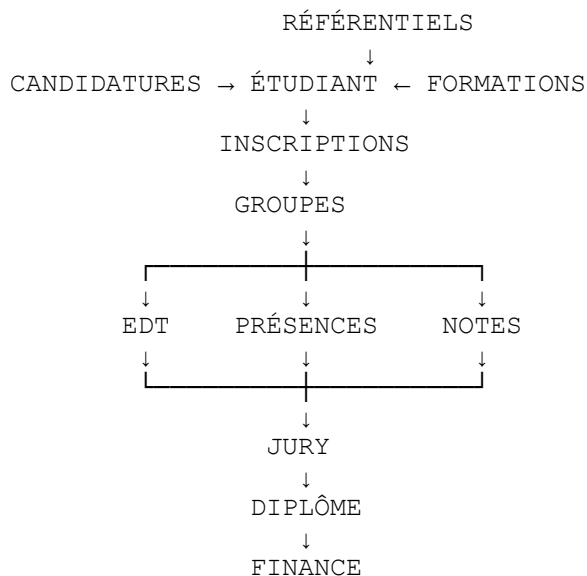
Le système pourra générer :

Attestation d'inscription
Attestation de scolarité
Attestation de fréquentation
Attestation de réussite
Relevé de notes
Certificat de scolarité

Les modèles doivent être paramétrables.

25. Connexion avec les autres modules

Le Module 03 devient le pivot :



26. Règles métier essentielles

Règle 1

Un étudiant possède un **matricule unique**.

Règle 2

Une personne ne doit pas posséder plusieurs fiches étudiant correspondant au même dossier.

Règle 3

Une inscription est toujours liée à une année académique.

Règle 4

Une inscription pédagogique doit être compatible avec la formation/parcours/niveau.

Règle 5

Un étudiant ne peut être simultanément actif dans deux groupes incompatibles.

Règle 6

Les changements de formation doivent être historisés.

Règle 7

Les redoublements ne créent pas un nouvel étudiant.

Règle 8

Une année académique clôturée ne doit plus être modifiable librement.

Règle 9

La radiation ne doit jamais supprimer les données historiques.

Règle 10

Les décisions académiques doivent être auditable.

27. API Django prévue

```
/api/students/  
/api/students/{id}/  
/api/students/{id}/history/  
/api/students/{id}/documents/  
/api/students/{id}/registrations/  
/api/students/{id}/academic-registrations/  
/api/students/{id}/groups/  
/api/students/{id}/results/  
/api/students/{id}/certificates/
```

/api/registrations/
/api/academic-registrations/
/api/groups/
/api/group-memberships/
/api/student-transfers/
/api/student-status-changes/
/api/student-withdrawals/
/api/student-suspensions/

28. Interface Flutter

Menu proposé :

ÉTUDIANTS & SCOLARITÉ

- └─ Tableau de bord
- └─ Étudiants
 - └─ Dossiers étudiants
 - └─ Inscriptions
 - └─ Administratives
 - └─ Pédagogiques
- └─ Groupes
- └─ Affectations
- └─ Progression académique
- └─ Redoublements
- └─ Transferts
- └─ Changements de formation
- └─ Suspensions
- └─ Abandons
- └─ Radiations
- └─ Documents
- └─ Attestations
- └─ Cartes étudiants
- └─ Statistiques

29. Tableau de bord Sclarité

Le responsable doit voir :

Étudiants actifs
Nouveaux étudiants
Réinscriptions
Inscriptions validées
Inscriptions en attente
Dossiers incomplets
Étudiants par formation
Étudiants par niveau
Étudiants par groupe
Redoublants
Étudiants suspendus
Abandons
Radiations
Diplômables

Avec filtres :

Année académique
Formation
Parcours
Niveau
Semestre
Groupe
Statut
Sexe

30. Modèle conceptuel central

La structure logique devient :

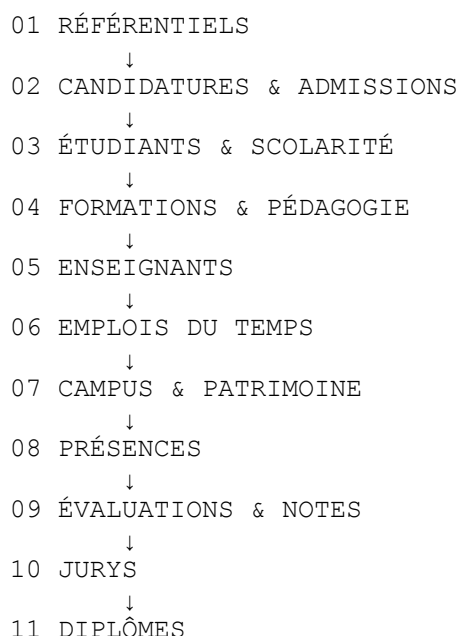
Candidate
↓
Admission
↓
Student
↓
AdministrativeRegistration
↓
AcademicRegistration
↓
StudentGroupMembership
↓
AcademicYear
↓
Formation
↓
Parcours
↓
Niveau
↓
Semestre
↓
Module

Puis :

AcademicRegistration
↓
Teaching
↓
Attendance
↓
Assessment
↓
Grade
↓
Result
↓
Jury
↓
Graduation

Point d'architecture essentiel

À ce stade, nous avons donc une chaîne cohérente :



La prochaine étape logique est le MODULE 04 — FORMATIONS & PÉDAGOGIE, où nous allons définir précisément la **maquette LMD**, UE/ECUE/modules, crédits, coefficients, CM/TD/TP/stages, volumes horaires, groupes pédagogiques, affectations d'enseignements et règles de progression.

Parfait. Nous poursuivons avec le **MODULE 04 — FORMATIONS & PÉDAGOGIE**, qui est le cœur académique du système LMD de l'INJS.

MODULE 04 — FORMATIONS & PÉDAGOGIE

1. Objectif

Ce module décrit **ce que l'INJS enseigne**, à qui, à quel niveau, pendant quelle période, avec quels volumes horaires, quels crédits, quels coefficients et selon quelle organisation pédagogique.

Il constitue la source de vérité pour :

- les formations ;
- les parcours ;

- les niveaux ;
- les semestres ;
- les UE ;
- les ECUE/modules ;
- les crédits ECTS ;
- les coefficients ;
- les CM/TD/TP/stages ;
- les groupes ;
- les enseignements ;
- les volumes horaires enseignants ;
- l'EDT ;
- les évaluations ;
- les jurys.

2. Architecture pédagogique

La structure recommandée est :

```

ANNÉE ACADEMIQUE
  ↓
FORMATION
  ↓
PARCOURS
  ↓
NIVEAU
  ↓
SEMESTRE
  ↓
UE
  ↓
ECUE / MODULE
  ↓
ENSEIGNEMENT
  ↓
GROUPE
  ↓
SÉANCE
  
```

Exemple :

```

2026-2027
  ↓
Licence
  ↓
Éducation et Motricité
  ↓
L1
  ↓
Semestre 1
  ↓
UE
  ↓
Module
  
```

3. Sous-modules

04 – FORMATIONS & PÉDAGOGIE

—	04.01 Formations
—	04.02 Parcours
—	04.03 Diplômes
—	04.04 Niveaux
—	04.05 Semestres
—	04.06 UE
—	04.07 ECUE
—	04.08 Modules
—	04.09 Maquettes
—	04.10 Versions de maquettes
—	04.11 Crédits
—	04.12 Coefficients
—	04.13 Volumes horaires
—	04.14 CM
—	04.15 TD
—	04.16 TP
—	04.17 Stages
—	04.18 Enseignements
—	04.19 Groupes pédagogiques
—	04.20 Affectations enseignants
—	04.21 Progression pédagogique
—	04.22 Calendrier académique
—	04.23 Statistiques pédagogiques

4. Formation

Une formation représente un cursus proposé par l'INJS.

Elle doit comporter :

Code
Libellé
Type de diplôme
Domaine
Mention
Durée
Nombre de semestres
Nombre de crédits
Statut
Date début
Date fin éventuelle
Description

Exemples de types :

Licence

Master
Formation professionnelle
Professorat
Autre

⚠ Les intitulés exacts des formations réellement ouvertes à une année donnée doivent provenir du référentiel officiel INJS et rester configurables.

5. Parcours

Une formation peut comporter plusieurs parcours.

Formation
↓
Parcours

Exemple conceptuel :

Formation : Licence
├─ Parcours A
├─ Parcours B
└─ Parcours C

Informations :

Code
Libellé
Formation
Domaine
Spécialité
Durée
Statut

6. Niveaux

Le système doit gérer au minimum une structure LMD :

Licence
├─ L1
├─ L2
└─ L3

Master
├─ M1
└─ M2

Mais les niveaux doivent rester configurables pour les autres formations de l'INJS.

7. Semestres

Chaque niveau est organisé en semestres.

L1
├── S1
└── S2

L2
├── S3
└── S4

L3
├── S5
└── S6

Un semestre possède :

Code
Libellé
Niveau
Ordre
Nombre de crédits
Période
Statut

8. UE — Unités d’enseignement

La maquette doit pouvoir regrouper les enseignements en UE.

SEMESTRE
↓
UE
↓
ECUE / MODULE

Une UE possède :

Code
Libellé
Semestre
Crédits
Coefficient
Type
Obligatoire / optionnelle

9. ECUE / Module

Le module doit être un objet pédagogique central.

Informations :

Code
Libellé
Description
UE
Semestre
Crédits
Coefficient
CM
TD
TP
Stage
VH total
Type d'évaluation
Statut

10. Volumes horaires

Pour respecter la structure déjà retenue dans le projet :

CM
TD
TP
STAGE

Le **VH total** est calculé :

$$\text{VH} = \text{CM} + \text{TD} + \text{TP} + \text{STAGE}$$

Sauf règles particulières configurées dans la maquette.

11. Exemple de module

Module :
Introduction à une discipline

CM :
20 h

TD :
10 h

TP :
10 h

Stage :
0 h

VH :
40 h

Crédits :
configurable

Coefficient :
configurable

Cet exemple est uniquement structurel : les valeurs officielles devront être saisies dans les maquettes INJS.

12. Maquette pédagogique

La **maquette** est l'un des objets les plus importants du système.

Elle définit le cursus officiel pour une période donnée.

FORMATION
↓
PARCOURS
↓
NIVEAU
↓
SEMESTRE
↓
UE
↓
MODULE
↓
Crédits
Coefficient
CM
TD
TP
Stage

13. Versionnement des maquettes

Il ne faut surtout pas modifier directement une maquette historique.

Exemple :

Maquette Licence X

Version 1
2025-2026
↓
Version 2
2026-2027
↓
Version 3
2027-2028

Une modification importante crée une nouvelle version.

Ainsi, les étudiants de 2025-2026 continuent d'être évalués selon **leur maquette**, même si celle-ci évolue ensuite.

14. États de la maquette

BROUILLON
EN RÉVISION
SOUMISE
VALIDÉE
PUBLIÉE
ARCHIVÉE

Une maquette publiée ne doit plus être modifiable librement.

15. Calcul du volume horaire

Pour chaque module :

$$VH = CM + TD + TP + STAGE$$

Pour une UE :

$$VH_{UE} = \sum VH \text{ des modules}$$

Pour un semestre :

$$VH_{\text{semestre}} = \sum VH \text{ des UE}$$

Pour un parcours :

$$VH_{\text{parcours}} = \sum VH \text{ des semestres}$$

Cela permettra d'identifier automatiquement :

- sous-volume ;
 - dépassement ;
 - module incomplet ;
 - volume non affecté ;
 - volume affecté mais non planifié.
-

16. Enseignement

Il faut distinguer **le module** de l'enseignement réellement dispensé.

Exemple :

```
Module
  ↓
Enseignement
  ↓
Type : CM
  ↓
Groupe
  ↓
Enseignant
  ↓
Volume prévu
  ↓
EDT
```

Ainsi un même module peut avoir :

```
CM → Enseignant A
TD → Enseignant B
TD → Enseignant C
TP → Enseignant D
```

17. Affectation enseignant

Une affectation pédagogique doit contenir :

```
Année académique
Formation
Parcours
Niveau
Semestre
Module
Type d'enseignement
Enseignant
Groupe
VH prévue
Date début
Date fin
Statut
```

18. CM / TD / TP

Le système doit différencier les types d'enseignement :

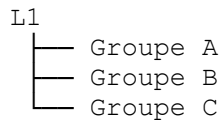
```
CM → Cours magistral
TD → Travaux dirigés
TP → Travaux pratiques
STAGE → Stage
```


Cette information alimentera directement le moteur EDT.

19. Groupes pédagogiques

Les groupes sont créés à partir des besoins réels.

Exemple :



Un module peut être enseigné :

Groupe A → TD
Groupe B → TD
Groupe C → TD

avec éventuellement des enseignants différents.

20. Décomposition du volume

Exemple :

Module X
VH = 60 h

CM : 20 h
TD : 20 h
TP : 20 h

Si 3 groupes suivent le TD :

$20 \text{ h} \times 3 \text{ groupes} = 60 \text{ h TD à assurer}$

Cette distinction est essentielle pour calculer correctement la **charge réelle des enseignants**.

21. Charge enseignant

Le système doit distinguer :

VH MAQUETTE
↓

VH À ASSURER
↓
VH AFFECTÉ
↓
VH PLANIFIÉ
↓
VH RÉALISÉ

Exemple :

Maquette : 20 h CM
Affecté : 20 h
Planifié : 18 h
Réalisé : 16 h

Le système doit détecter automatiquement l'écart.

22. Contrôle de cohérence pédagogique

Avant publication d'une maquette, le système doit contrôler :

Structure

Formation ✓
Parcours ✓
Niveau ✓
Semestre ✓
UE ✓
Module ✓

Volumes

CM ≥ 0
TD ≥ 0
TP ≥ 0
Stage ≥ 0

Crédits

Σ crédits modules = crédits attendus

Coefficients

Σ coefficients = règle de la maquette

Les règles exactes doivent être configurables.

23. Connexion avec l'EDT

Le Module 04 fournit les besoins.

Le Module 06 les transforme en calendrier.

```
MAQUETTE
↓
ENSEIGNEMENTS
↓
VH À PLANIFIER
↓
ENSEIGNANT
↓
GROUPE
↓
ESPACE
↓
CRÉNEAU
↓
EDT
```

Ainsi, l'EDT ne doit pas être construit manuellement sans référence à la maquette.

24. Connexion avec les évaluations

Chaque module peut définir son mode d'évaluation :

```
Contrôle continu
Contrôle terminal
CC + CT
Examen
Projet
Stage
Oral
Pratique
```

Les pondérations doivent être paramétrables.

Dans le cadre déjà défini pour le projet, le système pourra notamment gérer :

```
CC = 40 %
CT = 60 %
```

mais cette règle ne doit pas être codée définitivement si certaines formations utilisent d'autres pondérations.

25. Connexion avec les jurys

Le jury doit pouvoir récupérer :

Étudiant
↓
Inscription pédagogique
↓
Modules
↓
Notes
↓
Crédits
↓
Résultats
↓
Décision

Le jury ne doit donc pas recalculer manuellement les données de la maquette.

26. Calendrier pédagogique

Prévoir :

Année académique
↓
Semestre
↓
Période pédagogique
↓
Cours
↓
Évaluations
↓
Examens
↓
Jury

Événements :

Début des cours
Fin des cours
Période de TD
Période de TP
Examens
Rattrapages
Stages
Jury
Vacances
Fermeture

27. API Django

Structure recommandée :

```
/api/academics/formations/  
/api/academics/parcours/  
/api/academics/niveaux/  
/api/academics/semestres/  
  
/api/academics/ues/  
/api/academics/modules/  
  
/api/academics/maquettes/  
/api/academics/maquettes/{id}/versions/  
/api/academics/maquettes/{id}/validate/  
/api/academics/maquettes/{id}/publish/  
  
/api/academics/enseignements/  
/api/academics/affectations/  
/api/academics/groupes/  
  
/api/academics/volumes/  
/api/academics/workloads/
```

28. Interface Flutter

FORMATIONS & PÉDAGOGIE

```
— Tableau de bord  
— Formations  
— Parcours  
— Niveaux  
— Semestres  
— UE  
— Modules  
— Maquettes  
  — Liste  
  — Éditeur  
  — Versions  
  — Contrôles  
  — Validation  
— Volumes horaires  
— Enseignements  
— Affectations enseignants  
— Groupes pédagogiques  
— Calendrier académique  
— Statistiques
```

29. Tableau de bord pédagogique

Le responsable pédagogique doit pouvoir consulter :

Formations actives
Parcours
Niveaux
Étudiants
Modules
UE
Maquettes publiées

Maquettes en attente
VH total
VH affecté
VH planifié
VH réalisé
Enseignants affectés
Groupes

Avec alertes :

- ⚠ Module sans enseignant
 - ⚠ VH non affecté
 - ⚠ VH non planifié
 - ⚠ Conflit pédagogique
 - ⚠ Maquette incomplète
 - ⚠ Groupe sans enseignement
 - ⚠ Module sans évaluation
-

30. Modèle conceptuel central

Le modèle académique devient :

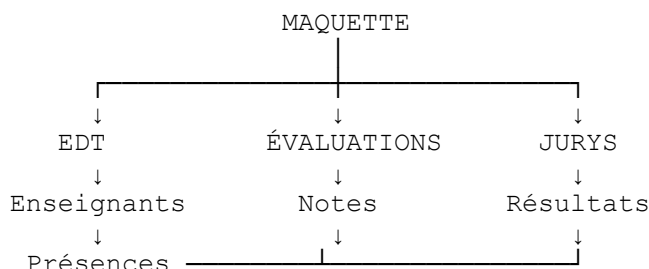
AcademicYear
↓
Formation
↓
Parcours
↓
Niveau
↓
Semestre
↓
UE
↓
Module
↓
Teaching
↓
TeacherAssignment
↓
StudentGroup
↓
TimetableSession

Et parallèlement :

Module
↓
Assessment
↓
Grade
↓
Result
↓
Jury

31. Principe fondamental pour l'application

Il faut maintenant considérer la **maquette pédagogique** comme la source de vérité académique.



Cela évitera les incohérences classiques d'un système universitaire où :

- l'EDT indique un module qui n'existe plus ;
- un enseignant est affecté à un module absent de la maquette ;
- un étudiant est évalué dans un module auquel il n'est pas inscrit ;
- les volumes horaires ne correspondent pas aux enseignements réellement planifiés ;
- le jury utilise une ancienne version de la maquette.

Prochaine étape : MODULE 05 — ENSEIGNANTS

Nous définirons alors de manière opérationnelle :

personnel enseignant → profil → grade → statut → spécialités → formations → modules → CM/TD/TP → groupes → affectations → charge horaire → EDT → séances → présences → évaluations → saisie des notes → validation → historique des charges.

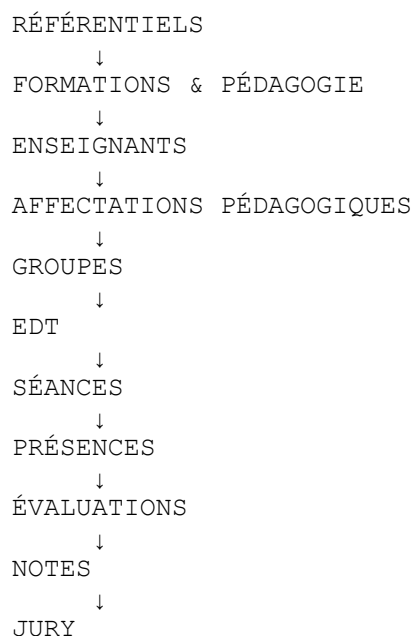
Cela permettra ensuite de construire proprement le **Module 06 — EMPLOIS DU TEMPS**, qui sera directement alimenté par les modules 04 et 05.

Très bien. Nous passons au **MODULE 05 — ENSEIGNANTS & CHARGE PÉDAGOGIQUE**.

Ce module doit être conçu pour que l'INJS puisse gérer non seulement les fiches des enseignants, mais surtout **qui enseigne quoi, à quelle formation, à quel groupe, avec quel volume horaire et sur quelle période.**

MODULE 05 — ENSEIGNANTS & CHARGE PÉDAGOGIQUE

1. Position dans le système



L'enseignant est donc relié simultanément à plusieurs modules.

2. Sous-modules

05 — ENSEIGNANTS

- 05.01 Fiches enseignants
- 05.02 Identité
- 05.03 Coordonnées
- 05.04 Statuts
- 05.05 Grades
- 05.06 Fonctions
- 05.07 Spécialités
- 05.08 Services / affectations administratives
- 05.09 Formations enseignées
- 05.10 Modules enseignés
- 05.11 Types d'enseignement
- 05.12 Groupes enseignés
- 05.13 Affectations pédagogiques
- 05.14 Charge horaire
- 05.15 Disponibilités
- 05.16 Indisponibilités
- 05.17 EDT enseignant
- 05.18 Séances réalisées
- 05.19 Évaluations

—	05.20	Saisie des notes
—	05.21	Encadrement
—	05.22	Stages
—	05.23	Historique
—	05.24	Statistiques

3. Fiche enseignant

Une fiche unique doit être créée pour chaque enseignant.

Identité

Matricule enseignant
Nom
Prénoms
Date de naissance
Sexe
Photo

Coordonnées

Téléphone
Email professionnel
Email secondaire
Adresse

Informations professionnelles

Grade
Statut
Fonction
Service
Direction
Spécialité
Date de recrutement
Date d'affectation
Statut actif/inactif

4. Ne pas confondre grade et fonction

Le système doit distinguer :

GRADE
≠
FONCTION
≠
STATUT

Par exemple, un enseignant peut avoir :

Grade → X

Fonction → Responsable pédagogique
Statut → Actif

Cette séparation est importante pour le référentiel RH.

5. Spécialités

Un enseignant peut posséder plusieurs spécialités.

ENSEIGNANT

├─ Spécialité A
├─ Spécialité B
└─ Spécialité C

Cela permettra plus tard d'aider à l'affectation automatique.

Exemple :

Spécialité :
Biomécanique

Modules compatibles :
Biomécanique
Physiologie
Analyse du mouvement

La compatibilité doit cependant rester configurable et validable par le responsable pédagogique.

6. Affectation administrative

Il faut distinguer :

Affectation administrative

Enseignant
↓
Direction
↓
Service
↓
Fonction

et :

Affectation pédagogique

Enseignant

↓
Formation
↓
Parcours
↓
Niveau
↓
Semestre
↓
Module
↓
Type d'enseignement
↓
Groupe

Un enseignant peut donc être administrativement affecté à un service tout en intervenant pédagogiquement dans plusieurs formations.

7. Affectation pédagogique

Objet central du module :

TeacherAssignment

Informations :

Année académique
Formation
Parcours
Niveau
Semestre
UE
Module
Type d'enseignement
Groupe
Enseignant
VH prévue
Date début
Date fin
Statut

8. Exemple

Enseignant : Enseignant A

Formation : Formation X
Niveau : L1
Semestre : S1
Module : Module X

CM : 20 h
TD : 10 h

TP : 0 h

Groupe : L1-A

Une autre affectation peut être :

Enseignant A
Module X
TD
Groupe L1-B
10 h

9. Charge horaire

Il faut absolument distinguer plusieurs notions.

VH MAQUETTE
↓
VH À ASSURER
↓
VH AFFECTÉ
↓
VH PLANIFIÉ
↓
VH RÉALISÉ

Exemple :

Maquette	30 h
À assurer	30 h
Affecté	30 h
Planifié	28 h
Réalisé	26 h

Le système doit afficher automatiquement les écarts.

10. Charge par enseignant

Tableau :

Enseignant	CM	TD	TP	Stage	Total
Enseignant A	20	30	10	0	60
Enseignant B	10	20	20	5	55

Les valeurs sont naturellement des exemples de structure.

11. Charge par formation

Le responsable doit également pouvoir afficher :

```
Formation X
  ↓
VH prévue
  ↓
VH affectée
  ↓
VH planifiée
  ↓
VH réalisée
```

Et détecter :

```
Module sans enseignant
VH non affectée
VH insuffisamment affectée
VH excédentaire
```

12. Charge par module

Exemple :

MODULE X

Maquette :

CM 20 h

TD 20 h

TP 10 h

Total = 50 h

Affectation :

CM → Enseignant A : 20 h

TD → Enseignant B : 20 h

TP → Enseignant C : 10 h

Couverture = 100 %

Le système doit pouvoir calculer automatiquement le taux de couverture.

13. Disponibilités

Pour l'EDT, chaque enseignant doit pouvoir déclarer ou recevoir des créneaux de disponibilité.

Lundi
08:00-10:00 disponible
10:00-12:00 disponible
14:00-16:00 indisponible

Cette information doit être exploitée par le moteur EDT.

14. Indisponibilités

Types :

Congé
Mission
Réunion
Formation
Absence
Maladie
Autre

Il faut distinguer :

INDISPONIBILITÉ PLANIFIÉE
INDISPONIBILITÉ EXCEPTIONNELLE

Une indisponibilité enregistrée doit pouvoir déclencher une détection des séances impactées.

15. Conflits enseignants

Le système doit interdire :

Enseignant A
↓
Lundi 10h-12h
↓
Cours X

et simultanément :

Enseignant A
↓
Lundi 10h-12h
↓
Cours Y

Un enseignant ne peut pas avoir deux séances simultanées.

16. Encadrement

Pour certaines activités, l'enseignant peut être :

Formateur
Enseignant
Encadrant
Responsable
Examineur
Surveillant
Jury

Ces rôles doivent être séparés.

Exemple :

Module
↓
Enseignant responsable
↓
Formateurs
↓
Encadrants

17. Évaluations

Un enseignant peut être associé à :

Évaluation
↓
Épreuve
↓
Groupe
↓
Étudiants

Il peut ensuite :

Saisir notes
Modifier brouillon
Soumettre
Consulter

selon ses permissions.

18. Validation des notes

Workflow :

ENSEIGNANT
↓
SAISIE
↓
BROUILLON
↓
SOUMISSION
↓
CONTRÔLE
↓
VALIDATION
↓
JURY
↓
VERROUILLAGE

Après verrouillage :

Modification directe → INTERDITE

Une correction nécessite une procédure contrôlée.

19. Enseignant responsable de module

Chaque module peut éventuellement avoir :

Responsable pédagogique

Mais il faut permettre plusieurs enseignants intervenants.

MODULE X
|
— Responsable → A
— CM → A
— TD → B
— TD → C
— TP → D

20. Stages

Le module doit pouvoir gérer l'encadrement des stages :

Étudiant
↓
Stage
↓
Structure d'accueil
↓
Encadrant INJS
↓
Tuteur

↓
Évaluation

Cela sera connecté au module des stages si celui-ci est développé plus finement.

21. Tableau de bord enseignant

Dans Flutter :

MON ESPACE ENSEIGNANT

- Tableau de bord
 - Mon profil
 - Mes formations
 - Mes modules
 - Mes groupes
 - Mes affectations
 - Ma charge horaire
 - Mes disponibilités
 - Mon EDT
 - Mes séances
 - Présences
 - Évaluations
 - Saisie des notes
 - Mes stages
 - Documents
 - Notifications
-

22. Tableau de bord responsable pédagogique

ENSEIGNANTS

Nombre total
Enseignants actifs
Enseignants indisponibles
Enseignants par grade
Enseignants par service

Pédagogie

Modules affectés
Modules non affectés
VH totale
VH affectée
VH planifiée
VH réalisée
Taux de couverture

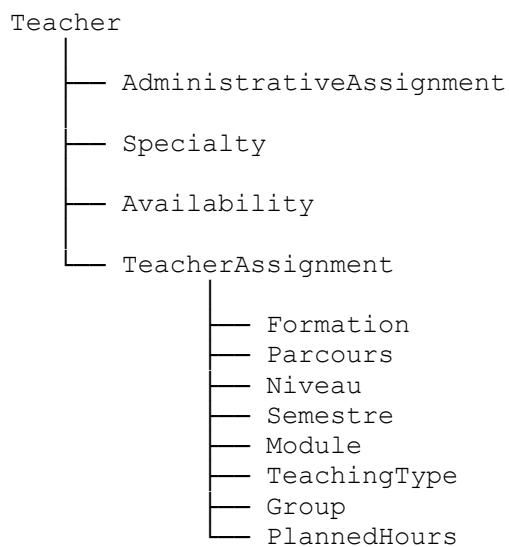
Alertes

- ⚠ Enseignant surchargé
 - ⚠ Module sans enseignant
 - ⚠ Conflit horaire
 - ⚠ VH non couverte
 - ⚠ Enseignant sans groupe
 - ⚠ Séance non réalisée
-

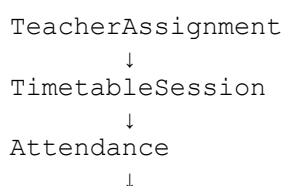
23. API Django

```
/api/teachers/  
/api/teachers/{id}/  
/api/teachers/{id}/assignments/  
/api/teachers/{id}/workload/  
/api/teachers/{id}/availability/  
/api/teachers/{id}/unavailability/  
/api/teachers/{id}/timetable/  
/api/teachers/{id}/sessions/  
/api/teachers/{id}/assessments/  
  
/api/teacher-assignments/  
/api/teacher-workloads/  
/api/teacher-availability/  
/api/teacher-unavailability/
```

24. Modèle conceptuel



Puis :



Assessment

↓

Grade

↓

Jury

25. Contrôles automatiques indispensables

Avant validation d'une affectation :

Enseignant

- ✓ actif
- ✓ disponible
- ✓ compétence compatible

Module

- ✓ existe
- ✓ appartient à la maquette
- ✓ année correcte

Groupe

- ✓ existe
- ✓ compatible avec formation/niveau

Volume

- ✓ $VH > 0$
- ✓ ne dépasse pas les besoins
- ✓ couverture calculée

EDT

- ✓ pas de conflit enseignant
 - ✓ pas de conflit groupe
 - ✓ pas de conflit espace
-

26. Relation avec les modules déjà définis

Le système global devient maintenant :

01 RÉFÉRENTIELS

↓

02 ADMISSIONS

↓

03 ÉTUDIANTS
 ↓
 04 FORMATIONS & PÉDAGOGIE
 ↓
 05 ENSEIGNANTS
 ↓
 06 EMPLOIS DU TEMPS
 ↓
 07 CAMPUS & PATRIMOINE
 ↓
 08 PRÉSENCES
 ↓
 09 ÉVALUATIONS & NOTES
 ↓
 10 JURYS
 ↓
 11 DIPLÔMES

Avec deux flux transversaux :

FINANCES —————→ INSCRIPTIONS
 FINANCES —————→ ADMINISTRATION

et :

ADMINISTRATION
 ↓
 ENSEIGNANTS
 ↓
 FORMATIONS
 ↓
 ÉTUDIANTS

27. Principe clé avant le Module 06

Nous avons maintenant suffisamment d'éléments pour construire le futur **moteur automatique d'EDT**.

Il devra prendre comme entrées :

FORMATION
 PARCOURS
 NIVEAU
 SEMESTRE
 MODULE
 CM / TD / TP / STAGE
 VH
 ENSEIGNANT
 GROUPE
 EFFECTIF
 ESPACE
 CAPACITÉ
 DISPONIBILITÉ ENSEIGNANT
 DISPONIBILITÉ ESPACE
 CALENDRIER

et produire :

JOUR
HEURE DÉBUT
HEURE FIN
ENSEIGNANT
FORMATION
GROUPE
MODULE
TYPE D'ENSEIGNEMENT
ESPACE

avec contrôle automatique des conflits.

Le prochain module sera donc le MODULE 06 — EMPLOIS DU TEMPS & PLANIFICATION, où nous définirons le moteur de génération automatique de l'EDT, les créneaux, contraintes, salles, groupes, enseignants, CM/TD/TP, examens, réservations, conflits, remplacement d'enseignants et gestion des espaces indisponibles.

Très bien. Nous passons au **MODULE 06 — EMPLOIS DU TEMPS & PLANIFICATION ACADÉMIQUE**, qui doit être conçu comme un véritable **moteur de planification**, et non comme un simple calendrier.

MODULE 06 — EMPLOIS DU TEMPS & PLANIFICATION

1. Objectif

Le module doit transformer les besoins pédagogiques définis dans les modules 04 et 05 en un planning exploitable :

MAQUETTE
↓
MODULES
↓
VOLUMES HORAIRES
↓
ENSEIGNEMENTS
↓
ENSEIGNANTS
↓
GROUPE
↓
ESPACES
↓
CRÉNEAUX
↓
CONTRAINTES
↓
MOTEUR DE PLANIFICATION
↓

EMPLOI DU TEMPS

↓

SÉANCES

↓

PRÉSENCES

L'objectif est que le système puisse répondre automatiquement à :

Qui enseigne quoi, à quel groupe, quand, pendant combien de temps et dans quel espace ?

2. Sous-modules

06 — EMPLOIS DU TEMPS & PLANIFICATION

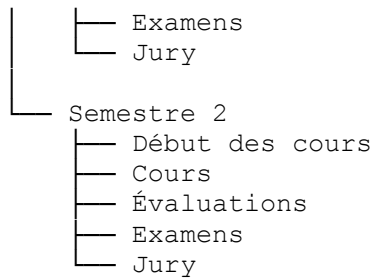
- 06.01 Calendrier académique
 - 06.02 Périodes pédagogiques
 - 06.03 Créneaux horaires
 - 06.04 Jours ouvrables
 - 06.05 Semaines pédagogiques
 - 06.06 Enseignements à planifier
 - 06.07 Groupes
 - 06.08 Enseignants
 - 06.09 Espaces
 - 06.10 Disponibilités
 - 06.11 Indisponibilités
 - 06.12 Contraintes
 - 06.13 Réservations
 - 06.14 Génération automatique
 - 06.15 Génération manuelle
 - 06.16 Optimisation
 - 06.17 Détection des conflits
 - 06.18 Validation
 - 06.19 Publication
 - 06.20 Modifications
 - 06.21 Remplacements
 - 06.22 Annulations
 - 06.23 Rattrapages
 - 06.24 Examens
 - 06.25 Historique
 - 06.26 Statistiques
-

3. Calendrier académique

Le calendrier constitue la première contrainte.

2026-2027

- Semestre 1
 - Début des cours
 - Cours
 - Évaluations



Le système doit gérer :

- périodes de cours ;
- vacances ;
- jours fériés ;
- examens ;
- stages ;
- interruptions ;
- rattrapages ;
- périodes administratives.

4. Créneaux horaires

Les créneaux doivent être configurables.

Exemple :

08:00 - 10:00
10:00 - 12:00
13:00 - 15:00
15:00 - 17:00
17:00 - 19:00

Mais il ne faut pas imposer ces créneaux dans le code.

Un cours peut également avoir :

08:00 - 09:00
09:00 - 10:30
14:30 - 17:30

selon les besoins.

5. Unité de planification : la séance

Il faut distinguer :

enseignement et séance.

Exemple :

MODULE : Biomécanique
↓
ENSEIGNEMENT : CM
↓
20 heures
↓
10 séances × 2 heures

Chaque séance devient un objet indépendant.

Séance
├── Date
├── Heure début
├── Heure fin
├── Module
├── Type
├── Enseignant
├── Groupe
├── Espace
└── Statut

6. Enseignement à planifier

Le moteur récupère les données du Module 04 :

Module
Type : TD
VH : 20 h

puis les affectations du Module 05 :

Enseignant : X
Groupe : L1-A

Le moteur doit ensuite déterminer :

20 h
↓
créneaux
↓
dates
↓
salle

7. Espaces de formation

Le moteur doit exploiter le référentiel physique :

SITE
↓
ZONE
↓
BÂTIMENT
↓
ESPACE

Un espace peut être :

Salle de cours
Salle TD
Salle TP
Amphithéâtre
Laboratoire
Gymnase
Terrain
Piscine
Piste d'athlétisme
Stade
Salle spécialisée
Espace polyvalent

8. Caractéristiques des espaces

Un simple champ *capacité* ne suffit pas.

Chaque espace doit pouvoir posséder des caractéristiques :

Capacité
Surface
Type
Équipements
Accessibilité
Matériel audiovisuel
Ordinateurs
Connexion Internet
Matériel scientifique
Matériel sportif
Vestiaires
Gradins
Éclairage
Sol spécialisé

9. Compatibilité espace ↔ enseignement

Le système doit pouvoir déterminer qu'un espace est compatible avec une activité.

Exemple :

TP de laboratoire
↓
Laboratoire compatible

✓
Salle de cours classique
X

Autre exemple :

Activité sportive
↓
Terrain / gymnase / piscine

Cette compatibilité doit être configurable.

10. Capacité

Le moteur doit contrôler :

Effectif groupe $\leq$ capacité espace

Exemple :

Groupe : 65 étudiants
Salle : capacité 40

Résultat :

✗ ESPACE INCOMPATIBLE

Il peut éventuellement proposer :

Salle 80 places
Amphithéâtre
Division du groupe

selon les règles autorisées.

11. Groupes

Le moteur doit connaître :

Groupe
Formation
Parcours
Niveau
Effectif

Exemple :

L1-A → 45 étudiants
L1-B → 43 étudiants
L1-C → 47 étudiants

Un même cours peut nécessiter :

CM → tous les groupes réunis
TD → groupes séparés
TP → sous-groupes

12. Enseignants

Le moteur doit utiliser :

Enseignant
Affectation
Type d'enseignement
Disponibilité
Indisponibilité
Charge

Il doit empêcher :

Enseignant A
10:00-12:00
Cours 1

et :

Enseignant A
10:00-12:00
Cours 2

13. Conflit groupe

Même logique :

Groupe L1-A
10:00-12:00
Module A

ne peut pas simultanément avoir :

Groupe L1-A
10:00-12:00
Module B

14. Conflit espace

Une salle ne peut pas accueillir deux séances simultanément.

Salle S-101
10:00-12:00
Cours A

et :

Salle S-101
10:00-12:00
Cours B

→ **conflit bloquant.**

15. Contraintes dures et souples

C'est un point majeur pour le moteur.

Contraintes dures

Elles ne peuvent jamais être violées.

Enseignant disponible
Groupe disponible
Salle disponible
Salle compatible
Capacité suffisante
Pas de chevauchement
Date dans période autorisée

Contraintes souples

Elles peuvent être violées avec pénalité.

Préférence enseignant
Préférence salle
Éviter cours très tôt
Éviter trous dans EDT
Limiter déplacements
Répartir les cours

16. Fonctionnement du moteur

Le principe :

1. Charger les enseignements
2. Charger les groupes
3. Charger les enseignants

4. Charger les espaces
 5. Charger les disponibilités
 6. Charger le calendrier
 7. Charger les contraintes
 8. Générer les possibilités
 9. Éliminer les conflits
 10. Optimiser
 11. Générer une proposition
 12. Contrôler
 13. Soumettre à validation
 14. Publier
-

17. Génération automatique

Le responsable clique :

« Générer l'emploi du temps »

Le système analyse :

150 enseignements
400 séances
25 enseignants
12 groupes
35 espaces

puis recherche une solution compatible.

Il doit produire :

- ✓ Séances planifiées
- ✓ Salles affectées
- ✓ Enseignants affectés
- ✓ Groupes affectés

et :

- ⚠ Séances non planifiées
 - ⚠ Raisons
 - ⚠ Contraintes bloquantes
-

18. Explication des échecs

Le système ne doit jamais simplement afficher :

« Génération impossible ».

Il doit expliquer :

3 séances non planifiées :

TD Module X - Groupe L2-A

→ Aucun espace disponible de capacité ≥ 45

→ Aucun créneau compatible entre 14h et 18h

TP Module Y

→ Laboratoires occupés sur les créneaux disponibles

C'est essentiel pour l'exploitation réelle.

19. Optimisation

Le moteur doit chercher à minimiser :

Conflits

+

Trous étudiants

+

Trous enseignants

+

Espaces inutilisés

+

Déplacements

+

Surcharge

Une fonction d'objectif peut conceptuellement être :

Score =

Pénalité conflits

+ pénalité trous

+ pénalité déplacements

+ pénalité déséquilibre

Les poids sont configurables.

20. Modification manuelle

La génération automatique ne doit pas empêcher l'intervention humaine.

Le responsable peut déplacer :

Cours A

Lundi 08h

Salle 101

vers :

Mardi 10h
Salle 204

Le système vérifie immédiatement :

- ✓ enseignant
 - ✓ groupe
 - ✓ salle
 - ✓ capacité
 - ✓ calendrier
-

21. Workflow de validation

BROUILLON
↓
GÉNÉRÉ
↓
À CONTRÔLER
↓
VALIDÉ
↓
PUBLIÉ
↓
ACTIF

Après publication, les modifications doivent être tracées.

22. Modification après publication

Une modification doit générer un événement :

Ancienne séance
↓
Modification
↓
Nouvelle séance
↓
Motif
↓
Auteur
↓
Date

Les étudiants et enseignants concernés peuvent recevoir une notification.

23. Annulation

Une séance peut devenir :

PLANIFIÉE
↓
ANNULÉE

avec :

motif
auteur
date

24. Rattrapage

Une séance annulée peut générer :

Séance originale
↓
ANNULATION
↓
À RATTRAPER
↓
Nouvelle séance

Le système doit conserver le lien entre les deux.

25. Remplacement d'enseignant

Exemple :

Cours prévu
↓
Enseignant indisponible
↓
Détection
↓
Recherche d'enseignants compatibles
↓
Proposition
↓
Validation
↓
Notification

Le remplacement doit être historisé.

26. EDT étudiant

Vue :

	LUN	MAR	MER	JEU	VEN
08-10	A	B	C	-	D
10-12	B	-	D	A	C
14-16	C	D	-	B	A

Chaque séance affiche :

Module
Enseignant
Salle
Type

27. EDT enseignant

Vue personnalisée :

MON EMPLOI DU TEMPS

Lundi

08h-10h → Module A – L1-A – Salle 101

14h-16h → Module B – L2-B – Salle 204

Mardi

10h-12h → Module C – L3-A – Gymnase

28. EDT par formation

Formation

↓

Parcours

↓

Niveau

↓

Groupe

↓

Semaine

↓

Jour

↓

Séances

29. EDT des espaces

C'est indispensable pour la gestion du patrimoine.

Salle S-101

Lundi

08-10 → L1-A
10-12 → L2-B
14-16 → L3-A

Mardi
08-10 → ...

Même principe pour :

- laboratoires ;
- gymnases ;
- terrains ;
- piscines ;
- amphithéâtres ;
- pistes ;
- salles spécialisées.

30. Réservation d'espace

Toutes les réservations doivent passer par le même moteur.

Types :

COURS
EXAMEN
RÉUNION
CONCOURS
STAGE
ÉVÉNEMENT
MAINTENANCE
AUTRE

Cela évite qu'une salle soit simultanément réservée par deux services.

31. Maintenance des espaces

Un espace peut être :

DISPONIBLE
OCCUPÉ
RÉSERVÉ
EN_MAINTENANCE
FERMÉ
HORS_SERVICE

Un espace en maintenance est automatiquement exclu de la génération EDT.

32. Examens

Les examens doivent utiliser le même moteur de ressources :

Examen
↓
Candidats / étudiants
↓
Groupes
↓
Surveillants
↓
Salles
↓
Capacités
↓
Créneaux

Le moteur doit pouvoir produire :

Planification
Convocations
Listes de salles
Listes d'étudiants
Surveillants
Émargements

33. API Django

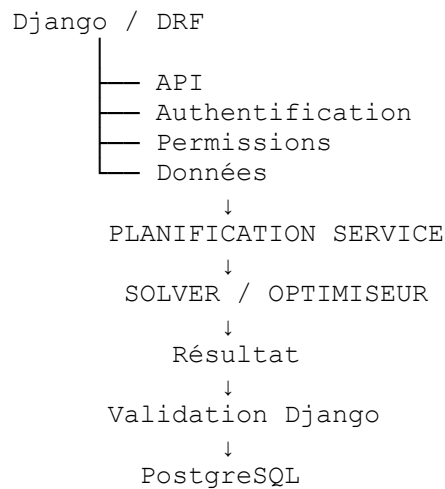
Architecture indicative :

```
/api/timetable/calendars/  
/api/timetable/periods/  
/api/timetable/slots/  
/api/timetable/teachings/  
/api/timetable/sessions/  
  
/api/timetable/constraints/  
/api/timetable/availability/  
/api/timetable/reservations/  
  
/api/timetable/generate/  
/api/timetable/validate/  
/api/timetable/publish/  
  
/api/timetable/conflicts/  
/api/timetable/replacements/  
/api/timetable/cancellations/  
/api/timetable/makeups/
```

34. Moteur technique

Pour une application Django moderne, je recommande de **séparer le moteur de planification du CRUD Django**.

Architecture :



Pour une première version, le moteur peut être implémenté avec un solveur de contraintes/optimisation adapté, plutôt que de multiplier des règles `if/else`.

35. Tableau de bord EDT

PLANIFICATION

Séances à planifier	520
Séances planifiées	505
Non planifiées	15
Conflits	0
Espaces utilisés	78 %
Charge enseignants	91 %

Alertes :

- ⚠ 15 séances non planifiées
- ⚠ 2 enseignants surchargés
- ⚠ 3 salles insuffisantes
- ⚠ 1 laboratoire indisponible

36. Module Flutter

EMPLOIS DU TEMPS

- Tableau de bord
- Calendrier
- Créneaux
- Enseignements

—	Séances
—	Génération automatique
—	Planification manuelle
—	Conflits
—	Espaces
—	Réservations
—	Examens
—	Rattrapages
—	Remplacements
—	Annulations
—	Validation
—	Publication
—	Statistiques

37. Ce que le système doit finalement permettre

Un responsable doit pouvoir faire :

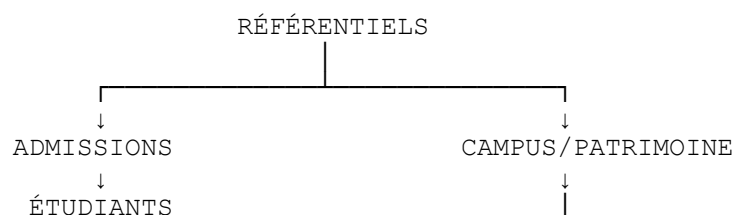
1. Sélectionner l'année
2. Sélectionner une formation
3. Sélectionner un semestre
4. Charger les enseignements
5. Vérifier les enseignants
6. Vérifier les groupes
7. Vérifier les espaces
8. Définir les contraintes
9. Lancer la génération
10. Examiner les conflits
11. Corriger éventuellement
12. Valider
13. Publier

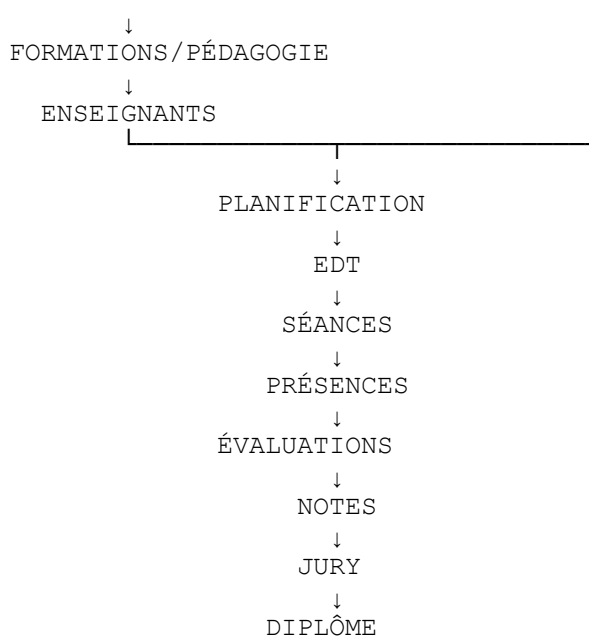
Et immédiatement :

Étudiant → voit son EDT
Enseignant → voit son EDT
Responsable → voit l'EDT global
Gestionnaire espace → voit l'occupation
Administration → voit les réservations

38. Architecture globale après le Module 06

Nous disposons maintenant d'une chaîne beaucoup plus solide :





Cette architecture permettra ensuite de connecter proprement le **Module 07 — CAMPUS, PATRIMOINE & ESPACES DE FORMATION**. Ce module sera particulièrement important pour l'INJS, car nous devons modéliser **sites** → **zones** → **bâtiments** → **niveaux** → **salles** → **laboratoires** → **gymnases** → **terrains** → **piscines** → **pistes** → **espaces spécialisés**, avec capacités, surfaces, équipements, disponibilité, maintenance et occupation par l'EDT.

Parfait. Nous passons au **MODULE 07 — CAMPUS, PATRIMOINE & ESPACES DE FORMATION**, indispensable pour que l'EDT de l'INJS puisse fonctionner réellement avec les infrastructures physiques.

MODULE 07 — CAMPUS, PATRIMOINE & ESPACES DE FORMATION

1. Objectif

Ce module doit permettre de répondre précisément à :

Où se déroule chaque activité de l'INJS, quelle est la capacité de l'espace, quels équipements possède-t-il, quand est-il disponible et qui l'utilise ?

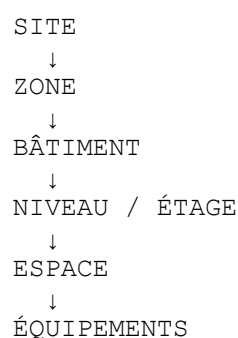
Il constitue le référentiel physique utilisé par :

- la planification ;
- les emplois du temps ;
- les cours ;
- les examens ;

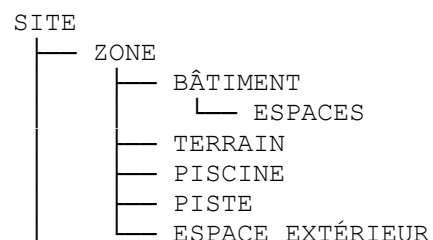
- les stages pratiques ;
- les activités sportives ;
- les réservations ;
- la maintenance ;
- l'administration ;
- la direction financière et comptable.

2. Architecture physique

La hiérarchie recommandée est :

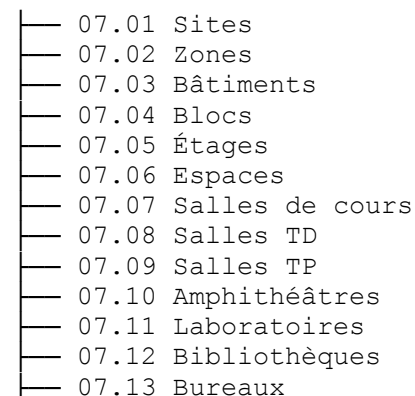


Mais certains espaces extérieurs doivent pouvoir être rattachés directement à une zone :



3. Sous-modules

07 — CAMPUS & PATRIMOINE



—	07.14 Salles spécialisées
—	07.15 Gymnases
—	07.16 Terrains
—	07.17 Piscines
—	07.18 Pistes d'athlétisme
—	07.19 Installations sportives
—	07.20 Espaces extérieurs
—	07.21 Équipements
—	07.22 Capacité
—	07.23 Surfaces
—	07.24 Disponibilité
—	07.25 Réservations
—	07.26 Maintenance
—	07.27 Incidents
—	07.28 Occupation
—	07.29 Plans / documents
—	07.30 Statistiques

4. Site

Un site représente une implantation physique.

Site
— Code
— Nom
— Adresse
— Localisation
— Description
— Responsable
— Statut
— Contact

Le système doit permettre plusieurs sites.

5. Zone

Une zone permet de regrouper plusieurs bâtiments ou installations.

Site
↓
Zone
— Bâtiment A
— Bâtiment B
— Gymnase
— Terrain
— Piscine

Informations :

Code
Libellé
Type

Site
Description
Surface
Statut

6. Bâtiment

Chaque bâtiment doit disposer d'un identifiant unique.

Bâtiment

- Code
- Nom
- Type
- Zone
- Nombre d'étages
- Surface totale
- Année de construction
- Statut
- Description

7. Niveau / étage

Exemple :

Bâtiment A

RDC

- Salle A01
- Salle A02
- Bureau A03

R+1

- Salle A11
- Salle A12
- Laboratoire A13

Le niveau doit être un objet propre.

8. Espace

Tous les espaces doivent hériter d'une structure commune.

SPACE

- Identifiant
- Code
- Nom
- Type
- Site
- Zone

—	Bâtiment
—	Étage
—	Surface
—	Capacité
—	Statut
—	Description

9. Types d'espaces

Le référentiel doit être configurable.

Enseignement

Salle de cours
Salle de TD
Salle de TP
Amphithéâtre
Laboratoire
Salle informatique
Salle spécialisée

Sport

Gymnase
Terrain de football
Terrain de basketball
Terrain de volleyball
Terrain de handball
Court de tennis
Piscine
Piste d'athlétisme
Salle de musculation
Salle de combat
Salle polyvalente

Administration

Bureau
Secrétariat
Salle de réunion
Salle du conseil
Archives

10. Capacité

Chaque espace possède :

Capacité normale
Capacité maximale

Exemple :

Salle X
Capacité normale : 40
Capacité maximale : 45

Le moteur EDT doit utiliser la capacité autorisée selon le type d'activité.

11. Surface

Le système doit conserver :

Surface utile
Surface totale

Exemple :

Salle X
Surface : 54 m²
Capacité : 40

Cela permettra ensuite d'effectuer des analyses de densité.

12. Équipements

Un espace peut posséder plusieurs équipements.

Salle

- Vidéoprojecteur
- Tableau
- Climatisation
- Wi-Fi
- Prises électriques
- Ordinateurs

Pour un laboratoire :

Laboratoire

- Équipements scientifiques
- Paillasses
- Électricité
- Eau
- Matériel spécialisé

Pour un gymnase :

Gymnase

- Sol sportif
- Paniers
- Buts
- Gradins

13. Équipement comme objet indépendant

Il est préférable de ne pas stocker uniquement :

```
"vidéoprojecteur": true
```

mais de créer :

```
Equipment
```

avec :

```
Code  
Désignation  
Type  
Marque  
Modèle  
Numéro de série  
Date acquisition  
État  
Valeur  
Date maintenance  
Statut
```

Cela permettra de relier le patrimoine physique à la comptabilité.

14. États des espaces

```
DISPONIBLE  
OCCUPÉ  
RÉSERVÉ  
MAINTENANCE  
FERMÉ  
HORS_SERVICE  
EN_RÉNOVATION
```

15. Disponibilité

Un espace doit avoir un calendrier d'utilisation.

```
Salle 101
```

```
Lundi  
08-10 → disponible  
10-12 → cours  
14-16 → cours
```

Mardi
08-10 → réservé
10-12 → disponible

Le moteur EDT consulte automatiquement cette disponibilité.

16. Réservation

Un espace peut être réservé indépendamment de l'EDT.

Exemples :

Réunion
Concours
Conférence
Formation
Examen
Cérémonie
Maintenance
Événement sportif

Workflow :

DEMANDE
↓
CONTRÔLE
↓
APPROBATION
↓
RÉSERVATION
↓
OCCUPATION
↓
LIBÉRATION

17. Conflits

Le système doit détecter :

Salle S101
10h-12h
Cours A

et :

Salle S101
10h-12h
Réunion B

→ **CONFLIT**

Même principe pour :

- gymnases ;
- piscines ;
- terrains ;
- laboratoires ;
- amphithéâtres.

18. Espaces sportifs

L'INJS ayant une forte composante sportive, ces espaces doivent être traités comme des ressources pédagogiques à part entière.

Exemple

PISCINE

—	Bassin
—	Longueur
—	Largeur
—	Profondeur
—	Nombre de lignes
—	Capacité
—	Vestiaires
—	Douches
—	Éclairage
—	Statut

19. Piste d'athlétisme

Prévoir notamment :

Type
Longueur
Nombre de couloirs
Surface
Capacité
Équipements
État

20. Terrain

Terrain

—	Type
—	Surface
—	Dimensions

- Capacité
 - Éclairage
 - Vestiaires
 - État
 - Disponibilité
-

21. Gymnase

Gymnase

- Surface
 - Dimensions
 - Capacité
 - Type de sol
 - Sports compatibles
 - Gradins
 - Vestiaires
 - Équipements
 - Disponibilité
-

22. Compatibilité pédagogique

Chaque espace doit pouvoir définir ses activités compatibles.

Exemple :

Gymnase A

Compatible :

- ✓ Basketball
- ✓ Handball
- ✓ Volleyball
- ✓ EPS

mais :

Piscine

Compatible :

- ✓ Natation
- ✓ Activités aquatiques

X Cours magistral

23. Règles de compatibilité

Créer une table :

SpaceActivityCompatibility

avec :

Space
Activity
Autorisé
Capacité
Contraintes

Ainsi, le système reste évolutif.

24. Connexion avec les formations

Un espace peut être utilisé par plusieurs formations.

Gymnase
↓
Formation A
Formation B
Formation C

Mais le système doit passer par les **séances** plutôt que de créer des relations rigides.

25. Connexion avec l'EDT

Le flux devient :

MODULE
↓
ENSEIGNEMENT
↓
GROUPE
↓
ENSEIGNANT
↓
ESPACE COMPATIBLE
↓
CRÉNEAU DISPONIBLE
↓
SÉANCE

Le Module 07 devient donc l'une des entrées principales du moteur du Module 06.

26. Maintenance

Chaque espace doit pouvoir avoir un historique :

Maintenance

- Type
- Date
- Description
- Prestataire
- Coût
- Responsable
- Date début
- Date fin
- Statut

Pendant la maintenance :

Espace = INDISPONIBLE

Le moteur EDT l'exclut automatiquement.

27. Incidents

Exemple :

Incident
Salle B204
Climatisation défectueuse

Le système peut automatiquement :

1. Marquer l'espace indisponible
 2. Identifier les séances impactées
 3. Proposer des espaces alternatifs
 4. Notifier le responsable
-

28. Gestion des capacités

Le système doit pouvoir calculer :

Capacité totale d'enseignement
Capacité disponible
Capacité occupée
Taux d'occupation

Exemple :

Salles de cours
Capacité totale : 800 places
Capacité occupée : 620
Taux : 77,5 %

29. Statistiques patrimoniales

Tableau de bord :

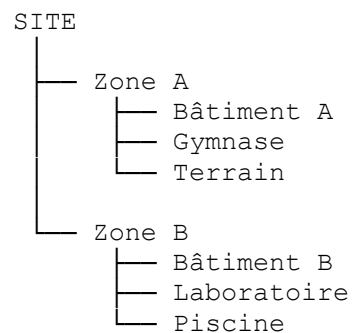
Sites
Zones
Bâtiments
Espaces
Salles
Laboratoires
Gymnases
Terrains
Piscines
Pistes

Puis :

Surface totale
Capacité totale
Espaces disponibles
Espaces en maintenance
Taux d'occupation

30. Cartographie

À terme, Flutter peut afficher :



Une future version peut intégrer une carte interactive.

31. API Django

/api/campus/sites/
/api/campus/zones/
/api/campus/buildings/
/api/campus/floors/
/api/campus/spaces/

/api/campus/space-types/

/api/campus/equipment/
/api/campus/compatibilities/

/api/campus/availability/
/api/campus/reservations/
/api/campus/maintenance/
/api/campus/incidents/

/api/campus/occupancy/
/api/campus/statistics/

32. Interface Flutter

CAMPUS & PATRIMOINE

—	Tableau de bord
—	Sites
—	Zones
—	Bâtiments
—	Étages
—	Espaces
—	— Salles
—	— Amphithéâtres
—	— Laboratoires
—	— Gymnases
—	— Terrains
—	— Piscines
—	— Pistes
—	Équipements
—	Disponibilités
—	Réservations
—	Occupation
—	Maintenance
—	Incidents
—	Plans
—	Statistiques

33. Fiche espace

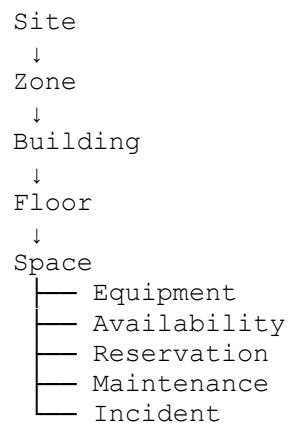
La fiche d'un espace doit présenter :

SALLE A-204	
Type	: Salle de cours
Capacité	: 50
Surface	: XX m ²
Bâtiment	: A
Niveau	: R+2
Statut	: Disponible
Équipements	
✓	Vidéoprojecteur
✓	Wi-Fi

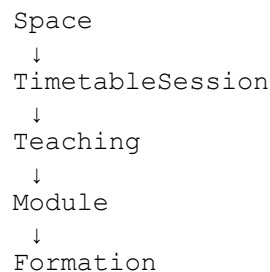
✓ Climatisation
Occupation
Lundi 08h-10h L1-A
10h-12h L2-B

34. Architecture de données

Le modèle devient :



Puis :



35. Point essentiel : ne pas mélanger espace et salle

Dans la base, je recommande :

Space

comme objet générique.

Puis :

SpaceType

détermine :

SALLE
LABORATOIRE
GYMNASE
TERRAIN
PISCINE
PISTE
BUREAU
AMPHITHÉÂTRE
...

Cela évite d'avoir 15 tables différentes avec une logique presque identique.

36. Exemple de structure Django

Conceptuellement :

Site
Zone
Building
Floor
SpaceType
Space
Equipment
SpaceEquipment
SpaceAvailability
SpaceReservation
Maintenance
Incident
SpaceActivityCompatibility

Puis les relations avec le système académique :

Teaching
TimetableSession
ExamSession

37. Principe de sécurité

Un enseignant ne doit pas pouvoir modifier le patrimoine.

Un gestionnaire des espaces peut :

- ✓ créer espace
- ✓ modifier espace
- ✓ déclarer maintenance
- ✓ gérer réservation

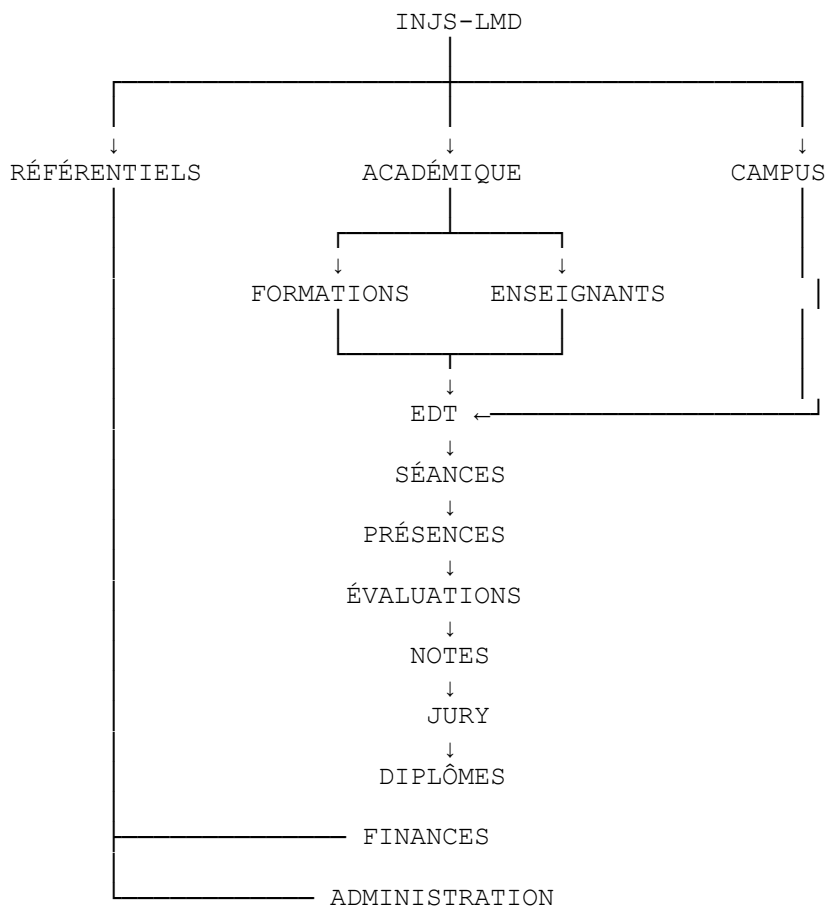
mais ne doit pas pouvoir :

- X modifier une note
- X modifier une maquette
- X modifier une affectation enseignant

Les permissions doivent donc être **fonctionnelles et granulaires**.

38. Vue globale de l'architecture

Nous avons maintenant :



Étape suivante

Nous avons maintenant posé les fondations de :

- 04 — Formations & Pédagogie**
- 05 — Enseignants & Charge pédagogique**
- 06 — Emplois du temps & Planification**
- 07 — Campus, Patrimoine & Espaces**

Le prochain bloc critique est :

MODULE 08 — PRÉSENCES & SUIVI DES ENSEIGNEMENTS

Il devra relier **étudiant + enseignant + groupe + séance + espace + QR code + émargement + absence + justification + seuil d'absence + rattrapage**, tout en alimentant automatiquement les statistiques de présence et, lorsque les règles le prévoient, les conditions d'accès aux évaluations.

Très bien. Nous passons au **MODULE 08 — PRÉSENCES, ÉMARGEMENT & SUIVI DES ENSEIGNEMENTS**.

Ce module doit être directement connecté à l'EDT, aux étudiants, aux enseignants, aux groupes, aux espaces et aux évaluations.

MODULE 08 — PRÉSENCES & SUIVI DES ENSEIGNEMENTS

1. Objectif

Le système doit permettre de savoir, pour chaque séance :

Quelle séance ? Quel enseignant ? Quel groupe ? Quels étudiants ? Quel espace ? Qui était présent, absent, en retard ou excusé ?

Le flux devient :

```
FORMATION
  ↓
MODULE
  ↓
ENSEIGNEMENT
  ↓
EDT
  ↓
SÉANCE
  ↓
ÉMARGEMENT
  ↓
PRÉSENCE / ABSENCE
  ↓
JUSTIFICATION
  ↓
STATISTIQUES
  ↓
ÉVENTUELLE INCIDENCE ACADÉMIQUE
```

2. Sous-modules

08 — PRÉSENCES

- 08.01 Séances
 - 08.02 Listes d'émargement
 - 08.03 Présences étudiants
 - 08.04 Présences enseignants
 - 08.05 QR Code
 - 08.06 Émargement manuel
 - 08.07 Retards
 - 08.08 Absences
 - 08.09 Justifications
 - 08.10 Validation
 - 08.11 Forçage contrôlé
 - 08.12 Rattrapages
 - 08.13 Alertes
 - 08.14 Seuils d'absence
 - 08.15 Statistiques
 - 08.16 Rapports
 - 08.17 Historique / audit
-

3. La séance est l'objet central

Le système ne doit pas créer une présence indépendamment d'une séance.

```
TimetableSession
    ↓
AttendanceSession
    ↓
StudentAttendance
```

Exemple :

```
Lundi 7 septembre
08h00-10h00
Module : X
Type : CM
Enseignant : Y
Groupe : L1-A
Salle : A101
```

Cette séance possède ensuite sa liste d'étudiants.

4. Ouverture d'une séance

Avant le début :

PLANIFIÉE

Au démarrage :

OUVERTE

Puis :

OUVERTE

↓

EN COURS

↓

CLÔTURÉE

5. Méthodes d'émargement

Le système peut proposer plusieurs méthodes.

QR Code

Étudiant

↓

Scanne QR

↓

Authentification

↓

Vérification séance

↓

Vérification groupe

↓

Enregistrement présence

Émargement enseignant

L'enseignant affiche :

☒ Étudiant A

☒ Étudiant B

☐ Étudiant C

☒ Étudiant D

Émargement administratif

Un utilisateur habilité peut enregistrer une présence selon ses droits.

6. Sécurité du QR Code

Le QR ne doit pas être un simple QR statique contenant :

student_id=123

Il doit utiliser un mécanisme temporaire.

Conceptuellement :

```
SESSION
↓
TOKEN TEMPORAIRE
↓
QR
↓
SCAN
↓
VALIDATION SERVEUR
```

Le serveur vérifie :

- ✓ étudiant authentifié
 - ✓ séance active
 - ✓ token valide
 - ✓ période autorisée
 - ✓ étudiant inscrit au groupe
 - ✓ présence non déjà enregistrée
-

7. Fenêtre d’émargement

On peut configurer :

Ouverture :
10 min avant

Fermeture :
15 min après début

Mais cette règle doit être paramétrable.

8. Localisation

Si l’INJS souhaite sécuriser davantage l’émargement, le système peut utiliser des mécanismes de contrôle liés à l’espace.

Par exemple :

```
QR séance
+
compte étudiant
+
séance active
+
espace attendu
```

Il faut cependant éviter de rendre la présence dépendante d'une géolocalisation GPS obligatoire si le contexte technique ou réglementaire ne le justifie pas.

9. Statuts de présence

Prévoir au minimum :

PRÉSENT
ABSENT
RETARD
ABSENCE JUSTIFIÉE
DISPENSÉ

Éventuellement :

PARTIELLEMENT PRÉSENT
PRÉSENCE_FORCÉE

10. Retard

Le retard doit être enregistré précisément.

Heure début séance : 08h00
Scan : 08h17

Retard : 17 minutes

Le système peut appliquer :

0-10 min → Présent
11-30 min → Retard
>30 min → Absent

Mais les seuils doivent être configurables.

11. Absence

Une absence doit être rattachée à :

Étudiant
Séance
Module
Groupe
Date
Type
Motif éventuel

12. Justification

Workflow :

```
ABSENCE
↓
ÉTUDIANT DÉPOSE JUSTIFICATIF
↓
EXAMEN
↓
ACCEPTÉ / REFUSÉ
↓
STATUT MIS À JOUR
```

Le justificatif peut être :

PDF
Image
Document

13. Validation de justification

Une personne habilitée peut :

- ✓ consulter
- ✓ accepter
- ✓ refuser
- ✓ demander complément

Chaque décision doit être historisée.

14. Forçage

Le forçage doit être extrêmement contrôlé.

Exemple :

L'étudiant était présent mais son téléphone n'a pas permis le scan.

Un utilisateur autorisé peut créer :

PRÉSENCE FORCÉE

avec :

Auteur
Date
Motif
Ancienne valeur
Nouvelle valeur

15. Audit

Aucune modification importante ne doit disparaître.

Exemple :

Étudiant : X
Séance : Y

08:05
ABSENT

08:30
→ Présent
Auteur : Responsable Z
Motif : QR défaillant

16. Présence enseignant

Le système peut également suivre :

Enseignant
↓
Séance
↓
Présence
↓
Heure d'arrivée
↓
Heure de départ

Cela permet de comparer :

VH planifiée
VH réalisée
VH effectivement assurée

17. Séance réalisée

Une séance doit pouvoir avoir :

PLANIFIÉE
EN_COURS

RÉALISÉE
ANNULÉE
REPORTÉE
NON_RÉALISÉE

18. Suivi du volume réellement dispensé

Exemple :

Module X

VH prévue : 30 h
VH planifiée : 30 h
VH réalisée : 26 h

Le responsable pédagogique voit immédiatement :

Écart : -4 h

Cela peut déclencher :

⚠ Rattrapage nécessaire

19. Rattrapage

Le système doit relier :

Séance originale
↓
Absence / annulation
↓
Rattrapage
↓
Nouvelle séance

La nouvelle séance reste liée à l'enseignement original.

20. Taux de présence étudiant

Calcul :

Taux présence =
 $\text{présences} / \text{séances attendues} \times 100$

Exemple :

20 séances

18 présences

Taux = 90 %

21. Statistiques par module

MODULE X

Présence : 91 %

Absence : 7 %

Retard : 2 %

22. Statistiques par étudiant

ÉTUDIANT X

Total séances : 120

Présences : 105

Absences : 10

Absences justifiées : 5

Retards : 8

Taux présence : 87,5 %

23. Seuil d'absence

Dans les règles déjà retenues pour le projet, le seuil de référence est :

25 %

Le système doit néanmoins le rendre configurable par :

Formation

Module

Année académique

Type d'enseignement

Il faut éviter de coder 25 % en dur dans Django.

24. Alerte automatique

Exemple :

Étudiant X

Absence :
23 %

⚠ Proche du seuil de 25 %

Puis :

Absence :
27 %

● Seuil dépassé

25. Conséquence académique

Il faut séparer :

TAUX D'ABSENCE

de :

DÉCISION ACADÉMIQUE

Le dépassement d'un seuil peut déclencher une **alerte ou une règle métier**, mais le système ne doit pas inventer automatiquement une sanction si le règlement applicable ne le prévoit pas.

26. Tableau de bord enseignant

MES SÉANCES

Aujourd'hui : 3

08h-10h
Module A
L1-A
Salle 101

10h-12h
Module B
L2-B
Salle 204

14h-16h
Module C
L3-A
Gymnase

Pour chaque séance :

Présents : 38

Absents : 4
Retards : 3

27. Tableau de bord étudiant

MA PRÉSENCE

Présence globale : 91 %

Modules :

Module A	96 %
Module B	88 %
Module C	79 %
Module D	94 %

Avec alerte :

⚠ Attention :
présence faible dans Module C.

28. Tableau de bord administration

PRÉSENCES

Étudiants attendus : 1 250
Présents : 1 104
Absents : 146

Taux global : 88,32 %

29. Tableau de bord pédagogique

SUIVI DES ENSEIGNEMENTS

VH planifiée	8 420 h
VH réalisée	7 960 h
Écart	460 h

Séances prévues	4 210
Réalisées	3 980
Annulées	85
Rattrapages	145

30. Notifications

Le système peut notifier :

Étudiant

Absence enregistrée
Justificatif accepté
Seuil approché
Cours annulé
Rattrapage programmé

Enseignant

Séance imminente
Salle modifiée
Cours annulé
Remplacement

Responsable

Taux d'absence élevé
Séances non réalisées
VH insuffisante

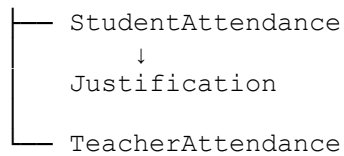
31. API Django

```
/api/attendance/sessions/  
/api/attendance/sessions/{id}/open/  
/api/attendance/sessions/{id}/close/  
  
/api/attendance/students/  
/api/attendance/students/{id}/stats/  
  
/api/attendance/scan/  
/api/attendance/qr/  
  
/api/attendance/absences/  
/api/attendance/late/  
/api/attendance/justifications/  
  
/api/attendance/overrides/  
/api/attendance/makeups/  
  
/api/attendance/statistics/  
/api/attendance/reports/
```

32. Modèle de données

Architecture recommandée :

```
TimetableSession  
|  
↓  
AttendanceSession  
|
```



Puis :

```
StudentAttendance
  ↓
AttendanceStatistics
  ↓
Alerts
```

33. QR Code

Le QR doit être lié à la séance et non simplement à l'étudiant.

AttendanceToken

```
session
token
created_at
expires_at
status
```

Le serveur peut renouveler périodiquement le token.

34. Protection contre la fraude

Le système doit notamment empêcher :

```
X double émargement
X QR expiré
X étudiant hors groupe
X séance inexistante
X séance fermée
X token réutilisé
X présence créée sans autorisation
```

Il doit journaliser les événements suspects.

35. Connexion avec les notes

Le flux complet devient :

ÉTUDIANT
↓
INSCRIPTION
↓
GROUPE
↓
MODULE
↓
SÉANCES
↓
PRÉSENCES
↓
ÉVALUATION
↓
NOTE

Mais :

présence ≠ note.

La présence doit rester une donnée indépendante pouvant alimenter certaines règles académiques configurées.

36. Connexion avec l'EDT

EDT
↓
Séance
↓
Enseignant
↓
Groupe
↓
Espace
↓
Présences

Si une séance est déplacée :

EDT modifié
↓
Séance mise à jour
↓
Émargement associé à la nouvelle date/heure/espace

37. Connexion avec le patrimoine

Le système sait :

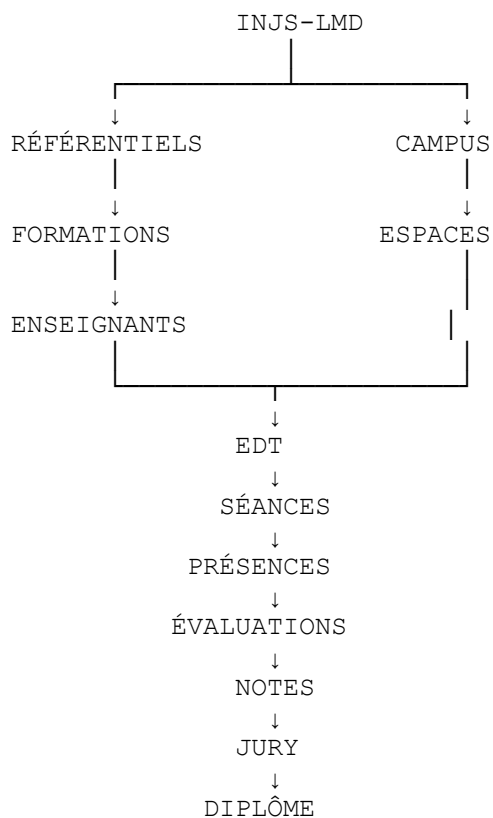
Salle A101
↓

Cours
↓
Présences
↓
Effectif réel

Cela permet ensuite de produire :

Taux d'utilisation de la salle
Nombre d'étudiants accueillis
Nombre de séances
Nombre d'heures occupées

38. Architecture globale désormais



39. Prochaine étape

Nous pouvons maintenant passer au **MODULE 09 — ÉVALUATIONS, NOTES & RÉSULTATS ACADÉMIQUES**.

Ce sera un module particulièrement important puisqu'il devra gérer de bout en bout :

Évaluation
→ Épreuve
→ Convocation

- Copie / résultat
- Saisie de note
- CC
- CT
- Moyenne
- Coefficient
- Crédits
- Validation UE
- Validation semestre
- Compensation
- Rattrapage
- Résultat annuel
- Décision
- Transmission au jury

Il constituera le lien direct entre l'activité pédagogique quotidienne et la décision officielle de réussite, passage, ajournement ou diplôme.

Parfait. Nous passons au **MODULE 09 — ÉVALUATIONS, NOTES & RÉSULTATS ACADÉMIQUES**. C'est le cœur du moteur LMD : il doit transformer les enseignements suivis en résultats académiques fiables, traçables et exploitables par les jurys.

MODULE 09 — ÉVALUATIONS, NOTES & RÉSULTATS ACADÉMIQUES

1. Objectif

Le module doit gérer toute la chaîne :

MAQUETTE
↓
UNITÉS D'ENSEIGNEMENT
↓
MODULES / ECUE
↓
ENSEIGNEMENTS
↓
ÉVALUATIONS
↓
ÉPREUVES
↓
NOTES
↓
CALCULS
↓
CRÉDITS
↓
RÉSULTATS
↓
JURY
↓
DÉCISION

Il doit être impossible de modifier silencieusement une note déjà validée.

2. Sous-modules

09 – ÉVALUATIONS & NOTES

—	09.01	Types d'évaluation
—	09.02	Évaluations
—	09.03	Épreuves
—	09.04	Calendrier des évaluations
—	09.05	Convocations
—	09.06	Listes d'étudiants
—	09.07	Surveillance
—	09.08	Saisie des notes
—	09.09	Import des notes
—	09.10	Contrôle des notes
—	09.11	CC
—	09.12	CT
—	09.13	Moyennes
—	09.14	Coefficients
—	09.15	Crédits ECTS
—	09.16	Validation UE
—	09.17	Validation semestre
—	09.18	Compensation
—	09.19	Rattrapages
—	09.20	Résultats annuels
—	09.21	Décisions
—	09.22	Jury
—	09.23	Procès-verbaux
—	09.24	Publication
—	09.25	Réclamations
—	09.26	Corrections
—	09.27	Historique
—	09.28	Statistiques

3. Ne pas confondre les objets

Le système doit distinguer :

ÉVALUATION

et :

ÉPREUVE

et :

NOTE

Exemple :

Évaluation
↓
Contrôle continu
↓
Épreuve CC #1
↓
Note étudiant

Puis :

Épreuve CT
↓
Note étudiant

4. Types d'évaluation

Le référentiel doit être configurable.

Par exemple :

CC – Contrôle continu
CT – Contrôle terminal
EXAMEN
RATTRAPAGE
PROJET
MÉMOIRE
STAGE
SOUTENANCE
AUTRE

5. Pondération

Dans le cadre déjà défini pour le projet :

CC = 40 %
CT = 60 %

Mais il faut **paramétrer** cette règle plutôt que coder définitivement 40/60.

Exemple :

Règle d'évaluation

CC : 40 %
CT : 60 %

Une autre formation pourrait avoir :

CC : 50 %
CT : 50 %

si le règlement applicable le prévoit.

6. Structure pédagogique

Le calcul doit respecter la structure LMD définie dans la maquette.

Exemple :

```
SEMESTRE
  ↓
UE
  ↓
ECUE / MODULE
  ↓
ÉVALUATIONS
  ↓
NOTES
```

Chaque niveau peut avoir :

```
Crédits
Coefficient
Note
Moyenne
```

7. Étudiant évalué

Une note doit toujours être rattachée à un contexte académique.

```
StudentGrade
├── Étudiant
├── Inscription
├── Formation
├── Année académique
├── Semestre
├── UE
├── Module
├── Évaluation
├── Épreuve
├── Note
├── Barème
└── Statut
```

8. Barème

Le système doit gérer :

Note maximale : 20

Note minimale : 0

Mais également pouvoir supporter d'autres barèmes :

/10

/20

/100

A/B/C

Validé / Non validé

Pour l'INJS-LMD, le barème principal peut être /20.

9. Saisie des notes

L'enseignant doit pouvoir saisir les notes de son épreuve.

Exemple :

CC – Biomécanique		
Matricule	Étudiant	Note
INJS001	Étudiant A	14,50
INJS002	Étudiant B	12,00
INJS003	Étudiant C	16,00

10. Contrôles automatiques

Lors de la saisie :

Note < 0

→ 

Note > 20

→ 

Étudiant non inscrit

→ 

Étudiant absent de l'épreuve

→  selon règle

Note déjà validée

→ 

11. Statuts de note

Chaque note doit avoir un cycle de vie.

BROUILLON
↓
SAISIE
↓
CONTRÔLÉE
↓
VALIDÉE
↓
PUBLIÉE

Après validation :

 Note verrouillée

12. Correction d'une note

Une note validée ne doit jamais être directement écrasée.

Exemple :

14,00
↓
demande de correction
↓
15,00

Historique :

Ancienne note : 14
Nouvelle note : 15
Motif : erreur de saisie
Auteur : X
Valideur : Y
Date : ...

13. Import Excel

Le système peut permettre :

Télécharger modèle
↓
Remplir notes
↓
Importer
↓
Contrôler
↓
Afficher erreurs
↓
Valider

Il faut toutefois éviter que l'import permette de contourner les contrôles métier.

14. Calcul CC

Exemple :

CC1 = 14
CC2 = 12
CC3 = 16

Moyenne :

$CC = (14 + 12 + 16) / 3$
 $CC = 14$

Le système doit également pouvoir gérer des coefficients différents entre évaluations.

15. Calcul CC/CT

Avec la règle :

CC = 40 %
CT = 60 %

Exemple :

CC = 14
CT = 12

Calcul :

Moyenne =
(14 × 0,40)
+
(12 × 0,60)
= 12,80 / 20

Le calcul doit être automatique et reproductible.

16. Coefficients

Exemple :

Module
Coefficient : 3
Crédit : 5

La moyenne du semestre doit utiliser les coefficients prévus par la maquette.

17. Crédits ECTS

Chaque UE/module peut avoir :

Crédits prévus
Crédits acquis
Crédits non acquis

Exemple :

UE 1
Crédits : 6
Résultat : validée
→ 6 crédits acquis

18. Validation d'une UE

Le moteur doit appliquer les règles du référentiel académique.

Exemple conceptuel :

Note UE $\geq$ seuil
↓
UE VALIDÉE
↓
Crédits acquis

Le seuil ne doit pas être supposé universel : il doit être paramétrable selon les règles académiques applicables.

19. Compensation

La compensation doit être un mécanisme configurable.

Exemple :

UE A : 14
UE B : 08
UE C : 12

Le moteur calcule la moyenne selon :

$$\frac{\text{notes} \times \text{coefficients}}{\text{somme coefficients}}$$

Puis applique les règles de compensation définies par le règlement.

20. Semestre

Le système produit :

SEMESTRE 1

UE 1	13,50	6 crédits
UE 2	11,20	5 crédits
UE 3	08,50	4 crédits
UE 4	14,00	5 crédits

Puis :

Moyenne S1 : 11,75
Crédits acquis : XX

21. Résultat du semestre

Le moteur peut produire :

ADMIS
AJOURNÉ
VALIDATION PARTIELLE

Mais les libellés et règles doivent être configurables.

22. Année académique

Le système consolide :

Semestre 1
+
Semestre 2
+
Rattrapages éventuels
+
Décisions de jury

pour construire le résultat annuel.

23. Rattrapage

Workflow :

```
Résultat initial
↓
UE non validée
↓
Éligibilité rattrapage
↓
Convocation
↓
Épreuve
↓
Nouvelle note
↓
Recalcul
↓
Nouveau résultat
```

24. Conservation des notes initiales

Important :

La note initiale ne doit pas être supprimée.

Session normale
Note : 08

Session rattrapage
Note : 13

Le système conserve les deux.

Il applique ensuite la règle de conservation prévue par le règlement.

25. Jury

Le module 09 prépare les données du jury.

```
Étudiant
↓
Résultats
↓
Crédits
```

↓
Moyennes
↓
Anomalies
↓
Proposition de décision
↓
JURY

Le jury doit pouvoir consulter :

Résultats
Absences
Notes
Crédits
UE
Semestres
Rattrapages

26. Décision du jury

Le jury peut prendre une décision officielle.

Exemples :

ADMIS
ADMIS AVEC DETTE
AJOURNÉ
AUTRE DÉCISION

Les valeurs exactes doivent venir du référentiel réglementaire de l'établissement.

27. Procès-verbal

Le système doit générer un PV.

Contenu :

INJS
Année académique
Formation
Niveau
Semestre
Session
Composition du jury
Liste étudiants
Résultats
Décisions
Signatures

Le PV doit être verrouillé après validation.

28. Publication

Workflow :

```
BROUILLON
↓
CONTRÔLÉ
↓
JURY
↓
VALIDÉ
↓
PUBLIÉ
```

L'étudiant voit ensuite :

Mes résultats

29. Relevé de notes

Le système doit générer automatiquement :

RELEVÉ DE NOTES

Étudiant
Matricule
Formation
Année académique

UE	Module	Note	Coef	Crédits
----	--------	------	------	---------

...				

Puis :

Moyenne semestre
Crédits acquis
Décision

30. Attestation

Après validation administrative :

Résultat
↓
Décision

↓
Attestation de réussite

Le document doit comporter un identifiant permettant sa vérification.

31. Anti-fraude documentaire

Le document peut intégrer :

Numéro unique
QR Code
Identifiant de vérification
Date d'émission
Signature électronique

La vérification pourrait fonctionner ainsi :

QR
↓
API publique
↓
Vérification document
↓
Authentique / Invalide

Sans exposer les données personnelles inutiles.

32. Réclamations

Un étudiant peut soumettre :

Réclamation note

Workflow :

Étudiant
↓
Réclamation
↓
Enseignant / responsable
↓
Examen
↓
Décision
↓
Historisation

33. Contrôle des anomalies

Avant jury :

- ⚠ Note manquante
- ⚠ Note > barème
- ⚠ UE sans résultat
- ⚠ Crédits incohérents
- ⚠ Coefficient manquant
- ⚠ Étudiant sans inscription
- ⚠ Rattrapage incohérent

Le système doit disposer d'un **contrôle de cohérence académique**.

34. Tableau de bord pédagogique

ÉVALUATIONS

Étudiants : 1 250

Notes attendues : 8 500

Notes saisies : 8 420

Notes manquantes : 80

Validation :

 99 %

35. Tableau de bord enseignant

MES ÉVALUATIONS

Biomécanique

CC1 → 45/45 notes

CC2 → 45/45 notes

CT → 45/45 notes

État : terminé

36. Tableau de bord étudiant

MES RÉSULTATS

Semestre 1

UE 1 14,20 ✓

UE 2 11,80 ✓

UE 3 8,70 ⚠
UE 4 13,50 ✓

Moyenne : 12,05
Crédits acquis : 24/30

37. Architecture Django

API :

```
/api/evaluations/types/  
/api/evaluations/  
/api/evaluations/{id}/exams/  
/api/evaluations/{id}/students/  
  
/api/grades/  
/api/grades/import/  
/api/grades/validate/  
/api/grades/corrections/  
  
/api/results/  
/api/results/semester/  
/api/results/year/  
  
/api/credits/  
/api/compensation/  
/api/remedial/  
  
/api/juries/  
/api/juries/{id}/deliberate/  
/api/juries/{id}/validate/  
  
/api/transcripts/  
/api/certificates/  
/api/complaints/
```

38. Architecture des données

Conceptuellement :

```
AcademicYear  
↓  
Enrollment  
↓  
Teaching  
↓  
Evaluation  
↓  
Assessment  
↓  
Grade  
↓  
Calculation  
↓
```

UEResult
↓
SemesterResult
↓
YearResult
↓
JuryDecision

39. Règle d'or du moteur de calcul

Il faut **séparer les données brutes des résultats calculés**.

Par exemple :

Grade

contient :

14

mais :

ModuleResult

contient :

Moyenne calculée

et :

SemesterResult

contient :

Moyenne semestre

Cela permet de recalculer et d'auditer les résultats.

40. Versioning des règles

C'est extrêmement important pour une application universitaire.

Une règle académique ne doit pas être modifiée rétroactivement.

Exemple :

Règlement 2026-2027
CC = 40 %

CT = 60 %

Si une nouvelle règle est adoptée :

Règlement 2027-2028

CC = ...

CT = ...

Les anciens étudiants continuent d'être calculés avec les règles applicables à leur année.

41. Audit complet

Le système doit pouvoir répondre à :

Qui a saisi cette note ?

Quand ?

Qui l'a validée ?

Quelle était la valeur précédente ?

Quelle règle a été utilisée ?

Quand le résultat a-t-il été calculé ?

Quand le jury l'a-t-il validé ?

Cela nécessite un journal d'audit robuste.

42. Architecture globale actuelle

Nous avons maintenant :

01 - AUTHENTIFICATION / UTILISATEURS
02 - ADMINISTRATION
03 - RÉFÉRENTIELS
04 - FORMATIONS / MAQUETTES
05 - ENSEIGNANTS
06 - PLANIFICATION / EDT
07 - CAMPUS / ESPACES
08 - PRÉSENCES
09 - ÉVALUATIONS / NOTES

Le parcours devient :

CANDIDATURE
↓
ADMISSION
↓
INSCRIPTION
↓
FORMATION
↓
GROUPE
↓
ENSEIGNEMENTS
↓
EDT
↓
SÉANCES
↓
PRÉSENCES
↓
ÉVALUATIONS
↓
NOTES
↓
CRÉDITS
↓
RÉSULTATS
↓
JURY
↓
DÉCISION
↓
DIPLOMATION

Prochaine brique : MODULE 10 — JURYS, DIPLOMATION & DOCUMENTS ACADÉMIQUES

Nous pourrions alors fermer la boucle académique avec :

jury → délibération → décisions → procès-verbal → résultats définitifs → relevé de notes → attestation → diplôme → registre des diplômés → vérification QR → archivage.

Ensuite, nous passerons aux **FRAIS SCOLAIRES & FINANCES**, puis à la **DIRECTION FINANCIÈRE ET COMPTABLE**, à l'**ADMINISTRATION**, et enfin à l'**architecture technique complète Django API + Flutter**, avec les rôles, permissions, workflows, modèles de données et API nécessaires pour construire l'application INJS de bout en bout.

Parfait. Nous passons au **MODULE 10 — JURYS, DIPLOMATION & DOCUMENTS ACADÉMIQUES**. C'est la dernière grande étape du parcours académique : elle transforme les résultats calculés en **décisions officielles**, puis en documents et en archives vérifiables.

MODULE 10 — JURYS, DIPLÔMATION & DOCUMENTS ACADÉMIQUES

1. Objectif

Le module doit gérer :

NOTES
↓
RÉSULTATS
↓
CONTRÔLE ACADÉMIQUE
↓
JURY
↓
DÉLIBÉRATION
↓
DÉCISION
↓
RÉSULTATS DÉFINITIFS
↓
DOCUMENTS ACADÉMIQUES
↓
DIPLÔME
↓
REGISTRE DES DIPLÔMÉS
↓
VÉRIFICATION
↓
ARCHIVAGE

2. Sous-modules

10 — JURYS & DIPLÔMATION

- 10.01 Sessions de jury
- 10.02 Composition des jurys
- 10.03 Membres
- 10.04 Convocations
- 10.05 Dossiers étudiants
- 10.06 Contrôle préalable
- 10.07 Délibération
- 10.08 Décisions
- 10.09 Procès-verbaux
- 10.10 Résultats définitifs
- 10.11 Relevés de notes
- 10.12 Attestations
- 10.13 Certificats
- 10.14 Diplômes
- 10.15 Numérotation
- 10.16 QR de vérification
- 10.17 Registre des diplômés
- 10.18 Rectification

- 10.19 Réédition
 - 10.20 Archivage
 - 10.21 Statistiques
-

3. Session de jury

Une session doit être un objet officiel.

```
JurySession
├── Année académique
├── Formation
├── Niveau
├── Semestre
├── Session
├── Date
├── Lieu
├── Président
├── Membres
├── Statut
└── Documents
```

Exemple :

```
Jury
Licence – Éducation et Motricité
Année : 2026-2027
Session : normale
Date : ...
```

4. Statuts du jury

```
BROUILLON
CONSTITUÉ
CONVOQUÉ
PRÉPARÉ
EN DÉLIBÉRATION
DÉLIBÉRÉ
VALIDÉ
CLOS
ARCHIVÉ
```

5. Composition

Le système doit permettre de définir :

```
Président
Vice-président
Membre
Rapporteur
Secrétaire
```

avec les responsabilités correspondantes.

6. Contrôle avant jury

Avant de permettre la délibération :

- ✓ étudiant correctement inscrit
- ✓ formation correcte
- ✓ résultats complets
- ✓ notes cohérentes
- ✓ calculs effectués
- ✓ crédits calculés
- ✓ rattrapages traités
- ✓ anomalies résolues

Le système doit bloquer la clôture si une anomalie bloquante subsiste.

7. Dossier étudiant pour jury

Le jury doit pouvoir consulter une fiche synthétique :

DOSSIER ÉTUDIANT
Identité Matricule Formation Parcours Niveau
S1 : 12,40 / 30 crédits S2 : 11,90 / 30 crédits Moyenne annuelle : 12,15 Crédits : 60 / 60
Proposition : ADMIS

8. Délibération

Le jury examine les propositions.

Le système doit permettre :

Proposition automatique

↓

Examen par jury

↓

Décision

La proposition automatique n'est **pas** la décision finale.

9. Décisions

Le référentiel doit permettre plusieurs décisions.

Par exemple :

ADMIS
AJOURNÉ
ADMIS AVEC DETTE
VALIDATION PARTIELLE
AUTRE

Les valeurs doivent être configurables conformément aux règles académiques applicables.

10. Modification par le jury

Le jury peut, lorsque le règlement l'autorise, modifier une proposition.

Exemple :

Calcul système
Moyenne : 9,95

Proposition : AJOURNÉ

Décision jury :
ADMIS

Cette décision doit être :

- explicitement enregistrée ;
 - motivée si nécessaire ;
 - attribuée aux membres habilités ;
 - audité.
-

11. Historique

Il faut conserver :

Calcul automatique
↓
Proposition
↓
Modification
↓
Décision finale

Aucune information ne doit être perdue.

12. Procès-verbal

Après délibération :

JURY
↓
PV
↓
VALIDATION
↓
SIGNATURE
↓
ARCHIVAGE

Le PV doit comporter notamment :

INJS
Formation
Année
Session
Date
Composition du jury
Étudiants
Résultats
Décisions
Observations
Signatures

13. Verrouillage

Une fois le PV validé :

 PV verrouillé

Toute modification ultérieure doit passer par une procédure officielle de rectification.

14. Résultat définitif

Après validation :

Résultat provisoire

↓

Décision jury

↓

Résultat définitif

Le résultat définitif devient la référence officielle.

15. Relevé de notes

Le système génère automatiquement le relevé.

RELEVÉ DE NOTES

Étudiant

Matricule

Formation

Parcours

Année académique

UE | Module | Note | Coef. | Crédits

...

Moyenne :

Crédits :

Décision :

16. Attestation de réussite

Une fois le résultat définitif validé :

Résultat définitif

↓

Attestation

Le document doit recevoir un numéro unique.

Exemple :

AT-2027-000125

17. Diplôme

Le diplôme ne doit pas être créé simplement parce qu'un étudiant possède une moyenne suffisante.

Le workflow doit être :

```
Résultats
↓
Décision du jury
↓
Conditions de diplomation
↓
Validation administrative
↓
Émission diplôme
```

18. Conditions de diplomation

Le moteur doit contrôler :

- ✓ formation validée
- ✓ niveau validé
- ✓ crédits requis acquis
- ✓ stages validés si requis
- ✓ mémoire validé si requis
- ✓ soutenance validée si requise
- ✓ décision du jury
- ✓ conditions administratives

Les conditions doivent être paramétrables par diplôme/formation.

19. Registre des diplômes

Chaque diplôme doit posséder un identifiant unique.

```
Diploma
├── Numéro
├── Étudiant
├── Formation
├── Diplôme
├── Année
├── Date d'émission
├── Jury
├── PV
├── Document
├── QR
└── Statut
```

20. Numérotation

Le système doit gérer une séquence contrôlée.

Exemple :

```
DIP-2027-000001
DIP-2027-000002
DIP-2027-000003
```

Il faut empêcher :

```
X doublon
X suppression arbitraire
X réutilisation d'un numéro
```

21. QR Code de vérification

Chaque diplôme peut posséder un QR.

```
QR
↓
Identifiant document
↓
Service de vérification
↓
Authentique / Invalide
```

La page publique ne doit révéler que les informations nécessaires à la vérification.

22. Vérification publique

Exemple :

VÉRIFICATION DE DOCUMENT

Numéro :
DIP-2027-000125

Statut :
✓ DOCUMENT AUTHENTIQUE

Diplôme :
Licence ...

Année :

2026-2027

Date d'émission :
...

Les données personnelles sensibles doivent rester protégées.

23. Réédition

Un document peut être :

ORIGINAL
RÉÉDITION
DUPLICATA

avec justification.

Exemple :

Duplicata
Motif : document perdu
Date : ...
Autorisé par : ...

24. Rectification

Une erreur sur un document doit passer par :

Demande
↓
Vérification
↓
Autorisation
↓
Correction
↓
Nouvelle version
↓
Historisation

L'ancienne version ne doit pas être effacée.

25. Version des documents

Exemple :

Diplôme V1
↓
Erreur détectée
↓
Diplôme V2

Le système conserve :

V1 → annulée/remplacée
V2 → active

selon la procédure applicable.

26. Archivage

À la fin du processus :

Dossier étudiant
├─ Inscription
├─ Notes
├─ Résultats
├─ Jury
├─ PV
├─ Relevés
├─ Attestation
└─ Diplôme

Le dossier devient une **archive académique**.

27. Traçabilité

Il faut pouvoir répondre des années plus tard :

Quel diplôme a été délivré ?

À quel étudiant ?

Dans quelle formation ?

Avec quelle décision de jury ?

Quel PV ?

Quel relevé ?

Qui a validé ?

Quand ?

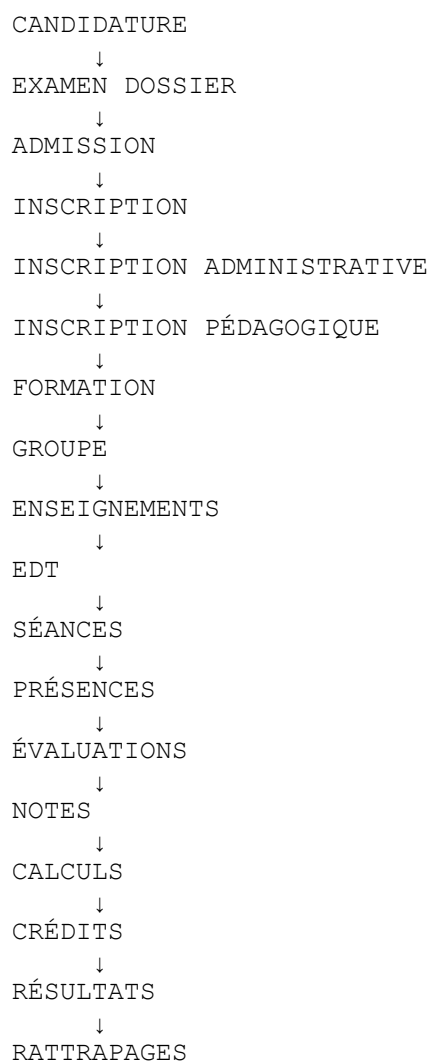
28. Tableau de bord diplôme

DIPLÔMATION 2026-2027

Étudiants présentés	320
Admis	280
Ajournés	40
Diplômes à produire	280
Diplômes produits	260
Diplômes remis	220

29. Workflow complet

Voici maintenant le parcours académique complet :



↓
RÉSULTATS DÉFINITIFS
↓
JURY
↓
DÉCISION
↓
PV
↓
DIPLÔMATION
↓
RELEVÉ
↓
ATTESTATION
↓
DIPLÔME
↓
REGISTRE DES DIPLÔMÉS
↓
ARCHIVAGE

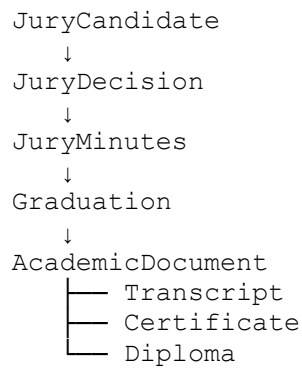
30. API Django

Architecture REST indicative :

```
/api/juries/  
/api/juries/{id}/members/  
/api/juries/{id}/students/  
/api/juries/{id}/deliberate/  
/api/juries/{id}/validate/  
/api/juries/{id}/close/  
  
/api/jury-decisions/  
/api/jury-minutes/  
  
/api/results/final/  
/api/transcripts/  
/api/attestations/  
/api/diplomas/  
/api/diplomas/{id}/verify/  
/api/diplomas/{id}/reissue/  
/api/diplomas/{id}/duplicate/  
  
/api/graduates/  
/api/archives/
```

31. Architecture des données

Student
↓
Enrollment
↓
AcademicResults
↓



32. Séparation essentielle

Il faut distinguer :

RÉSULTAT CALCULÉ

de :

DÉCISION DU JURY

et :

DOCUMENT ÉMIS

Par exemple :

Moyenne calculée : 9,95

↓
Proposition : AJOURNÉ

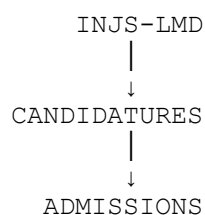
↓
Décision jury : ADMIS

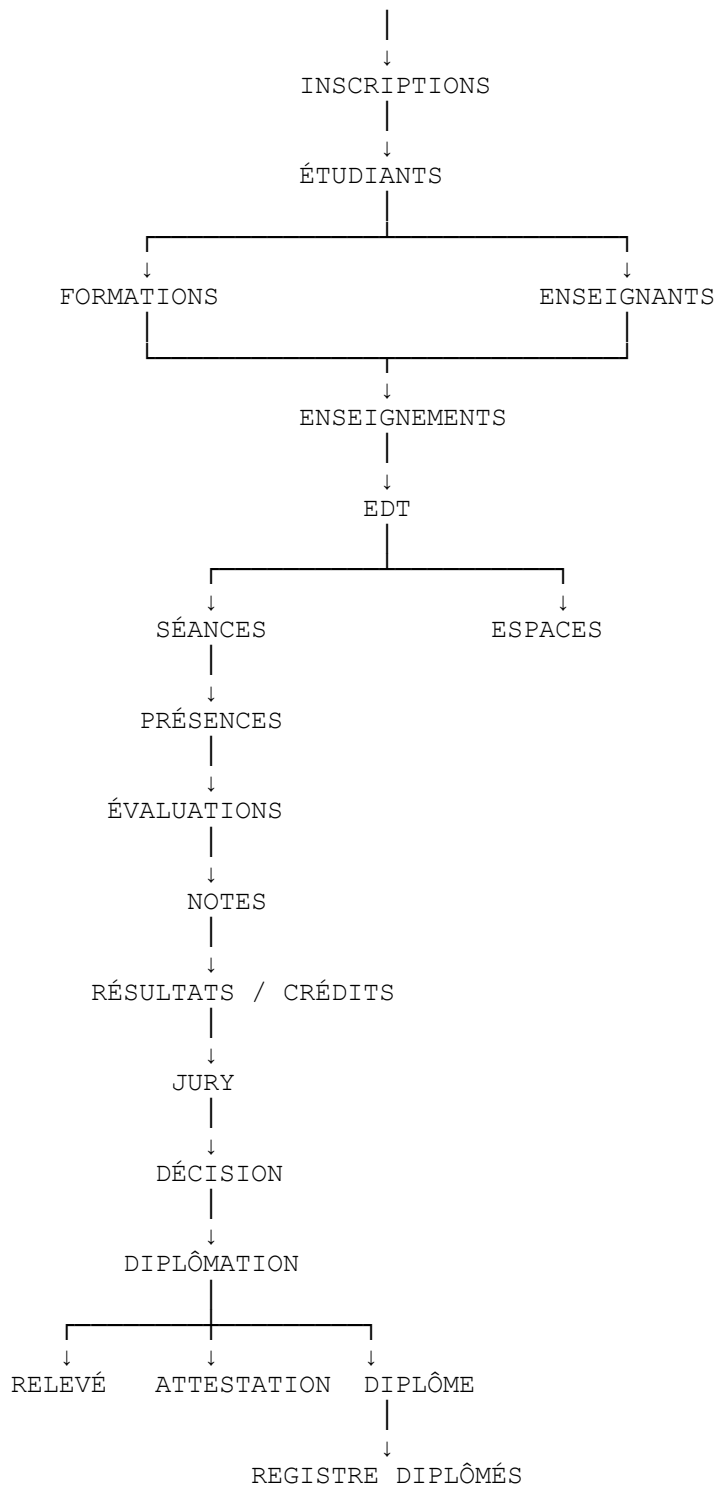
↓
Diplôme : ÉMIS

Ces quatre événements doivent rester traçables.

33. Architecture académique complète

Nous pouvons maintenant représenter le noyau LMD :





34. Les prochains modules

Nous avons maintenant terminé la conception du **noyau académique**.

La prochaine partie doit traiter l'argent :

MODULE 11 — FRAIS SCOLAIRES & FACTURATION

Avec :

Frais d'inscription
Frais de scolarité
Frais de formation
Frais d'examen
Frais de dossier
Frais de stage
Autres frais
Échéanciers
Factures
Paielements
Soldes
Impayés
Exonérations
Bourses
Remboursements
Reçus

Puis :

MODULE 12 — DIRECTION FINANCIÈRE & COMPTABLE

Avec :

Budget
Engagements
Dépenses
Recettes
Trésorerie
Banques
Caisses
Comptabilité
Pièces justificatives
Rapprochement bancaire
Fournisseurs
Clients
Immobilisations
États financiers
Contrôles
Audit

Puis nous traiterons **l'administration générale**, les **RH**, les **courriers/documents**, les **secrétariats**, les **archives**, les **notifications**, puis enfin la **conception technique complète Django + API + Flutter + PostgreSQL + sécurité + rôles/permissions + architecture des modules**.

Cela nous permettra de passer ensuite de la **description fonctionnelle de l'INJS** à un véritable **cahier d'architecture et de spécifications directement exploitable par Cline pour développer l'application**.

Parfait. Nous poursuivons avec le **MODULE 11 — FRAIS SCOLAIRES, FACTURATION & FINANCES ÉTUDIANTES**.

L'objectif est de construire une gestion financière **complètement paramétrable**, afin de ne pas figer dans le code des montants ou des règles qui pourraient changer d'une année académique à l'autre.

MODULE 11 — FRAIS SCOLAIRES & FINANCES ÉTUDIANTES

1. Objectif

Le module doit gérer le cycle :

```
ÉTUDIANT
↓
INSCRIPTION
↓
TARIFICATION
↓
FACTURATION
↓
ÉCHÉANCIER
↓
PAIEMENT
↓
REÇU
↓
SOLDE
↓
RELANCE
↓
RÉGULARISATION
↓
COMPTABILISATION
```

2. Sous-modules

11 — FINANCES ÉTUDIANTS

- 11.01 Grilles tarifaires
- 11.02 Types de frais
- 11.03 Tarifs par formation
- 11.04 Tarifs par niveau
- 11.05 Frais d'inscription
- 11.06 Frais de scolarité
- 11.07 Frais d'examen
- 11.08 Frais de stage
- 11.09 Autres frais

—	11.10	Exonérations
—	11.11	Bourses
—	11.12	Réductions
—	11.13	Factures
—	11.14	Échéanciers
—	11.15	Paielements
—	11.16	Reçus
—	11.17	Impayés
—	11.18	Relances
—	11.19	Remboursements
—	11.20	Annulations
—	11.21	Régularisations
—	11.22	Clôtures
—	11.23	Rapports
—	11.24	Audit

3. Principe fondamental : aucun montant en dur

Il ne faut surtout pas écrire dans Flutter ou Django :

```
scolarité = 300000
```

Le montant doit provenir d'une grille tarifaire.

```
Année académique
    ↓
Formation
    ↓
Niveau
    ↓
Type de frais
    ↓
Tarif
```

4. Grille tarifaire

Objet :

```
TariffGrid
```

avec :

```
Année académique
Formation
Parcours
Niveau
Type de frais
Montant
Devise
Date début
```


Date fin
Statut

Exemple conceptuel :

2026-2027
Licence X
Niveau L1
Frais d'inscription
Montant : configurable

5. Types de frais

Le référentiel doit pouvoir contenir :

INSCRIPTION
SCOLARITÉ
FORMATION
DOSSIER
CONCOURS
EXAMEN
CARTE ÉTUDIANT
STAGE
SOUTENANCE
DIPLÔME
RELEVÉ
ATTESTATION
AUTRE

Les libellés doivent rester configurables.

6. Facturation

Lorsqu'un étudiant devient financièrement redevable :

Inscription
↓
Application tarifaire
↓
Facture

La facture contient :

Numéro
Étudiant
Inscription
Année
Formation
Lignes
Montant brut
Réduction

Exonération
Net à payer
Date émission
Échéance
Statut

7. Exemple de facture

INJS
FACTURE

Étudiant : XXXXX
Matricule : XXXXX
Formation : XXXXX
Année : 2026-2027

Frais d'inscription	XXX
Scolarité	XXX
Autre	XXX

Total	XXX
Réduction	XXX
Net à payer	XXX

Les montants ci-dessus sont volontairement des **variables de configuration**.

8. Statuts facture

BROUILLON
ÉMISE
PARTIELLEMENT_PAYÉE
PAYÉE
EN_RETARD
ANNULÉE
REMBOURSÉE
CLÔTURÉE

9. Échéancier

Une facture peut être divisée.

Exemple :

Total : X FCFA

Échéance 1 → X
Échéance 2 → X
Échéance 3 → X

Chaque échéance possède :

Montant
Date
Statut
Date paiement

10. Paiement

Objet :

Payment

Informations :

Numéro transaction
Facture
Étudiant
Montant
Date
Mode de paiement
Référence externe
Statut
Caissier / opérateur

11. Modes de paiement

Le système doit pouvoir gérer :

ESPÈCES
CHÈQUE
VIREMENT
CARTE BANCAIRE
ORANGE MONEY
MTN MONEY
MOOV MONEY
AUTRE

Les moyens réellement disponibles doivent être activables/désactivables dans les paramètres.

12. Paiement électronique

Pour un paiement mobile :

Étudiant
↓
Facture
↓

Choix moyen paiement
↓
Prestataire
↓
Transaction
↓
Confirmation
↓
Païement INJS
↓
Reçu

Il faut conserver la référence externe du prestataire.

13. Paiement partiel

Le système doit supporter :

Facture : 500 000
Païement : 200 000

Résultat :

Payé : 200 000
Reste : 300 000
Statut : PARTIELLEMENT_PAYÉE

14. Paiement excédentaire

Si :

Facture : 300 000
Païement : 350 000

le système doit détecter :

⚠ CRÉDIT ÉTUDIANT : 50 000

et permettre selon les règles :

Affectation à une autre facture
OU
Remboursement

15. Reçu

Après confirmation du paiement :

PAIEMENT
↓
VALIDATION
↓
REÇU

Le reçu possède :

Numéro unique
Date
Étudiant
Montant
Mode paiement
Référence transaction
Facture
Opérateur
QR éventuel

16. Annulation

Une transaction ne doit pas être simplement supprimée.

Paie ment
↓
Demande annulation
↓
Autorisation
↓
Annulation
↓
Écriture inverse / régularisation
↓
Audit

17. Exonérations

Créer :

Exemption

avec :

Étudiant
Type de frais
Taux ou montant
Motif
Justificatif
Date début
Date fin
Autorité
Statut

18. Réduction

Exemple :

Frais : 300 000
Réduction : 20 %

Net : 240 000

La réduction doit être historisée.

19. Bourses

Objet :

Scholarship

avec :

Étudiant
Organisme
Type
Montant
Période
Conditions
Statut
Justificatifs

Le système doit pouvoir distinguer :

Bourse totale
Bourse partielle
Prise en charge
Exonération

20. Impayés

Le système calcule automatiquement :

Total facturé
-
Total payé
=
Solde

Puis :

Solde > 0

+
échéance dépassée
=
IMPAYÉ

21. Relances

Workflow :

Échéance dépassée
↓
Alerte
↓
Relance 1
↓
Relance 2
↓
Relance 3

Les modèles de relance sont configurables.

22. Blocage administratif

Attention : le système ne doit pas coder arbitrairement :

étudiant impayé = interdiction automatique de tout service.

Il faut créer des **règles administratives configurables**.

Par exemple :

```
FinancialRule
├── Condition
├── Service concerné
├── Action
├── Exceptions
└── Date d'effet
```

Cela permet de respecter les décisions et règlements de l'établissement.

23. Solde étudiant

L'étudiant doit pouvoir consulter :

MA SITUATION FINANCIÈRE

Total facturé : XXX
Total payé : XXX
Solde : XXX

Prochaine échéance :
XX/XX/XXXX

24. Historique

2026-09-01
Facture créée

2026-09-03
Païement reçu

2026-09-03
Reçu généré

2026-10-01
Échéance

2026-10-05
Relance

25. Tableau de bord finance étudiants

FINANCES ÉTUDIANTS

Facturé	XXX
Encaissé	XXX
Reste à recouvrer	XXX

Taux recouvrement	XX %
-------------------	------

Factures :	
Payées	XXX
Partielles	XXX
Impayées	XX

26. Statistiques par formation

Formation A
Facturé : XXX
Payé : XXX
Impayé : XXX

Formation B
Facturé : XXX
Payé : XXX
Impayé : XXX

27. Statistiques par année

2024-2025
2025-2026
2026-2027

Le système doit pouvoir comparer :

Recettes
Taux de recouvrement
Impayés
Nombre d'étudiants

28. Caisse

Pour les paiements physiques :

CAISSE

Ouverture

↓

Encaissements

↓

Décaissements autorisés

↓

Contrôle

↓

Clôture

Une caisse doit avoir :

Caisse
Date
Responsable
Solde ouverture
Encaissements
Décaissements
Solde théorique
Solde réel
Écart

29. Contrôle de caisse

À la clôture :

Solde théorique : XXX
Solde réel : XXX
Écart : XXX

Tout écart doit être enregistré et justifié.

30. Connexion comptable

Un paiement étudiant ne doit pas seulement disparaître dans le module Finance.

Il doit pouvoir produire une information comptable :

```
Paie ment
↓
Recette
↓
Compte comptable
↓
Journal
↓
Trésorerie
```

C'est le lien avec le **MODULE 12 — DIRECTION FINANCIÈRE & COMPTABILITÉ**.

31. Architecture Django

```
finance/
├── tariff.py
├── invoice.py
├── invoice_line.py
├── payment.py
├── payment_method.py
├── receipt.py
├── installment.py
├── exemption.py
├── scholarship.py
├── refund.py
├── collection.py
├── reminder.py
├── cash_register.py
└── audit.py
```

32. API

```
/api/finance/tariffs/
/api/finance/invoices/
/api/finance/invoices/{id}/lines/
/api/finance/installments/

/api/finance/payments/
/api/finance/payments/{id}/confirm/
/api/finance/payments/{id}/cancel/

/api/finance/receipts/
```

/api/finance/refunds/

/api/finance/exemptions/
/api/finance/scholarships/

/api/finance/reminders/
/api/finance/cash-registers/

/api/finance/student/{id}/balance/
/api/finance/statistics/

33. Sécurité financière

Les opérations financières sensibles doivent être soumises à des permissions fortes :

CONSULTATION
SAISIE
VALIDATION
ANNULATION
REMBOURSEMENT
CLÔTURE

Il faut éviter qu'un même utilisateur puisse systématiquement :

créer paiement
+
valider paiement
+
annuler paiement
+
rembourser paiement

sans contrôle.

34. Séparation des responsabilités

Architecture recommandée :

CAISSIER
↓
SAISIT

RESPONSABLE FINANCE
↓
CONTRÔLE

COMPTABILITÉ
↓
COMPTABILISE

DIRECTION
↓

35. Principe d'intégrité

Ne jamais supprimer physiquement une opération financière validée.

Au lieu de :

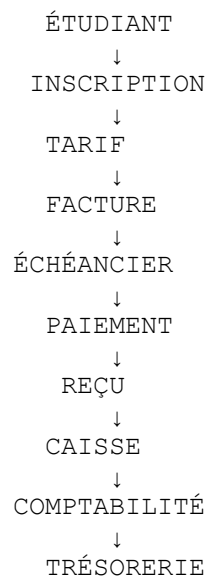
DELETE payment

utiliser :

ANNULATION
RÉGULARISATION
REMBOURSEMENT
ÉCRITURE INVERSE

avec audit.

36. Vue globale Finance



37. Ce que le Module 11 apporte à l'application

À ce stade, app-injs-lmd-2026 pourra suivre simultanément :



QUI ?
Étudiant



QUOI ?
Formation / diplôme



QUEL ENSEIGNEMENT ?
Modules / cours



QUAND ?
EDT / séances



OÙ ?
Campus / salle



PRÉSENCE ?
Émargement



QUEL RÉSULTAT ?
Notes / crédits



QUELLE SITUATION ?
Frais / paiements / solde



QUELLE DÉCISION ?
Jury / diplôme

Étape suivante : MODULE 12 — DIRECTION FINANCIÈRE & COMPTABILITÉ

Nous allons maintenant séparer clairement **la finance des étudiants** de la **comptabilité et de la gestion financière globale de l'INJS**.

Nous traiterons notamment :

budget → programmation → engagements → fournisseurs → dépenses → recettes → caisse → banques → trésorerie → comptabilité → pièces justificatives → rapprochement bancaire → immobilisations → états financiers → contrôle → audit, avec les liens automatiques vers les inscriptions, paiements, achats, patrimoine et administration.

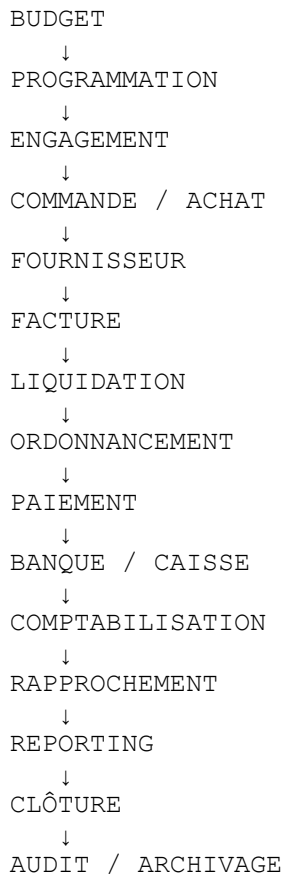
Parfait. Nous poursuivons avec le **MODULE 12 — DIRECTION FINANCIÈRE & COMPTABILITÉ**, en le séparant clairement du module précédent **FINANCES ÉTUDIANTS**.

MODULE 12 — DIRECTION FINANCIÈRE & COMPTABILITÉ

12.1. Objectif du module

Ce module assure la gestion financière et comptable globale de l'INJS.

Il ne gère pas uniquement les paiements des étudiants. Il couvre l'ensemble du cycle :



Le système doit également permettre de relier automatiquement les opérations financières provenant des autres modules :

- Finances étudiants ;
- Campus & patrimoine ;
- Administration ;
- Ressources humaines ;
- Achats ;
- Documents ;
- éventuellement projets/programmes spécifiques.

Important : les règles comptables et budgétaires exactes applicables à l'INJS doivent rester **paramétrables** dans le logiciel. Il ne faut pas coder en dur des règles réglementaires qui pourraient évoluer.

12.2. Sous-modules

Le module doit être organisé ainsi :

12. DIRECTION FINANCIÈRE & COMPTABILITÉ

- 12.1 Tableau de bord financier
 - 12.2 Exercices budgétaires
 - 12.3 Budgets
 - 12.4 Programmes et activités
 - 12.5 Lignes budgétaires
 - 12.6 Engagements
 - 12.7 Achats
 - 12.8 Demandes d'achat
 - 12.9 Bons de commande
 - 12.10 Fournisseurs
 - 12.11 Contrats fournisseurs
 - 12.12 Factures fournisseurs
 - 12.13 Dépenses
 - 12.14 Recettes
 - 12.15 Caisse
 - 12.16 Banques
 - 12.17 Trésorerie
 - 12.18 Paiements
 - 12.19 Comptabilité
 - 12.20 Plan comptable
 - 12.21 Journaux
 - 12.22 Écritures comptables
 - 12.23 Pièces justificatives
 - 12.24 Rapprochement bancaire
 - 12.25 Immobilisations
 - 12.26 Amortissements
 - 12.27 Clôtures
 - 12.28 Contrôles
 - 12.29 Audit
 - 12.30 Rapports financiers
 - 12.31 États financiers
 - 12.32 Archivage
-

12.3. Tableau de bord financier

Le tableau de bord de la Direction financière doit présenter notamment :

Budget

- budget initial ;
- budget révisé ;
- budget engagé ;
- budget liquidé ;
- budget ordonnancé ;
- budget payé ;
- disponible ;
- taux d'exécution.

Recettes

- recettes prévues ;
- recettes constatées ;
- recettes encaissées ;
- recettes restant à recouvrer.

Dépenses

- dépenses engagées ;
- dépenses réalisées ;
- dépenses payées ;
- dépenses en attente.

Trésorerie

- solde caisse ;
- solde bancaire ;
- total trésorerie ;
- mouvements récents ;
- prévisions d'encaissement ;
- prévisions de décaissement.

Fournisseurs

- factures reçues ;
- factures validées ;
- factures en attente ;
- factures échues ;
- montant dû.

Étudiants

Le module doit récupérer du **Module 11 — Finances étudiants** :

- montant facturé ;
- montant encaissé ;
- impayés ;
- avances ;
- remboursements.

12.4. Exercices budgétaires

Créer un référentiel :

ExerciceBudgétaire

Champs :

id
code
libellé
date_debut
date_fin
statut
date_ouverture
date_clôture
description

Statuts :

BROUILLON
OUVERT
EN_EXECUTION
EN_CLOTURE
CLOS
ARCHIVE

Un exercice clôturé ne doit plus pouvoir être modifié normalement.

12.5. Budgets

Le système doit permettre de créer plusieurs budgets.

Exemple :

Budget 2026
├── Fonctionnement
├── Investissement
├── Formation
├── Équipements
├── Infrastructures
└── Projets

Structure :

Budget
├── Exercice
├── Type
├── Code
├── Libellé
├── Montant initial
├── Révisions
├── Montant actuel
└── Statut

Types :

- fonctionnement ;
- investissement ;
- projet ;
- programme ;

- autre.

12.6. Lignes budgétaires

Chaque budget doit être détaillé en lignes.

Exemple :

Budget 2026

```
|
|— Personnel
|— Fournitures
|— Électricité
|— Eau
|— Télécommunications
|— Maintenance
|— Informatique
|— Équipements sportifs
|— Travaux
|— Missions
|— Autres dépenses
```

Une ligne doit notamment contenir :

```
code
budget
rubrique
programme
activité
montant_initial
révisions
montant_actuel
montant_engagé
montant_liquidé
montant_ordonnancé
montant_payé
solde_disponible
```

Calcul :

```
Solde disponible =
Budget actuel - Engagements
```

12.7. Engagements

Avant toute dépense significative, le système doit permettre d'enregistrer un engagement.

```
Engagement
|— numéro
|— exercice
|— ligne budgétaire
```

— bénéficiaire
— objet
— montant
— date
— demandeur
— validateur
— statut
— pièces justificatives

Statuts :

BROUILLON
SOUMIS
CONTRÔLÉ
VALIDÉ
REJETÉ
ANNULÉ
CLÔTURÉ

Le système doit empêcher automatiquement un engagement supérieur au disponible budgétaire, sauf règle spécifique autorisée.

12.8. Achats

Le module doit gérer le cycle achat :

Demande d'achat
↓
Validation
↓
Consultation / sélection
↓
Commande
↓
Réception
↓
Facture
↓
Validation
↓
Paieement

12.9. Demandes d'achat

Une demande peut être créée par :

- direction ;
- service ;
- département ;
- secrétariat ;

- responsable ;
- unité administrative.

Informations :

numéro
demandeur
service
date
objet
justification
articles
quantités
estimation
ligne budgétaire
priorité
statut

12.10. Bons de commande

Le système doit générer un numéro unique.

Informations :

numéro
fournisseur
date
demande d'origine
engagement
articles
quantités
prix
TVA si applicable
total
conditions
date livraison
statut

Statuts :

BROUILLON
VALIDÉ
ENVOYÉ
PARTIELLEMENT_LIVRÉ
LIVRÉ
ANNULÉ
CLOS

12.11. Fournisseurs

Créer un référentiel central :

Fournisseur

Informations :

- code fournisseur ;
- raison sociale ;
- nom commercial ;
- type ;
- adresse ;
- téléphone ;
- email ;
- identifiants administratifs ;
- coordonnées bancaires ;
- personne de contact ;
- statut ;
- documents ;
- historique.

Un fournisseur ne doit pas être recréé à chaque commande.

12.12. Factures fournisseurs

Cycle :

Réception
↓
Enregistrement
↓
Contrôle
↓
Rapprochement commande/réception
↓
Validation
↓
Liquidation
↓
Ordonnancement
↓
Paielement

Informations :

numéro facture
fournisseur
date facture
date réception
commande
engagement
montant HT
taxes
montant TTC
échéance

statut
pièce justificative

Le système doit détecter :

- facture en double ;
- montant incohérent ;
- fournisseur inexistant ;
- commande inexistante ;
- dépassement budgétaire ;
- facture déjà payée.

12.13. Dépenses

Toute dépense doit être traçable :

Budget
→ Engagement
→ Facture
→ Validation
→ Paiement
→ Comptabilisation

Aucune dépense validée ne doit pouvoir être supprimée physiquement.

Une annulation doit créer une opération corrective.

12.14. Recettes

Les recettes doivent provenir de plusieurs sources :

Recettes étudiants
Subventions
Dotations
Prestations
Locations
Partenariats
Projets
Dons
Autres recettes

Le système doit permettre de distinguer :

Recette prévue
Recette constatée
Recette encaissée
Recette restante

Les recettes issues du **Module 11** doivent pouvoir être automatiquement intégrées au module financier général.

12.15. Caisse

Gestion d'une ou plusieurs caisses :

Caisse principale
Caisse secondaire
Caisse événementielle
Autre

Cycle :

Ouverture
↓
Entrées
↓
Sorties
↓
Contrôle
↓
Inventaire
↓
Clôture

Le système doit calculer :

Solde théorique
vs
Solde physique

et détecter :

Excédent
Déficit
Écart

Toute différence doit être justifiée.

12.16. Banques

Créer le référentiel :

Compte bancaire

Informations :

- banque ;
- agence ;
- numéro/IBAN selon contexte ;
- devise ;
- titulaire ;
- solde initial ;
- solde comptable ;
- solde bancaire ;
- statut.

Les coordonnées bancaires sensibles doivent être protégées selon les droits utilisateurs.

12.17. Trésorerie

Le système doit offrir une vision consolidée :

```
CAISSE
+
BANQUES
+
ENCAISSEMENTS ATTENDUS
-
PAIEMENTS À VENIR
=
POSITION DE TRÉSORERIE
```

Fonctions :

- prévision de trésorerie ;
 - encaissements ;
 - décaissements ;
 - échéances ;
 - solde prévisionnel ;
 - alertes de tension de trésorerie.
-

12.18. Paiements

Un paiement peut concerner :

- fournisseur ;
- personnel ;
- remboursement étudiant ;
- prestataire ;
- organisme ;
- autre bénéficiaire.

Informations :

numéro
bénéficiaire
montant
date
mode
banque/caisse
référence
objet
pièce
statut

Modes :

- espèces ;
- chèque ;
- virement ;
- carte ;
- mobile money ;
- autre.

12.19. Comptabilité

La comptabilité doit être séparée de la simple gestion financière.

Architecture :

Opération financière
↓
Pièce justificative
↓
Contrôle
↓
Écriture comptable
↓
Journal
↓
Grand livre
↓
Balance
↓
États financiers

12.20. Plan comptable

Le plan comptable doit être **configurable**.

Structure :

```
Classe
├── Compte
│   └── Sous-compte
```

Exemple générique :

```
Classe
└── ↓
    Compte principal
        └── ↓
            Sous-compte
```

Ne pas figer dans le code les numéros comptables : ils doivent être stockés dans les référentiels.

12.21. Journaux

Prévoir notamment :

```
Journal de banque
Journal de caisse
Journal des achats
Journal des ventes/recettes
Journal des opérations diverses
Journal de paie
Autres journaux
```

Chaque écriture doit appartenir à un journal.

12.22. Écritures comptables

Structure :

```
Écriture
├── numéro
├── date
├── journal
├── exercice
├── référence opération
├── libellé
├── pièce justificative
├── lignes
│   ├── compte
│   ├── débit
│   └── crédit
```

Règle fondamentale :

TOTAL DÉBIT = TOTAL CRÉDIT

Le système doit refuser une écriture déséquilibrée.

12.23. Pièces justificatives

Toute opération financière importante doit pouvoir être accompagnée de documents :

- facture ;
- bon de commande ;
- bon de livraison ;
- reçu ;
- contrat ;
- autorisation ;
- justificatif bancaire ;
- pièce de caisse ;
- décision ;
- procès-verbal ;
- autre document.

Chaque document doit être :

identifié
versionné
daté
rattaché à l'opération
protégé
archivé

12.24. Rapprochement bancaire

Le système doit comparer :

Relevé bancaire
↑
Comptabilité

Il doit identifier :

- opérations correspondantes ;
- opérations manquantes ;
- opérations non rapprochées ;
- doublons ;
- écarts de montant ;
- écarts de date.

Statuts :

NON_RAPPROCHÉ

12.25. Immobilisations

Le module financier doit être connecté au **Module 07 — Campus & Patrimoine**.

Une immobilisation peut être :

- bâtiment ;
- terrain ;
- véhicule ;
- matériel informatique ;
- équipement sportif ;
- mobilier ;
- machine ;
- équipement technique ;
- autre actif.

Cycle :

Acquisition
↓
Enregistrement
↓
Mise en service
↓
Affectation
↓
Maintenance
↓
Inventaire
↓
Amortissement éventuel
↓
Sortie

Chaque immobilisation doit avoir un identifiant patrimonial unique.

12.26. Amortissements

Si applicable selon les règles comptables retenues, le système doit calculer :

- base amortissable ;
- durée ;
- méthode ;
- date de mise en service ;

- dotation ;
- cumul ;
- valeur nette.

Les paramètres doivent être configurables.

12.27. Clôture

Prévoir :

Clôture mensuelle

Contrôle
→ rapprochement
→ validation
→ clôture

Clôture annuelle

Contrôle général
↓
Rapprochements
↓
Inventaires
↓
Corrections
↓
Validation
↓
Clôture
↓
Archivage

Après clôture, les modifications doivent passer par un mécanisme contrôlé de rectification.

12.28. Contrôles internes

Le système doit appliquer une séparation des responsabilités.

Exemple :

Demandeur
↓
Responsable
↓
Finances
↓
Validation

↓
Comptabilité
↓
Païement
↓
Contrôle

Éviter qu'un seul utilisateur puisse :

créer
+
valider
+
payer
+
comptabiliser

une même opération sensible.

Les permissions doivent donc être basées sur les rôles.

12.29. Audit

Chaque opération financière doit conserver :

créateur
date création
modificateur
date modification
ancienne valeur
nouvelle valeur
action
IP/session si applicable
justification

Actions :

CRÉATION
MODIFICATION
VALIDATION
REJET
ANNULATION
PAIEMENT
RECTIFICATION
CLÔTURE
RÉOUVERTURE
ARCHIVAGE

Aucune suppression physique des opérations financières validées.

12.30. Rapports financiers

Prévoir au minimum :

Budget

- budget initial ;
- budget révisé ;
- exécution budgétaire ;
- engagements ;
- dépenses ;
- solde.

Recettes

- recettes par source ;
- recettes par période ;
- recettes par formation ;
- recettes étudiants ;
- impayés.

Dépenses

- dépenses par nature ;
- dépenses par service ;
- dépenses par fournisseur ;
- dépenses par ligne budgétaire ;
- dépenses par période.

Trésorerie

- caisse ;
- banques ;
- mouvements ;
- prévisions.

Comptabilité

- journaux ;
 - grand livre ;
 - balance ;
 - comptes ;
 - écritures ;
 - opérations non rapprochées.
-

12.31. Exports

Formats :

PDF
Excel
CSV

Les exports doivent utiliser les **codes et libellés métier**, jamais les IDs internes Django.

Exemple :



```
formation_id = 17
```



```
Formation = Licence Éducation et Motricité  
Code = L-EM
```

12.32. Architecture Django recommandée

Le module peut être réparti entre :

```
apps/  
├── finance/  
│   ├── budgets  
│   ├── purchases  
│   ├── suppliers  
│   ├── invoices  
│   ├── payments  
│   ├── cash  
│   ├── treasury  
│   └── financial_reports  
├── accounting/  
│   ├── accounts  
│   ├── journals  
│   ├── entries  
│   ├── reconciliation  
│   ├── closing  
│   └── accounting_reports  
└── audit/
```

12.33. Modèles principaux

Architecture logique :

FiscalYear
↓
Budget
↓
BudgetLine
↓
Commitment
↓
PurchaseRequest
↓
PurchaseOrder
↓
Supplier
↓
SupplierInvoice
↓
Payment
↓
Bank/Cash
↓
AccountingEntry
↓
Journal
↓
Account

Avec en parallèle :

StudentPayment
↓
Revenue
↓
AccountingEntry

et :

CampusAsset
↓
FixedAsset
↓
Depreciation
↓
AccountingEntry

12.34. APIs principales

Prévoir notamment :

/api/finance/fiscal-years/
/api/finance/budgets/
/api/finance/budget-lines/
/api/finance/commitments/

/api/finance/purchase-requests/
/api/finance/purchase-orders/

```
/api/finance/suppliers/  
/api/finance/supplier-contracts/  
/api/finance/supplier-invoices/  
  
/api/finance/expenses/  
/api/finance/revenues/  
  
/api/finance/payments/  
/api/finance/cash-registers/  
/api/finance/bank-accounts/  
/api/finance/treasury/  
  
/api/accounting/accounts/  
/api/accounting/journals/  
/api/accounting/entries/  
/api/accounting/reconciliation/  
/api/accounting/closures/  
  
/api/accounting/fixed-assets/  
/api/accounting/depreciation/  
  
/api/finance/reports/  
/api/accounting/reports/  
  
/api/finance/dashboard/  
/api/accounting/dashboard/
```

12.35. Relations avec les autres modules

Le module 12 devient un **hub financier** du système.

Module 11 — Finances étudiants

```
Facture étudiant  
↓  
Paielement étudiant  
↓  
Recette  
↓  
Comptabilité
```

Module 07 — Campus & Patrimoine

```
Acquisition équipement  
↓  
Immobilisation  
↓  
Inventaire  
↓  
Amortissement  
↓  
Comptabilité
```

Module 14 — Administration

Décision
Autorisation
Contrat
Service
↓
Opération financière

Module 05 — Enseignants / RH

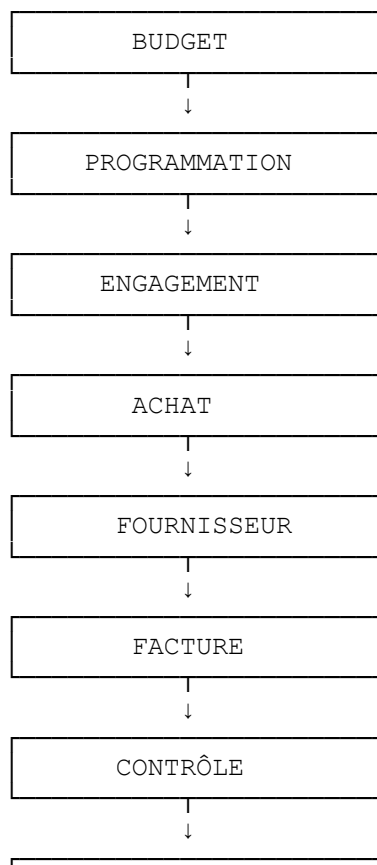
Rémunération
Honoraires
Missions
Indemnités
↓
Finance
↓
Comptabilité

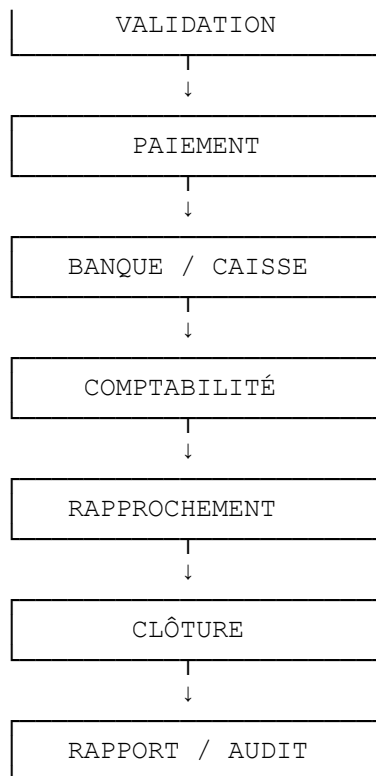
Module 15 — Documents

Toutes les pièces financières doivent pouvoir être rattachées au système documentaire.

12.36. Workflow global

Le workflow financier général doit finalement être :





12.37. Principe architectural essentiel

Il faut absolument maintenir la séparation :

Module 11 — FINANCES ÉTUDIANTS

« Combien cet étudiant doit-il et combien a-t-il payé ? »

versus

Module 12 — DIRECTION FINANCIÈRE & COMPTABILITÉ

« Comment l'ensemble des opérations financières de l'INJS est-il budgété, engagé, encaissé, payé, comptabilisé, contrôlé et clôturé ? »

Le paiement étudiant constitue donc **une source d'opération financière**, mais ne doit pas être confondu avec toute la comptabilité de l'établissement.

Prochaine étape logique : MODULE 13 — ADMINISTRATION & RESSOURCES HUMAINES, avec la gestion des personnels, services, directions, affectations, postes, contrats, congés, absences, missions, carrières, dossiers administratifs et workflow de validation.

Parfait. Nous poursuivons avec le **MODULE 13 — ADMINISTRATION & RESSOURCES HUMAINES**, en conservant la même logique : architecture complète,

opérationnelle, configurable et directement exploitable ensuite pour l'implémentation Django/Flutter.

MODULE 13 — ADMINISTRATION & RESSOURCES HUMAINES

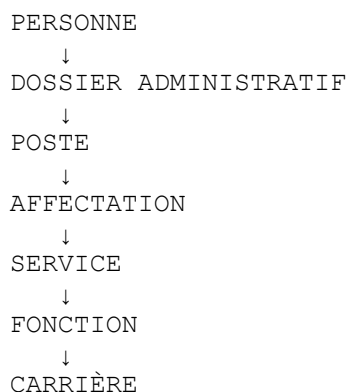
13.1. Objectif

Ce module constitue le système central de gestion administrative de l'INJS.

Il doit gérer :

- l'organisation administrative ;
- les directions ;
- les services ;
- les départements ;
- les unités ;
- les personnels ;
- les enseignants sur leur volet administratif ;
- les contrats ;
- les postes ;
- les affectations ;
- les carrières ;
- les congés ;
- les absences ;
- les missions ;
- les formations du personnel ;
- les dossiers administratifs ;
- les décisions ;
- les actes administratifs ;
- les workflows de validation ;
- les archives RH ;
- les statistiques du personnel.

Il faut distinguer :



et, pour les enseignants :

PERSONNEL
↓
DOSSIER RH
+
DOSSIER ENSEIGNANT
↓
AFFECTATIONS ACADÉMIQUES

13.2. Sous-modules

13. ADMINISTRATION & RESSOURCES HUMAINES

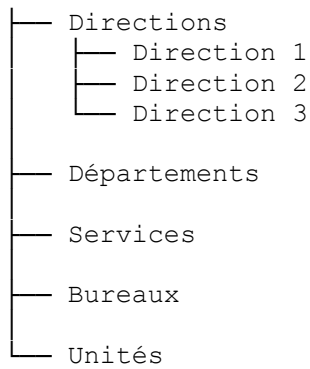
- 13.1 Organisation administrative
- 13.2 Directions
- 13.3 Départements
- 13.4 Services
- 13.5 Unités
- 13.6 Postes
- 13.7 Fonctions
- 13.8 Personnel
- 13.9 Dossiers administratifs
- 13.10 Contrats
- 13.11 Affectations
- 13.12 Carrières
- 13.13 Grades / situations administratives
- 13.14 Congés
- 13.15 Absences
- 13.16 Missions
- 13.17 Ordres de mission
- 13.18 Formations du personnel
- 13.19 Évaluations professionnelles
- 13.20 Sanctions / récompenses
- 13.21 Documents administratifs
- 13.22 Décisions administratives
- 13.23 Workflow de validation
- 13.24 Archives RH
- 13.25 Statistiques RH
- 13.26 Tableau de bord RH
- 13.27 Contrôle et audit

13.3. Organisation administrative

L'application doit permettre de représenter l'organisation réelle de l'INJS sans coder sa structure en dur.

Hierarchie proposée :

INJS
|
├── Direction Générale



La structure doit être configurable.

13.4. Structure organisationnelle

Créer un modèle générique :

`OrganizationalUnit`

Champs :

`id`
`code`
`nom`
`type`
`parent`
`responsable`
`description`
`telephone`
`email`
`localisation`
`date_creation`
`date_fin`
`statut`

Types :

`DIRECTION`
`DEPARTEMENT`
`SERVICE`
`BUREAU`
`UNITE`
`CELLULE`
`SECRETARIAT`
`AUTRE`

Le champ `parent` permet de créer une structure hiérarchique.

13.5. Directions

Une direction peut avoir :

- directeur ;
- directeur adjoint ;
- secrétariat ;
- services ;
- personnels ;
- budget ;
- documents ;
- activités.

Exemple :

```
Direction
├── Responsable
├── Adjoint
├── Secrétariat
├── Services
└── Personnel
```

13.6. Services

Chaque service doit disposer de :

```
code
nom
direction
responsable
adjoint
localisation
telephone
email
effectif
statut
```

Le système doit pouvoir afficher automatiquement l'organigramme.

13.7. Postes

Un **poste** représente une position dans l'organisation.

Exemple générique :

```
POSTE
├── code
```


—	intitulé
—	service
—	fonction
—	catégorie
—	nombre de postes
—	statut
—	description

Il faut distinguer :

POSTE
≠
PERSONNE

Un poste peut exister avant d’être occupé.

13.8. Fonctions

Créer un référentiel :

Fonction

Exemples génériques :

- Directeur ;
- Directeur adjoint ;
- Chef de service ;
- Responsable ;
- Secrétaire ;
- Agent administratif ;
- Enseignant ;
- Formateur ;
- Technicien ;
- Comptable ;
- Informaticien ;
- Agent d’accueil ;
- Agent d’entretien ;
- etc.

Les intitulés réels doivent rester configurables.

13.9. Personnel

Le modèle central :

Personnel

Informations :

Identité

matricule
nom
prénoms
date_naissance
lieu_naissance
sexe
nationalité
photo

Coordonnées

adresse
telephone
email
contact_urgence

Administration

matricule_administratif
date_entree
date_recrutement
statut
type_personnel

Situation

fonction
poste
service
direction
responsable

13.10. Types de personnel

Référentiel configurable :

FONCTIONNAIRE
AGENT_CONTRACTUEL
AGENT_TEMPORAIRE
VACATAIRE
ENSEIGNANT
ENSEIGNANT_VACATAIRE
PERSONNEL_ADMINISTRATIF
PERSONNEL_TECHNIQUE
PERSONNEL_DE_SOUTIEN
STAGIAIRE
AUTRE

Un même personnel peut avoir plusieurs rôles fonctionnels, mais une seule fiche administrative principale.

13.11. Dossier administratif

Chaque personnel dispose d'un dossier numérique :

DOSSIER RH

- Identité
- Situation administrative
- Recrutement
- Contrats
- Affectations
- Carrière
- Congés
- Absences
- Missions
- Formations
- Évaluations
- Décisions
- Documents
- Archives

13.12. Contrats

Créer :

ContratPersonnel

Informations :

numero
personnel
type
date_debut
date_fin
fonction
poste
service
remuneration
conditions
document
statut

Statuts :

BROUILLON
ACTIF
SUSPENDU
EXPIRÉ

RÉSILIÉ
CLOS

Alertes automatiques avant expiration.

13.13. Affectations

Une personne peut évoluer dans l'organisation.

Il faut donc conserver l'historique :

```
Personnel
↓
Affectation 1
↓
Affectation 2
↓
Affectation 3
```

Une affectation doit contenir :

```
personnel
poste
fonction
service
direction
date_debut
date_fin
motif
acte_affectation
statut
```

Ne jamais écraser l'ancienne affectation.

13.14. Carrière

Le système doit conserver l'historique professionnel :

```
Recrutement
↓
Affectation
↓
Promotion
↓
Mutation
↓
Changement de fonction
↓
Nouvelle affectation
```

↓
Situation actuelle

Événements :

- recrutement ;
- titularisation si applicable ;
- nomination ;
- promotion ;
- avancement ;
- mutation ;
- détachement ;
- mise à disposition ;
- suspension ;
- reprise ;
- départ.

Les règles exactes doivent être configurables selon le statut du personnel.

13.15. Situation administrative

Prévoir un référentiel :

SituationAdministrative

Exemples :

ACTIF
CONGÉ
MISSION
FORMATION
SUSPENDU
ABSENT
DÉTACHÉ
MIS À DISPOSITION
DISPONIBILITÉ
SORTI
AUTRE

13.16. Congés

Le système doit gérer :

Demande
↓
Contrôle des droits
↓
Validation responsable
↓

Validation RH

↓

Décision

↓

Calendrier

Une demande contient :

personnel
type_congé
date_debut
date_fin
nombre_jours
motif
justificatif
remplaçant
statut

Statuts :

BROUILLON
SOUMIS
EN_ATTENTE
VALIDÉ
REFUSÉ
ANNULÉ
CLOS

13.17. Types de congés

Référentiel configurable :

CONGÉ_ANNUEL
CONGÉ_MALADIE
CONGÉ_MATERNITÉ
CONGÉ_EXCEPTIONNEL
CONGÉ_FORMATION
AUTRE

Les droits et règles de calcul ne doivent pas être codés en dur.

13.18. Absences

Distinguer :

ABSENCE
≠
CONGÉ

Une absence peut être :

- justifiée ;
- non justifiée ;
- autorisée ;
- non autorisée.

Le système doit enregistrer :

date
 heure début
 heure fin
 personnel
 motif
 justificatif
 statut
 validateur

13.19. Missions

Gestion des missions administratives :

Mission

- personnel
- objet
- destination
- date départ
- date retour
- moyen transport
- ordre de mission
- budget
- frais
- statut
- rapport

Cycle :

Demande
 ↓
 Validation
 ↓
 Ordre de mission
 ↓
 Départ
 ↓
 Mission
 ↓
 Retour
 ↓
 Rapport
 ↓
 Liquidation des frais

13.20. Ordres de mission

Le système doit pouvoir générer un document officiel avec :

- numéro ;
- agent ;
- fonction ;
- objet ;
- destination ;
- période ;
- autorité ;
- moyens ;
- prise en charge ;
- signatures ;
- QR de vérification éventuellement.

13.21. Formations du personnel

Créer :

FormationPersonnel

Permet de suivre :

- formations suivies ;
- formations demandées ;
- formations validées ;
- organismes ;
- dates ;
- coûts ;
- attestations ;
- compétences acquises.

Workflow :

Besoin
↓
Demande
↓
Validation
↓
Inscription
↓
Formation
↓
Attestation
↓
Archivage

13.22. Évaluations professionnelles

Le système peut gérer les évaluations périodiques :

Période
↓
Objectifs
↓
Évaluation
↓
Entretien
↓
Validation
↓
Plan d'amélioration

Informations :

- évaluateur ;
- agent ;
- période ;
- objectifs ;
- résultats ;
- appréciations ;
- compétences ;
- observations ;
- décision.

Les grilles doivent être configurables.

13.23. Sanctions et récompenses

Créer un historique administratif.

Récompenses

- félicitations ;
- distinction ;
- attestation ;
- reconnaissance.

Sanctions

- avertissement ;
- blâme ;
- autre mesure prévue par les règles applicables.

Les informations sensibles doivent être fortement protégées.

Toute décision doit être rattachée à un document officiel.

13.24. Documents administratifs

Le dossier RH doit permettre de stocker :

Acte de recrutement
Contrat
Décision
Affectation
Promotion
Congé
Ordre de mission
Attestation
Diplôme
Formation
Évaluation
Correspondance
Autre

Chaque document :

type
numéro
date
personnel
fichier
version
statut
auteur
date_upload

13.25. Décisions administratives

Créer un modèle :

AdministrativeDecision

Cycle :

Brouillon
↓
Soumission
↓
Contrôle
↓
Validation
↓
Signature
↓
Notification

↓
Archivage

13.26. Workflow de validation

Le workflow doit être configurable.

Exemple :

```
AGENT
↓
CHEF DE SERVICE
↓
DIRECTION
↓
RH
↓
DIRECTION GÉNÉRALE
```

Selon la nature du dossier, le circuit peut changer.

Il ne faut donc pas coder un workflow unique.

13.27. Signature et validation

Pour les documents sensibles :

```
Création
↓
Validation
↓
Signature
↓
Horodatage
↓
Version finale
↓
Archivage
```

Une version signée ne doit pas être silencieusement remplacée.

13.28. Gestion des enseignants

Le module 13 gère le **volet administratif** des enseignants.

Le module 05 gère leur **activité académique**.

Ainsi :

MODULE 13
Personnel
Contrat
Affectation
Carrière
Congés
Mission
Situation administrative
|
↓
MODULE 05
Enseignant
Formation
Module enseigné
Groupes
CM / TD / TP
Charge horaire
Notes

Cela évite de dupliquer la fiche enseignant.

13.29. Liaison avec la paie/finance

Le module doit pouvoir transmettre les données nécessaires au Module 12 :

Personnel
↓
Situation
↓
Contrat
↓
Rémunération
↓
Éléments variables
↓
Finance / Comptabilité

Exemples :

- salaires ;
- vacations ;
- heures supplémentaires ;
- missions ;
- indemnités ;
- remboursements.

Les montants et règles de calcul doivent être configurables.

13.30. Tableau de bord RH

Le tableau de bord doit présenter :

Effectifs

Personnel total
Personnel actif
Enseignants
Administratifs
Techniques
Contractuels
Autres

Répartition

- par direction ;
- par service ;
- par fonction ;
- par statut ;
- par type de contrat.

Mouvements

- recrutements ;
- départs ;
- mutations ;
- promotions ;
- affectations.

Congés

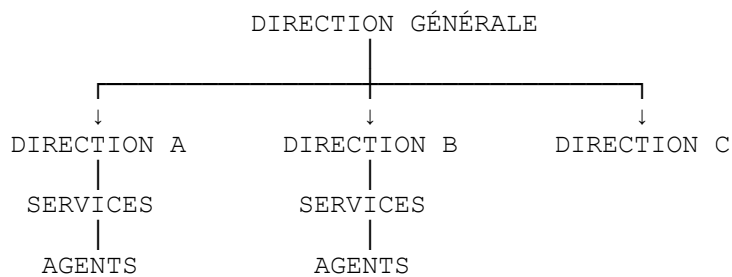
- demandes ;
- validées ;
- refusées ;
- en cours.

Alertes

- contrats arrivant à échéance ;
- documents manquants ;
- affectations expirantes ;
- formations à renouveler ;
- dossiers incomplets.

13.31. Organigramme dynamique

L'application doit pouvoir générer :



L'organigramme doit être alimenté directement par les données du référentiel administratif.

13.32. API principales

Prévoir :

```
/api/hr/organization/  
/api/hr/directions/  
/api/hr/departments/  
/api/hr/services/  
/api/hr/units/
```

```
/api/hr/positions/  
/api/hr/functions/
```

```
/api/hr/personnel/  
/api/hr/personnel/{id}/profile/  
/api/hr/personnel/{id}/documents/  
/api/hr/personnel/{id}/career/
```

```
/api/hr/contracts/  
/api/hr/assignments/  
/api/hr/career-events/
```

```
/api/hr/leave-types/  
/api/hr/leaves/  
/api/hr/absences/
```

```
/api/hr/missions/  
/api/hr/mission-orders/
```

```
/api/hr/training/  
/api/hr/evaluations/
```

```
/api/hr/decisions/  
/api/hr/workflows/
```

```
/api/hr/dashboard/  
/api/hr/statistics/
```

13.33. Modèles Django principaux

Architecture recommandée :

```
apps/  
└─ hr/  
    ├── organization  
    ├── personnel  
    ├── positions  
    ├── contracts  
    ├── assignments  
    ├── career  
    ├── leave  
    ├── absence  
    ├── missions  
    ├── training  
    ├── evaluations  
    ├── decisions  
    ├── documents  
    ├── workflows  
    └── reports
```

Modèles principaux :

```
OrganizationUnit  
Position  
Function  
Personnel  
PersonnelDocument  
EmploymentContract  
Assignment  
CareerEvent  
AdministrativeStatus  
LeaveType  
LeaveRequest  
Absence  
Mission  
MissionOrder  
Training  
ProfessionalEvaluation  
AdministrativeDecision  
Workflow  
WorkflowStep
```

13.34. Sécurité et confidentialité

Le module RH contient des données particulièrement sensibles.

Il faut donc appliquer les droits N0–N4 déjà définis dans le système.

Exemple :

```
N0 → accès très limité  
N1 → consultation selon périmètre  
N2 → gestion opérationnelle  
N3 → validation / supervision  
N4 → administration
```

Mais surtout :

un utilisateur ne doit accéder qu'aux données nécessaires à sa fonction.

Prévoir également :

- journalisation ;
- historique ;
- contrôle des téléchargements ;
- restrictions sur les documents ;
- traçabilité des consultations sensibles ;
- gestion des versions ;
- archivage.

13.35. Règle d'intégrité

Une règle fondamentale :

PERSONNEL
↓
UN SEUL DOSSIER RH CENTRAL
↓
PLUSIEURS AFFECTATIONS HISTORIQUES
↓
PLUSIEURS CONTRATS SI NÉCESSAIRE
↓
PLUSIEURS ÉVÉNEMENTS DE CARRIÈRE

Il ne faut jamais créer une nouvelle fiche personnel simplement parce que l'agent change de service ou de fonction.

13.36. Workflow global RH

RECRUTEMENT
↓
CRÉATION DOSSIER
↓
CONTRAT / STATUT
↓
AFFECTATION
↓
PRISE DE FONCTION
↓
ACTIVITÉ
↓
CONGÉS / ABSENCES
↓
MISSIONS / FORMATIONS

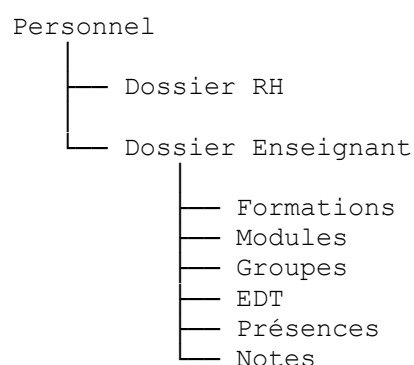
↓
ÉVALUATION
↓
CARRIÈRE
↓
MUTATION / PROMOTION
↓
NOUVELLE AFFECTATION
↓
DÉPART
↓
ARCHIVAGE

13.37. Principe de séparation des modules

Il faut conserver cette architecture :

Module	Responsabilité
05 — Enseignants	Activité pédagogique de l'enseignant
12 — Finance & Comptabilité	Flux financiers et comptables
13 — Administration & RH	Situation administrative du personnel
07 — Campus & Patrimoine	Locaux, équipements et patrimoine
15 — Documents	Gestion documentaire transverse
18 — Administration système	Comptes, rôles, permissions et sécurité

Ainsi, une même personne peut être :



sans duplication de l'identité.

Prochaine étape

MODULE 14 — DOCUMENTS, COURRIERS & ARCHIVES, couvrant la GED de l'INJS, courriers entrants/sortants, parapheur, visa, signature, modèles de documents, numérotation, QR de vérification, classement, versions, archivage et recherche documentaire.

Parfait. Nous poursuivons avec le **MODULE 14 — DOCUMENTS, COURRIERS & ARCHIVES**, conçu comme une **GED transverse** de l'INJS, et non comme un simple espace de stockage de fichiers.

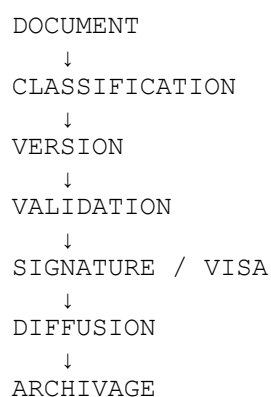
MODULE 14 — DOCUMENTS, COURRIERS & ARCHIVES

14.1. Objectif

Le module doit centraliser la gestion du patrimoine documentaire de l'INJS :

- documents administratifs ;
- documents académiques ;
- documents financiers ;
- documents RH ;
- courriers entrants ;
- courriers sortants ;
- notes ;
- décisions ;
- arrêtés ;
- contrats ;
- conventions ;
- procès-verbaux ;
- rapports ;
- attestations ;
- diplômes ;
- pièces justificatives ;
- archives.

Le principe central :



Le module doit être utilisé par les autres modules de l'application.

14.2. Sous-modules

14. DOCUMENTS, COURRIERS & ARCHIVES

- 14.1 GED
- 14.2 Types de documents
- 14.3 Catégories documentaires
- 14.4 Classement
- 14.5 Documents
- 14.6 Versions
- 14.7 Pièces jointes
- 14.8 Modèles de documents
- 14.9 Numérotation
- 14.10 Courrier entrant
- 14.11 Courrier sortant
- 14.12 Courrier interne
- 14.13 Affectation du courrier
- 14.14 Visa
- 14.15 Validation
- 14.16 Signature
- 14.17 Diffusion
- 14.18 Notifications
- 14.19 Parapheur
- 14.20 Recherche documentaire
- 14.21 Indexation
- 14.22 Archivage
- 14.23 Conservation
- 14.24 Destruction contrôlée
- 14.25 Vérification QR
- 14.26 Traçabilité
- 14.27 Statistiques

14.3. GED centrale

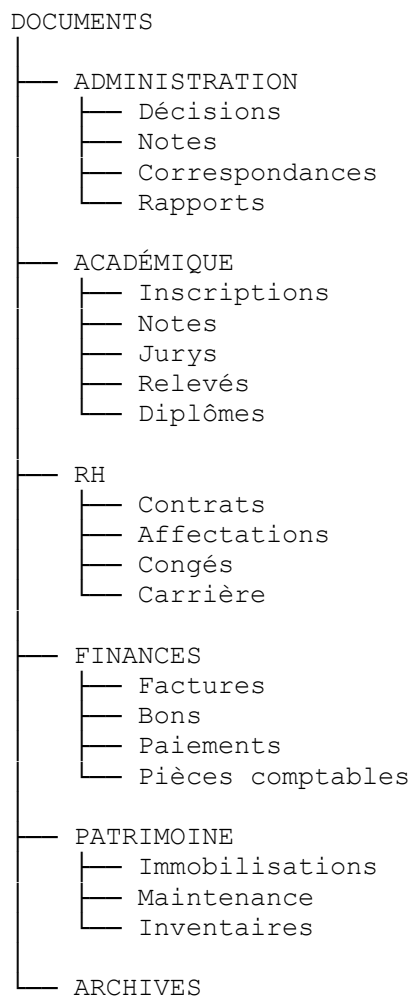
La GED doit permettre de retrouver rapidement un document à partir de :

- numéro ;
- titre ;
- type ;
- catégorie ;
- auteur ;
- service ;
- direction ;
- personne concernée ;
- étudiant ;
- enseignant ;
- fournisseur ;
- exercice ;

- formation ;
- année académique ;
- date ;
- mots-clés ;
- statut.

14.4. Classification documentaire

Créer une arborescence configurable :



Cette arborescence doit être modifiable par les administrateurs autorisés.

14.5. Types de documents

Le référentiel doit contenir notamment :

COURRIER_ENTRANT
COURRIER_SORTANT
NOTE
NOTE_DE_SERVICE
DECISION
CONVOCATION
RAPPORT
PROCES_VERBAL
CONTRAT
CONVENTION
FACTURE
RECU
ATTESTATION
CERTIFICAT
RELEVE
DIPLOME
JUSTIFICATIF
ORDRE_DE_MISSION
ACTE_ADMINISTRATIF
DOCUMENT_ACADEMIQUE
DOCUMENT_RH
DOCUMENT_FINANCIER
AUTRE

Les types réels doivent être configurables.

14.6. Modèle Document

Créer un objet central :

Document

Champs :

code
numero
titre
type
categorie
sous_categorie
description
auteur
service
date_document
date_creation
date_modification
statut
confidentialite
version_courante
fichier_principal

14.7. Statuts

Prévoir :

BROUILLON
EN_COURS
SOUMIS
EN_REVISION
EN_VALIDATION
VALIDÉ
SIGNÉ
PUBLIÉ
DIFFUSÉ
ARCHIVÉ
ANNULÉ
REEMPLACÉ

14.8. Confidentialité

Chaque document doit avoir un niveau de confidentialité.

Exemple :

PUBLIC
INTERNE
CONFIDENTIEL
TRÈS_CONFIDENTIEL

Les droits d'accès doivent être déterminés par :

Utilisateur
+
Rôle
+
Service
+
Type document
+
Niveau confidentialité
+
Périmètre

14.9. Versions

Un document ne doit pas être écrasé.

Exemple :

Document D-2026-001
|
— V1
— V2
— V3
— V4 ← version actuelle

Chaque version conserve :

- auteur ;
- date ;
- fichier ;
- motif de modification ;
- statut.

Une version signée doit rester figée.

14.10. Pièces jointes

Un document peut comporter plusieurs pièces :

```
Document principal
├── Annexe 1
├── Annexe 2
├── Justificatif
└── Autre pièce
```

Chaque pièce doit être identifiable.

14.11. Modèles de documents

Créer un système de modèles.

Exemples :

- attestation de scolarité ;
- certificat ;
- convocation ;
- décision ;
- ordre de mission ;
- relevé de notes ;
- procès-verbal ;
- lettre ;
- courrier administratif ;
- certificat de réussite.

Le système injecte automatiquement les données.

Exemple :

```
{{ nom }}
{{ prenom }}
```

```
{{ matricule }}  
{{ formation }}  
{{ année_académique }}  
{{ date }}
```

14.12. Numérotation automatique

Le système doit gérer les numéros officiels.

Exemple :

```
DEC-2026-0001  
DEC-2026-0002  
DEC-2026-0003
```

Ou :

```
INJS/DG/2026/0001
```

Le format doit être configurable par type de document.

Paramètres :

- préfixe ;
- année ;
- compteur ;
- suffixe ;
- service ;
- séparateur.

14.13. Courrier entrant

Workflow :

```
Réception  
↓  
Enregistrement  
↓  
Numérisation  
↓  
Classification  
↓  
Affectation  
↓  
Traitement  
↓  
Réponse  
↓  
Classement
```


↓
Archivage

Informations :

numéro_arrivée
date_réception
expéditeur
objet
destinataire
service
priorité
référence
pièces_jointes
statut

14.14. Courrier sortant

Workflow :

Rédaction
↓
Contrôle
↓
Visa
↓
Signature
↓
Numérotation
↓
Envoi
↓
Preuve d'envoi
↓
Archivage

Informations :

- numéro ;
 - destinataire ;
 - objet ;
 - référence ;
 - auteur ;
 - service ;
 - date ;
 - moyen d'envoi ;
 - pièces ;
 - statut.
-

14.15. Courrier interne

Le système doit permettre :

Direction A
↓
Service B
↓
Agent C

avec :

- note interne ;
- transmission ;
- instruction ;
- accusé de réception ;
- commentaire ;
- échéance.

14.16. Affectation du courrier

Un courrier peut être affecté à :

- une direction ;
- un service ;
- un responsable ;
- un agent.

Exemple :

COURRIER
↓
DIRECTION
↓
SERVICE
↓
RESPONSABLE
↓
AGENT

Chaque transfert doit être tracé.

14.17. Visa

Le visa permet à une personne autorisée de contrôler le document avant validation.

Statuts :

EN_ATTENTE

VISÉ
REFUSÉ
AVEC_OBSERVATIONS

Le visa doit enregistrer :

utilisateur
date
heure
décision
commentaire

14.18. Validation

Workflow générique :

Créateur
↓
Contrôleur
↓
Responsable
↓
Direction
↓
Validation finale

Le workflow dépend du type de document.

14.19. Signature

Prévoir une couche de signature configurable.

Le système doit distinguer :

VISA
≠
VALIDATION
≠
SIGNATURE

Une signature doit conserver :

- signataire ;
- date ;
- heure ;
- document signé ;
- version ;
- statut.

14.20. Parapheur électronique

Le parapheur doit présenter les documents en attente :

À SIGNER
À VISER
À VALIDER
À REJETER
EN RETARD

Pour chaque document :

Objet
Auteur
Service
Date
Échéance
Priorité
Action

14.21. Diffusion

Après validation/signature :

Document
↓
Destinataires
↓
Notification
↓
Consultation
↓
Accusé réception

Le système doit conserver la preuve de diffusion.

14.22. Recherche documentaire

La recherche doit être multicritère.

Exemple :

Numéro
Titre
Type
Date
Service

Auteur
Formation
Étudiant
Fournisseur
Année
Mot-clé

Filtres :

- période ;
- statut ;
- confidentialité ;
- service ;
- type ;
- auteur.

14.23. Indexation

Prévoir :

- mots-clés ;
- tags ;
- catégories ;
- références ;
- entités liées.

Exemple :

Document
|— étudiant
|— formation
|— année académique
|— service
|— type

Cela permet de retrouver tous les documents liés à une même entité.

14.24. OCR

Pour les documents numérisés, prévoir éventuellement :

PDF / Image
↓
OCR
↓
Texte indexable
↓

Recherche

L'OCR doit être considéré comme une fonction configurable.

Le fichier original doit toujours être conservé.

14.25. Archivage

Cycle :

```
ACTIF
↓
ARCHIVAGE
↓
ARCHIVÉ
↓
CONSERVATION
↓
SORT FINAL
```

Le système doit conserver :

- date d'archivage ;
 - responsable ;
 - emplacement logique ;
 - durée de conservation ;
 - motif ;
 - statut.
-

14.26. Durée de conservation

Prévoir un référentiel :

```
Type document
→ durée de conservation
→ règle
→ sort final
```

Les durées réglementaires exactes doivent être paramétrables et validées par l'administration compétente.

14.27. Destruction contrôlée

Aucun document ne doit être supprimé directement.

Workflow :

```
Proposition
↓
Contrôle
↓
Autorisation
↓
Procès-verbal
↓
Destruction
↓
Trace d'archivage
```

La trace de l'opération doit rester dans l'audit.

14.28. QR Code de vérification

Les documents officiels peuvent comporter un QR permettant de vérifier leur authenticité.

Exemple :

```
Document officiel
↓
QR
↓
Portail de vérification
↓
Référence
↓
Authenticité
```

La page publique ne doit exposer que les informations minimales nécessaires.

Exemple :

```
Document authentique
Référence : INJS/DIP/2026/000245
Type : Attestation
Date : 03/09/2026
Statut : VALIDE
```

Ne jamais exposer les IDs internes Django.

14.29. Documents académiques

Le module doit recevoir les documents générés par :

Admissions

- convocations ;
- décisions d'admission.

Scolarité

- certificats ;
- attestations ;
- cartes.

Évaluations

- relevés ;
- résultats.

Jurys

- PV ;
- décisions.

Diplômes

- diplômes ;
- duplicatas ;
- attestations de réussite.

14.30. Documents financiers

Connexion avec les Modules 11 et 12 :

Facture étudiant
Paie
Reçu
Facture fournisseur
Bon de commande
Pièce comptable
Relevé bancaire
Justificatif

Tous peuvent être stockés dans la GED.

14.31. Documents RH

Connexion au Module 13 :

Contrat
Affectation
Décision
Congé
Ordre de mission
Attestation
Évaluation
Formation
Acte administratif

14.32. Documents patrimoine

Connexion au Module 07 :

Fiche équipement
Facture
Garantie
Maintenance
Inventaire
Réforme
Procès-verbal

14.33. Sécurité documentaire

Le système doit appliquer :

RBAC
+
Permissions par module
+
Permissions par service
+
Confidentialité
+
Historique
+
Versioning
+
Audit

Prévoir notamment :

- consultation ;
- téléchargement ;
- modification ;
- suppression logique ;
- validation ;
- signature ;
- archivage.

14.34. Audit documentaire

Chaque action doit être journalisée :

CRÉATION
CONSULTATION
TÉLÉCHARGEMENT
MODIFICATION
NOUVELLE_VERSION
VISA
VALIDATION
SIGNATURE
DIFFUSION
ARCHIVAGE
RESTAURATION
DESTRUCTION

14.35. Tableau de bord

Le tableau de bord doit afficher :

Courrier

- courriers reçus ;
- courriers à traiter ;
- courriers en retard ;
- courriers sortants.

Documents

- documents créés ;
- en validation ;
- à signer ;
- signés ;
- archivés.

Parapheur

À VISER	12
À VALIDER	8
À SIGNER	5
EN RETARD	3

Archives

- documents archivés ;
- documents arrivant en fin de conservation ;

- documents nécessitant une action.

14.36. Modèles Django

Architecture recommandée :

```
apps/  
├── documents/  
│   ├── models/  
│   │   ├── document.py  
│   │   ├── document_version.py  
│   │   ├── document_type.py  
│   │   ├── category.py  
│   │   ├── attachment.py  
│   │   ├── template.py  
│   │   ├── correspondence.py  
│   │   ├── workflow.py  
│   │   ├── signature.py  
│   │   ├── distribution.py  
│   │   └── archive.py  
│   ├── api/  
│   ├── services/  
│   ├── permissions/  
│   └── reports/
```

Modèles principaux :

```
Document  
DocumentType  
DocumentCategory  
DocumentVersion  
DocumentAttachment  
DocumentTemplate  
DocumentNumberingRule
```

```
IncomingMail  
OutgoingMail  
InternalMail  
MailAssignment
```

```
DocumentWorkflow  
WorkflowStep  
DocumentVisa  
DocumentValidation  
DocumentSignature  
DocumentDistribution
```

```
ArchiveRecord  
RetentionRule  
DestructionRecord
```

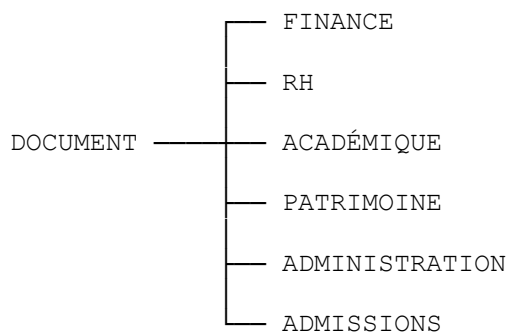
14.37. APIs principales

```
/api/documents/  
/api/documents/types/  
/api/documents/categories/  
/api/documents/templates/  
/api/documents/numbering/  
  
/api/documents/versions/  
/api/documents/attachments/  
  
/api/mail/incoming/  
/api/mail/outgoing/  
/api/mail/internal/  
/api/mail/assignments/  
  
/api/documents/workflows/  
/api/documents/visa/  
/api/documents/validation/  
/api/documents/signatures/  
/api/documents/distribution/  
  
/api/documents/search/  
/api/documents/archive/  
/api/documents/retention/  
/api/documents/destruction/  
  
/api/documents/verify/  
/api/documents/dashboard/  
/api/documents/statistics/
```

14.38. Principe fondamental : le document est transverse

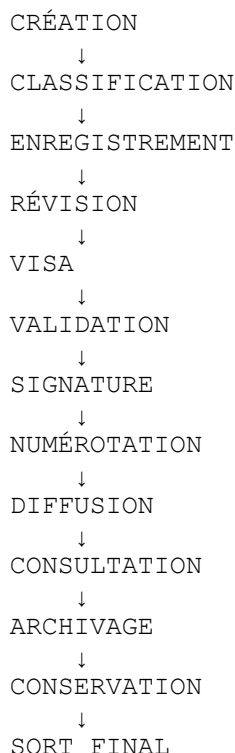
Un document ne doit pas être prisonnier d'un seul module.

Exemple :



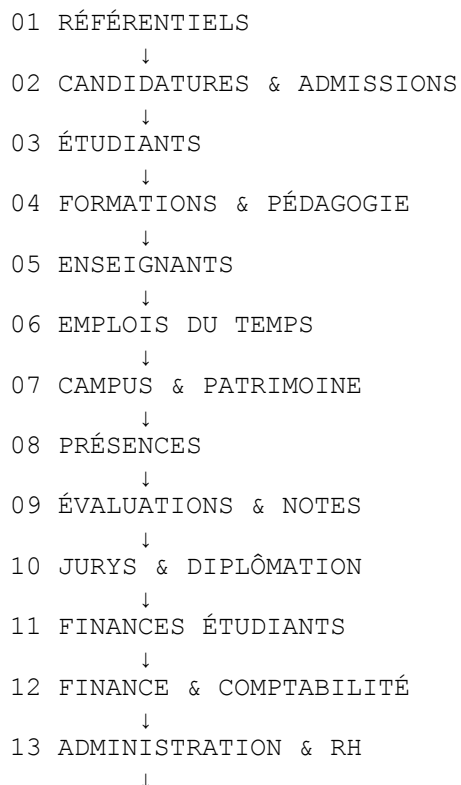
Le document reste dans **une GED centrale**, tandis que les autres modules ne stockent que la relation vers celui-ci.

14.39. Workflow documentaire global



Architecture globale désormais obtenue

Les modules étudiés forment maintenant une chaîne cohérente :



Les prochains modules peuvent maintenant compléter le système avec :

MODULE 15 — NOTIFICATIONS & COMMUNICATIONS

→ notifications internes, email, SMS, push, WhatsApp si intégration retenue, alertes, campagnes et modèles de messages.

MODULE 16 — RAPPORTS, STATISTIQUES & BI

→ tableaux de bord, indicateurs LMD, statistiques étudiants, formations, enseignants, finances, patrimoine, présences, résultats et exports décisionnels.

MODULE 17 — PORTAILS & SERVICES EN LIGNE

→ portail candidat, étudiant, enseignant, personnel, direction et éventuellement portail public de vérification.

MODULE 18 — ADMINISTRATION DU SYSTÈME & SÉCURITÉ

→ utilisateurs, rôles N0–N4, permissions, sessions, sécurité, audit, paramètres système, sauvegardes et administration technique.

Parfait. Nous poursuivons avec le **MODULE 15 — NOTIFICATIONS & COMMUNICATIONS**.

MODULE 15 — NOTIFICATIONS & COMMUNICATIONS

15.1. Objectif

Ce module constitue le système central de communication de l'application INJS.

Il doit permettre d'envoyer, recevoir, programmer, tracer et contrôler les communications destinées :

- aux étudiants ;
- aux candidats ;
- aux enseignants ;
- aux personnels administratifs ;
- aux responsables ;
- aux directions ;
- aux services ;
- aux groupes ;
- aux utilisateurs de l'application.

Il doit centraliser plusieurs canaux :

- Notifications internes
- Push mobile
- Email
- SMS
- Messages internes
- Autres canaux intégrables

Les canaux externes doivent rester **configurables** et ne doivent pas être codés en dur.

15.2. Sous-modules

15. NOTIFICATIONS & COMMUNICATIONS

- 15.1 Tableau de bord
 - 15.2 Notifications
 - 15.3 Messages internes
 - 15.4 Notifications push
 - 15.5 Emails
 - 15.6 SMS
 - 15.7 Modèles de messages
 - 15.8 Campagnes
 - 15.9 Destinataires
 - 15.10 Groupes de diffusion
 - 15.11 Programmation
 - 15.12 Alertes automatiques
 - 15.13 Notifications académiques
 - 15.14 Notifications administratives
 - 15.15 Notifications financières
 - 15.16 Notifications RH
 - 15.17 Notifications documentaires
 - 15.18 Accusés de réception
 - 15.19 Historique
 - 15.20 Statistiques
 - 15.21 Paramètres
-

15.3. Notification centrale

Créer un modèle :

Notification

Champs :

code
titre
message
type
priorité
canal
destinataire
objet lié

date_creation
date_envoi
date_lecture
date_expiration
statut

15.4. Types de notifications

Prévoir un référentiel configurable :

INFORMATION
ALERTE
RAPPEL
URGENT
VALIDATION
REJET
ÉCHÉANCE
SUCCÈS
ERREUR
ANNULATION
MODIFICATION
PUBLICATION

15.5. Priorités

BASSE
NORMALE
HAUTE
URGENTE
CRITIQUE

Une notification critique peut nécessiter une confirmation de lecture.

15.6. Canaux

Le système doit supporter :

IN_APP
PUSH
EMAIL
SMS

Prévoir une architecture permettant ultérieurement d'ajouter :

WHATSAPP
MESSAGERIE_EXTERNE
AUTRE

sans modifier toute l'application.

15.7. Notification In-App

Chaque utilisateur possède une boîte de notifications.

Exemple :

 Notifications

5 nouvelles

- Votre inscription a été validée
- Votre emploi du temps a été modifié
- Une note est disponible
- Votre facture arrive à échéance
- Un document attend votre validation

Actions :

- lire ;
 - marquer comme lu ;
 - archiver ;
 - ouvrir l'objet lié.
-

15.8. Notifications push

Pour l'application mobile :

Événement
↓
Serveur
↓
Service Push
↓
Téléphone

Le système doit gérer plusieurs appareils pour un même utilisateur.

Exemple :

Utilisateur
├─ Smartphone
├─ Tablette
└─ Navigateur

Chaque appareil possède son propre token.

15.9. Email

Prévoir :

- serveur SMTP ;
- adresse d'expédition ;
- modèles ;
- pièces jointes ;
- copie éventuelle ;
- accusé ;
- journal d'envoi.

Le système doit conserver :

Destinataire
Objet
Date
Statut
Erreur éventuelle

15.10. SMS

Le SMS doit être traité via un fournisseur configurable.

Architecture :

```
INJS
↓
API SMS
↓
Opérateur
↓
Destinataire
```

Prévoir :

- fournisseur ;
- clé API ;
- expéditeur ;
- quota ;
- coût éventuel ;
- statut ;
- erreur.

Les secrets d'intégration doivent être stockés dans une configuration sécurisée, jamais dans Flutter.

15.11. Modèles de messages

Créer :

MessageTemplate

Exemple :

Titre :
"Validation de votre inscription"

Message :
"Bonjour {{ prenom }},

Votre inscription pour {{ formation }}
au titre de l'année {{ annee }} a été validée."

Variables dynamiques :

```
{{ nom }}
{{ prenom }}
{{ matricule }}
{{ formation }}
{{ niveau }}
{{ semestre }}
{{ année }}
{{ date }}
{{ montant }}
{{ numéro }}
{{ lien }}
```

15.12. Campagnes

Une campagne permet d'envoyer un même message à un grand nombre de destinataires.

Exemple :

```
Campagne
├── Étudiants L1
├── Étudiants L2
├── Enseignants
└── Personnel administratif
```

Une campagne doit pouvoir être :

- immédiate ;
- programmée ;
- récurrente.

15.13. Groupes de diffusion

Créer des groupes dynamiques :

Groupe
├── Étudiants
├── Formation
├── Niveau
├── Groupe pédagogique
├── Enseignants
├── Service
└── Personnel

Exemple :

Licence Éducation et Motricité
↓
L1
↓
Groupe A
↓
42 étudiants

15.14. Destinataires dynamiques

Il doit être possible de cibler :

Une personne

Étudiant X

Un groupe

Groupe L1-A

Une formation

Licence Éducation et Motricité

Un service

Service X

Une population

Tous les enseignants

15.15. Programmation

Prévoir :

Envoi immédiat
Envoi différé
Date/heure programmée
Récurrence

Exemple :

Chaque lundi à 08:00
→ rappel des activités de la semaine

15.16. Alertes automatiques

Le système doit pouvoir générer automatiquement des notifications.

Exemples :

Étudiant

Inscription validée
Note publiée
Absence importante
Facture échue
Document disponible
Résultat publié

Enseignant

Cours imminent
Salle modifiée
Cours annulé
Évaluation à saisir
Note à valider
Jury programmé

Personnel

Congé validé
Mission validée
Document à signer
Contrat arrivant à échéance

Direction

Budget dépassé
Facture importante
Anomalie
Dossier en attente

15.17. Notifications académiques

Connexion avec :

- Admissions ;
- Étudiants ;
- Formations ;
- EDT ;
- Présences ;
- Notes ;
- Jurys ;
- Diplômes.

Exemples :

ADMISSION

→ "Votre admission est disponible."

EDT

→ "Votre cours de demain a été déplacé."

NOTE

→ "Une nouvelle note est disponible."

JURY

→ "Les résultats du jury sont publiés."

DIPLÔME

→ "Votre attestation est disponible."

15.18. Notifications financières

Connexion avec les Modules 11 et 12.

Exemples :

Facture créée

↓

Échéance proche

↓

Retard

↓

Païement reçu

↓

Solde régularisé

Exemple de message :

Votre paiement a été enregistré. Votre nouveau solde est disponible dans votre espace étudiant.

15.19. Notifications RH

Connexion au Module 13.

Exemples :

Demande de congé reçue
Congé validé
Congé refusé
Mission validée
Formation acceptée
Document RH disponible
Contrat bientôt expiré

15.20. Notifications documentaires

Connexion au Module 14.

Exemples :

Document à viser
Document à valider
Document à signer
Document diffusé
Document archivé
Courrier reçu
Courrier urgent

15.21. Accusé de réception

Certains messages doivent pouvoir exiger :

- ☐ Lecture
- ☐ Accusé de réception
- ☐ Confirmation obligatoire

Le système conserve :

destinataire
date_envoi
date_lecture
date_confirmation

15.22. Statuts d'envoi

BROUILLON
PROGRAMMÉ
EN_ATTENTE
ENVOI_EN_COURS
ENVOYÉ
LIVRÉ
LU
ÉCHEC
ANNULÉ
EXPIRÉ

Pour les canaux qui le permettent, distinguer :

ENVOYÉ
≠
LIVRÉ
≠
LU

15.23. File d'envoi

Les notifications doivent passer par une file asynchrone.

Architecture recommandée :

Django
↓
Queue
↓
Worker
↓
Email / SMS / Push

Avec par exemple Celery + Redis si cette architecture est retenue.

Avantages :

- ne pas bloquer l'API ;
 - réessais automatiques ;
 - traitement massif ;
 - suivi des erreurs.
-

15.24. Gestion des erreurs

En cas d'échec :

Tentative 1
↓
Échec
↓
Réessai
↓
Tentative 2
↓
Échec
↓
Tentative 3
↓
ÉCHEC DÉFINITIF

Le système doit conserver le motif d'erreur.

15.25. Préférences utilisateur

Chaque utilisateur doit pouvoir définir ses préférences lorsque la politique de l'établissement le permet :

Email	☒
Push	☒
SMS	☒
In-App	☒

Mais certaines notifications administratives ou critiques peuvent être obligatoires.

15.26. Plages horaires

Prévoir des règles pour éviter les envois non urgents à des heures inappropriées.

Exemple :

Notifications normales :
08:00 → 20:00

Notifications urgentes :
24h/24

Les plages doivent être configurables.

15.27. Historique

Chaque notification doit être traçable :

Qui ?
Quoi ?
Quand ?
À qui ?
Par quel canal ?
Quel statut ?
Quelle erreur ?

15.28. Statistiques

Le système doit produire :

Volume

- notifications envoyées ;
- emails ;
- SMS ;
- push ;
- messages internes.

Performance

- taux de livraison ;
- taux d'ouverture ;
- taux d'échec ;
- temps moyen d'envoi.

Campagnes

- destinataires ;
 - envoyés ;
 - livrés ;
 - lus ;
 - échecs.
-

15.29. Tableau de bord

Exemple :

NOTIFICATIONS	
Aujourd'hui	1 284
En attente	32
Envoyées	1 197

Échecs	55
Taux livraison	95,7 %

15.30. Modèles Django

Architecture :

```

apps/
├── notifications/
│   ├── models/
│   │   ├── notification.py
│   │   ├── template.py
│   │   ├── campaign.py
│   │   ├── recipient.py
│   │   ├── device.py
│   │   ├── delivery.py
│   │   ├── preference.py
│   │   └── schedule.py
│   ├── services/
│   │   ├── email.py
│   │   ├── sms.py
│   │   ├── push.py
│   │   └── dispatcher.py
│   ├── tasks/
│   ├── api/
│   └── reports/

```

Modèles principaux :

```

Notification
NotificationTemplate
NotificationRecipient
NotificationCampaign
NotificationDelivery
NotificationDevice
NotificationPreference
NotificationSchedule
NotificationEvent

```

15.31. APIs

```

/api/notifications/
/api/notifications/unread/
/api/notifications/{id}/read/
/api/notifications/{id}/archive/

/api/notifications/templates/
/api/notifications/campaigns/
/api/notifications/recipients/

/api/notifications/devices/

```

/api/notifications/preferences/

/api/notifications/email/

/api/notifications/sms/

/api/notifications/push/

/api/notifications/schedules/

/api/notifications/history/

/api/notifications/statistics/

/api/notifications/dashboard/

15.32. Sécurité

Les notifications doivent respecter les permissions.

Un utilisateur ne doit pas pouvoir envoyer arbitrairement une notification officielle au nom de l'INJS.

Prévoir :

Création

↓

Autorisation

↓

Validation si nécessaire

↓

Envoi

↓

Audit

Les clés API, mots de passe SMTP et tokens push ne doivent jamais être stockés dans le frontend.

15.33. Architecture événementielle

À terme, les autres modules peuvent déclencher des événements :

Admission validée

↓

Événement

↓

Notification

Paieement reçu

↓

Événement

↓

Reçu + notification

Note publiée

↓

Événement

↓

Notification étudiant
EDT modifié
↓
Événement
↓
Notification enseignant + étudiants

Cette architecture évite de mettre du code de notification directement dans chaque module métier.

15.34. Workflow global

ÉVÉNEMENT MÉTIER
↓
DÉTERMINATION DES DESTINATAIRES
↓
APPLICATION DES PRÉFÉRENCES
↓
CHOIX DU CANAL
↓
GÉNÉRATION DU MESSAGE
↓
FILE D'ENVOI
↓
ENVOI
↓
LIVRAISON
↓
LECTURE / ACCUSÉ
↓
HISTORIQUE
↓
STATISTIQUES

15.35. Principe architectural essentiel

Les modules métiers ne doivent **pas** gérer directement SMTP, SMS ou Firebase.

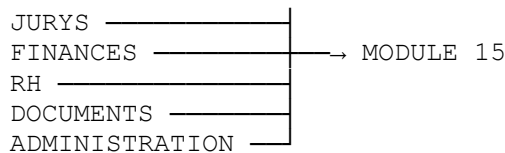
Ils doivent seulement produire un événement :

"NOTIFICATION_REQUIRED"

Le module 15 se charge ensuite du reste.

Ainsi :

ADMISSIONS	_____
ÉTUDIANTS	_____
EDT	_____
PRÉSENCES	_____
NOTES	_____



Cela rend le système beaucoup plus évolutif.

Architecture globale mise à jour

- 01 RÉFÉRENTIELS
- 02 CANDIDATURES & ADMISSIONS
- 03 ÉTUDIANTS
- 04 FORMATIONS & PÉDAGOGIE
- 05 ENSEIGNANTS
- 06 EMPLOIS DU TEMPS
- 07 CAMPUS & PATRIMOINE
- 08 PRÉSENCES
- 09 ÉVALUATIONS & NOTES
- 10 JURYS & DIPLÔMATION
- 11 FINANCES ÉTUDIANTS
- 12 FINANCE & COMPTABILITÉ
- 13 ADMINISTRATION & RH
- 14 DOCUMENTS, COURRIERS & ARCHIVES
- 15 NOTIFICATIONS & COMMUNICATIONS

Suite logique

MODULE 16 — RAPPORTS, STATISTIQUES & BUSINESS INTELLIGENCE (BI) :
il constituera la couche décisionnelle de l'INJS et consolidera les données des modules académiques, étudiants, enseignants, EDT, présences, résultats, finances, RH, patrimoine et administration.

Parfait. Nous poursuivons avec le **MODULE 16 — RAPPORTS, STATISTIQUES & BUSINESS INTELLIGENCE (BI)**.

MODULE 16 — RAPPORTS, STATISTIQUES & BUSINESS INTELLIGENCE

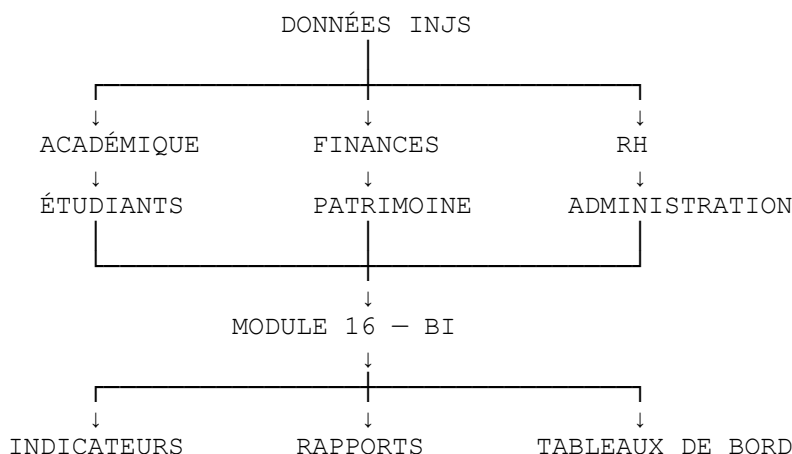
16.1. Objectif

Ce module constitue la **couche décisionnelle** du système INJS.

Il ne doit pas modifier directement les données métiers. Il les **exploite, agrège et présente** afin de permettre :

- le pilotage de l'établissement ;
- le suivi académique ;
- le suivi des étudiants ;
- le suivi des enseignants ;
- le suivi des infrastructures ;
- le suivi des présences ;
- le suivi des résultats ;
- le suivi financier ;
- le suivi RH ;
- le suivi administratif ;
- la production des statistiques officielles ;
- l'aide à la décision.

Architecture :



16.2. Sous-modules

16. RAPPORTS, STATISTIQUES & BI

- 16.1 Tableau de bord général
- 16.2 Tableaux de bord par profil
- 16.3 Indicateurs
- 16.4 KPI
- 16.5 Statistiques étudiants
- 16.6 Statistiques candidatures
- 16.7 Statistiques admissions
- 16.8 Statistiques formations
- 16.9 Statistiques modules
- 16.10 Statistiques enseignants
- 16.11 Statistiques EDT
- 16.12 Statistiques présences
- 16.13 Statistiques évaluations
- 16.14 Statistiques résultats
- 16.15 Statistiques jurys
- 16.16 Statistiques diplômes

16.17 Statistiques financières
16.18 Statistiques RH
16.19 Statistiques patrimoine
16.20 Statistiques administratives
16.21 Rapports
16.22 Rapports périodiques
16.23 Rapports personnalisés
16.24 Exports
16.25 Planification
16.26 Comparaisons
16.27 Alertes
16.28 BI
16.29 Audit des indicateurs

16.3. Principe fondamental

Le module 16 doit être en lecture analytique.

Il ne doit pas modifier directement :

- les étudiants ;
- les notes ;
- les paiements ;
- les affectations ;
- les EDT ;
- les contrats ;
- les documents.

Exemple :

Module 09
↓
Notes validées
↓
Module 16
↓
Statistiques de réussite

16.4. Tableau de bord général

Le Directeur général doit disposer d'une vue consolidée.

Exemple :

TABLEAU DE BORD INJS	
Étudiants actifs	2 458
Candidatures	1 825
Enseignants	186
Formations	24

Taux moyen de présence	91,4 %
Taux moyen de réussite	78,2 %
Budget exécuté	64,8 %
Recettes encaissées	XX FCFA
Occupation espaces	XX %

Les valeurs ci-dessus sont uniquement des **exemples d’affichage** et ne constituent pas des données réelles de l’INJS.

16.5. KPI

Créer un référentiel :

KPI

Chaque indicateur possède :

```
code
nom
description
module_source
formule
unité
périodicité
niveau
objectif
seuil_alerte
statut
```

16.6. Catégories de KPI

Académique

- taux de réussite ;
- taux d’échec ;
- taux de validation ;
- crédits obtenus ;
- moyenne générale ;
- taux de passage ;
- taux de redoublement.

Étudiants

- effectif ;
- nouveaux inscrits ;
- réinscriptions ;
- abandons ;

- sorties ;
- étudiants par formation ;
- étudiants par niveau.

Enseignants

- effectif ;
- charge horaire ;
- heures prévues ;
- heures réalisées ;
- taux d'utilisation.

Présences

- taux de présence ;
- taux d'absence ;
- absences justifiées ;
- retards.

Finances

- recettes ;
- dépenses ;
- budget ;
- taux d'exécution ;
- impayés ;
- trésorerie.

Patrimoine

- nombre d'espaces ;
- capacité ;
- taux d'occupation ;
- équipements ;
- immobilisations ;
- maintenance.

16.7. Statistiques étudiants

Le système doit permettre de croiser :

Étudiants
 — Année
 — Formation
 — Parcours
 — Niveau
 — Semestre
 — Groupe

- Catégorie
- Grade
- Sexe
- Statut
- Situation académique

Exemples :

Combien d'étudiants sont inscrits en L2 Éducation et Motricité ?

Combien d'étudiants sont actifs cette année ?

Quelle est la répartition des étudiants par formation ?

16.8. Candidatures

Indicateurs :

- Candidatures reçues
- Dossiers complets
- Dossiers incomplets
- Admissibles
- Non admissibles
- Admis
- Refusés
- Liste d'attente

Calculs :

Taux d'admission =
 $\text{Admis} / \text{Candidatures} \times 100$

16.9. Formations

Pour chaque formation :

- Effectif
- Capacité
- Nombre de groupes
- Enseignants
- Modules
- Volume horaire
- Taux de réussite
- Taux d'abandon
- Crédits

Comparer plusieurs années :

2024 → 2025 → 2026

16.10. Modules pédagogiques

Pour chaque module :

- nombre d'étudiants ;
- enseignant(s) ;
- VH ;
- CM ;
- TD ;
- TP ;
- évaluations ;
- moyenne ;
- taux de validation ;
- taux d'échec.

16.11. Enseignants

Le tableau de bord doit permettre de mesurer :

Enseignant
— Formations
— Modules
— Groupes
— CM
— TD
— TP
— VH prévue
— VH affectée
— VH réalisée
— Évaluations
— Notes saisies

Indicateur :

Taux de réalisation VH =
$$\text{VH réalisée} / \text{VH planifiée} \times 100$$

16.12. Emplois du temps

Indicateurs :

- nombre de séances ;
- heures programmées ;
- heures réalisées ;

- heures annulées ;
 - heures reportées ;
 - conflits détectés ;
 - taux d'occupation des salles ;
 - taux d'occupation des enseignants ;
 - taux d'occupation des groupes.
-

16.13. Campus & patrimoine

Connexion au Module 07.

Statistiques :

Sites
Zones
Bâtiments
Étages
Espaces
Salles
Amphithéâtres
Laboratoires
Gymnases
Terrains
Piscines
Pistes
Bureaux

Indicateurs :

Capacité totale
Surface totale
Occupation
Disponibilité
Maintenance
Hors service

16.14. Présences

Le système doit pouvoir produire :

Par étudiant

Séances
Présences
Absences
Retards
Absences justifiées
Taux de présence

Par formation

Formation
→ Niveau
→ Groupe
→ Module
→ Séances

Par enseignant

- séances réalisées ;
- séances ouvertes ;
- présences saisies ;
- anomalies.

16.15. Évaluations

Indicateurs :

- nombre d'évaluations ;
- notes saisies ;
- notes validées ;
- notes en attente ;
- anomalies ;
- moyennes ;
- distributions ;
- taux de réussite.

Exemple :

Module X

Moyenne : 12,84 / 20
Minimum : 04,00
Maximum : 19,00
Validation : 76 %

16.16. Résultats académiques

Le système doit calculer :

Admis
Ajournés
Admis avec dette
Validation partielle
Redoublants

Indicateurs :

Taux de réussite
Taux d'échec
Taux de validation
Taux de passage
Taux de redoublement

16.17. Jurys

Statistiques :

- nombre de jurys ;
 - étudiants examinés ;
 - décisions ;
 - validations ;
 - ajournements ;
 - diplômes attribués.
-

16.18. Diplômes

Indicateurs :

Diplômes produits
Diplômes remis
Attestations produites
Duplicatas
Rectifications
Documents vérifiés

16.19. Finance étudiants

Connexion Module 11 :

Montant facturé
Montant payé
Reste à payer
Taux de recouvrement
Nombre d'impayés
Nombre d'étudiants débiteurs

16.20. Finance & comptabilité

Connexion Module 12 :

Budget

Engagements
Dépenses
Recettes
Trésorerie
Banques
Caisse
Fournisseurs
Factures

16.21. RH

Connexion Module 13 :

Personnel
Effectifs
Congés
Absences
Contrats
Affectations
Missions
Formations
Départs
Recrutements

16.22. Rapports

Le système doit proposer des rapports standards.

Académiques

- état des inscriptions ;
- état des effectifs ;
- résultats par formation ;
- résultats par module ;
- taux de réussite ;
- crédits ;
- jurys.

Administratifs

- état des personnels ;
- organigramme ;
- affectations ;
- congés ;
- missions.

Financiers

- budget ;

- exécution ;
- recettes ;
- dépenses ;
- trésorerie.

Patrimoine

- espaces ;
- capacité ;
- occupation ;
- équipements ;
- maintenance.

16.23. Rapports personnalisés

Un utilisateur autorisé doit pouvoir sélectionner :

Source
↓
Champs
↓
Filtres
↓
Groupement
↓
Tri
↓
Indicateurs
↓
Graphique
↓
Export

Exemple :

Formation
→ Niveau
→ Année
→ Effectif
→ Taux de réussite

16.24. Filtres temporels

Tous les rapports doivent pouvoir utiliser :

Aujourd'hui
Cette semaine
Ce mois
Ce trimestre

Cette année
Année académique
Exercice financier
Période personnalisée

16.25. Comparaison temporelle

Exemple :

	2024	2025	2026
Effectifs	XXX	XXX	XXX
Réussite	XX%	XX%	XX%
Présence	XX%	XX%	XX%
Recettes	XXX	XXX	XXX

16.26. Visualisations

Prévoir :

- cartes KPI ;
- tableaux ;
- graphiques en barres ;
- courbes ;
- secteurs ;
- histogrammes ;
- jauges ;
- cartes thermiques ;
- graphiques comparatifs.

Les graphiques doivent être générés à partir des données réelles.

16.27. Alertes décisionnelles

Le système peut générer :

- ⚠ Taux de réussite inférieur au seuil
- ⚠ Budget proche du plafond
- ⚠ Forte hausse des absences
- ⚠ Salle fortement sous-dimensionnée
- ⚠ Factures fournisseurs en retard
- ⚠ Contrats arrivant à échéance
- ⚠ Modules sans notes validées

Les seuils sont configurables.

16.28. Rapports programmés

Un rapport peut être généré automatiquement :

Chaque jour
Chaque semaine
Chaque mois
Chaque trimestre
Chaque année

Puis :

Génération
↓
PDF / Excel
↓
Notification
↓
Archivage

16.29. Exports

Formats :

PDF
XLSX
CSV

Les exports doivent respecter les permissions de l'utilisateur.

16.30. BI avancée

À terme, prévoir une architecture permettant :

Données opérationnelles
↓
Extraction
↓
Transformation
↓
Agrégation
↓
Indicateurs
↓
BI

Pour les gros volumes, une base analytique ou des tables d'agrégation pourront être introduites ultérieurement.

Il n'est pas nécessaire de mettre en place un data warehouse complexe dès la première version.

16.31. Snapshot des indicateurs

Pour les statistiques historiques importantes, il est recommandé de conserver des snapshots.

Exemple :

```
KPI
|
├── 2024
├── 2025
└── 2026
```

Cela évite qu'une modification future des données opérationnelles ne change silencieusement un rapport historique déjà publié.

16.32. Audit des indicateurs

Chaque rapport doit pouvoir indiquer :

```
Source des données
Date de calcul
Période
Formule
Filtres
Utilisateur
Version de l'indicateur
```

Ainsi, un chiffre présenté à la Direction peut être expliqué et reproduit.

16.33. Sécurité

Les rapports doivent respecter les permissions.

Exemple :

```
Étudiant
→ ses propres données
```

Enseignant
→ ses groupes/modules

Chef de service
→ son service

Direction
→ son périmètre

Direction générale
→ vision globale

Administrateur autorisé
→ configuration technique

16.34. Modèles Django

Architecture recommandée :

```
apps/  
├── reports/  
│   ├── models/  
│   │   ├── kpi.py  
│   │   ├── report.py  
│   │   ├── report_filter.py  
│   │   ├── dashboard.py  
│   │   ├── widget.py  
│   │   ├── snapshot.py  
│   │   └── schedule.py  
│   ├── services/  
│   │   ├── analytics.py  
│   │   ├── aggregation.py  
│   │   └── export.py  
│   ├── api/  
│   └── permissions/
```

Modèles principaux :

KPI
KPIFormula
KPIValue
Report
ReportFilter
Dashboard
DashboardWidget
ReportSnapshot
ScheduledReport
ReportExecution

16.35. APIs

```
/api/reports/  
/api/reports/kpis/  
/api/reports/kpis/{id}/values/  
  
/api/reports/dashboards/  
/api/reports/widgets/  
  
/api/reports/students/  
/api/reports/admissions/  
/api/reports/academics/  
/api/reports/teachers/  
/api/reports/timetable/  
/api/reports/attendance/  
/api/reports/results/  
/api/reports/juries/  
/api/reports/diplomas/  
  
/api/reports/finance/  
/api/reports/hr/  
/api/reports/campus/  
/api/reports/administration/  
  
/api/reports/custom/  
/api/reports/export/  
/api/reports/schedules/  
/api/reports/snapshots/  
/api/reports/executions/
```

16.36. Architecture des tableaux de bord par profil

Il ne faut pas avoir un seul dashboard pour tout le monde.

Direction générale

Vue globale INJS

Direction des études

Étudiants
Formations
Résultats
Présences
Enseignants
EDT

Direction financière

Budget
Recettes
Dépenses
Trésorerie

DRH

Effectifs
Contrats
Congés
Carrière

Responsable pédagogique

Formations
Modules
Groupes
EDT
Notes
Résultats

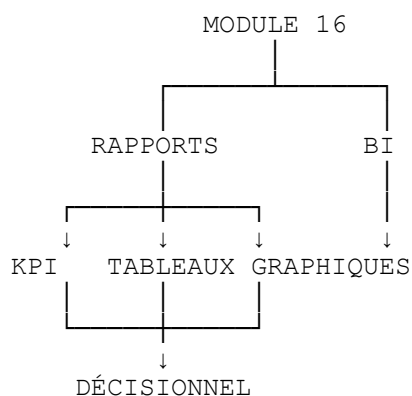
Enseignant

Mes groupes
Mes séances
Mes présences
Mes évaluations
Mes notes

Étudiant

Mes notes
Mes crédits
Mes absences
Ma situation
Mes paiements

16.37. Architecture globale du Module 16



16.38. Principe essentiel

Le module 16 doit répondre rapidement à des questions telles que :

Combien d'étudiants sont inscrits aujourd'hui ?

Quelle formation a le plus grand effectif ?

Quel est le taux de réussite par formation ?

Quels modules présentent le plus d'échecs ?

Quel est le taux de présence ?

Quelle est la charge horaire des enseignants ?

Quel est le taux d'occupation des salles ?

Quel est le montant des impayés étudiants ?

Quel est le taux d'exécution budgétaire ?

Quelle est la situation de la trésorerie ?

Quels contrats arrivent à échéance ?

Quels bâtiments ou équipements sont en maintenance ?

L'objectif est de passer de :

DONNÉES

à :

INFORMATIONS

puis :

INDICATEURS

puis :

DÉCISION

Architecture INJS complétée

Nous avons maintenant :

01 RÉFÉRENTIELS

02 CANDIDATURES & ADMISSIONS

03 ÉTUDIANTS

04 FORMATIONS & PÉDAGOGIE

05 ENSEIGNANTS

06 EMPLOIS DU TEMPS

07 CAMPUS & PATRIMOINE
08 PRÉSENCES
09 ÉVALUATIONS & NOTES
10 JURYS & DIPLÔMATION
11 FINANCES ÉTUDIANTS
12 FINANCE & COMPTABILITÉ
13 ADMINISTRATION & RH
14 DOCUMENTS, COURRIERS & ARCHIVES
15 NOTIFICATIONS & COMMUNICATIONS
16 RAPPORTS, STATISTIQUES & BI

Prochaine étape :

MODULE 17 — PORTAILS & SERVICES EN LIGNE

Ce module permettra de définir précisément les espaces utilisateurs :

PORTAIL PUBLIC
↓
PORTAIL CANDIDAT
↓
PORTAIL ÉTUDIANT
↓
PORTAIL ENSEIGNANT
↓
PORTAIL PERSONNEL
↓
PORTAIL RESPONSABLE
↓
PORTAIL DIRECTION
↓
ADMINISTRATION

avec leurs **menus, tableaux de bord, fonctionnalités, droits N0–N4, workflows et APIs**, tout en évitant de dupliquer les modules métier déjà définis.

Parfait. Nous passons au **MODULE 17 — PORTAILS & SERVICES EN LIGNE**.

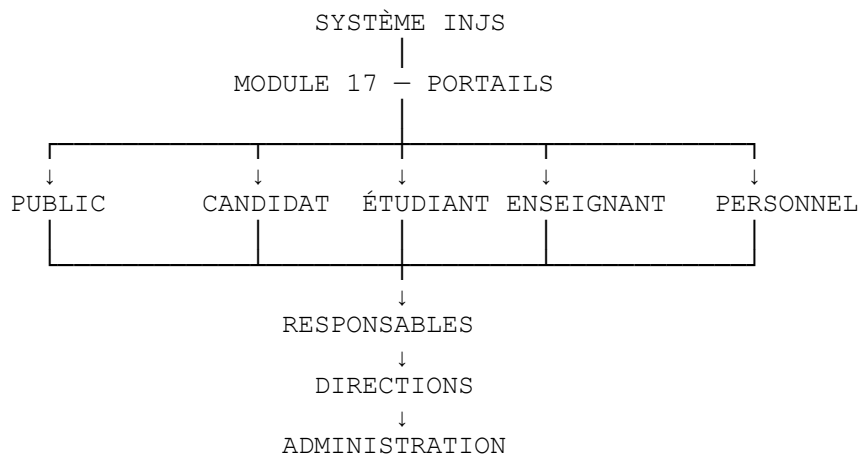
MODULE 17 — PORTAILS & SERVICES EN LIGNE

17.1. Objectif

Le Module 17 constitue la **couche d'accès utilisateur** au système INJS.

Il ne doit pas recréer les fonctionnalités métiers des Modules 01 à 16. Il doit plutôt permettre à chaque population d'accéder, selon ses droits, aux services qui lui sont destinés.

Architecture :



17.2. Les différents portails

Le système doit prévoir les espaces suivants :

- 17.1 Portail public
 - 17.2 Portail candidat
 - 17.3 Portail étudiant
 - 17.4 Portail enseignant
 - 17.5 Portail personnel
 - 17.6 Portail responsable
 - 17.7 Portail direction
 - 17.8 Portail administration
 - 17.9 Services publics de vérification
 - 17.10 Services API
-

17.3. PORTAIL PUBLIC

Accessible sans authentification.

Il doit présenter les informations publiques de l'INJS.

Contenu

- Accueil
- Présentation de l'INJS
- Formations
- Conditions d'admission
- Concours
- Calendrier académique
- Actualités
- Communiqués
- Avis
- Résultats publiés
- Documents publics
- Contacts
- Localisation
- FAQ

Services

- consultation des formations ;
- consultation des campagnes ouvertes ;
- téléchargement des avis ;
- consultation des calendriers ;
- consultation des résultats publiés ;
- vérification d'un document ;
- contact avec l'INJS.

17.4. PORTAIL CANDIDAT

Accessible après création d'un compte candidat.

Workflow :

```
Création compte
↓
Activation
↓
Profil candidat
↓
Choix de campagne
↓
Choix formation
↓
Dossier
↓
Documents
↓
Soumission
↓
Suivi
↓
Convocation
↓
Concours / sélection
↓
Résultat
↓
Admission
```

17.4.1. Tableau de bord candidat

Afficher :

Mes candidatures
Dossiers incomplets
Documents manquants
Convocations
Dates importantes

17.4.2. Services candidat

- créer une candidature ;
- modifier son dossier avant soumission ;
- déposer les pièces ;
- remplacer une pièce refusée ;
- payer les frais si applicables ;
- télécharger reçu ;
- télécharger convocation ;
- consulter centre/lieu/salle ;
- consulter résultat ;
- suivre admission ;
- envoyer une demande ou réclamation.

Après soumission, les données sensibles doivent devenir contrôlées/figées selon les règles de la campagne.

17.5. PORTAIL ÉTUDIANT

C'est l'espace principal de l'étudiant.

Menu

Tableau de bord
Mon profil
Ma scolarité
Mon inscription
Ma formation
Mon parcours
Mes modules
Mes groupes
Mon emploi du temps
Mes présences
Mes évaluations
Mes notes
Mes résultats
Mes crédits
Mes jurys
Mes documents
Mes paiements
Mes notifications
Mes demandes
Aide

17.6. Tableau de bord étudiant

Exemple :

TABLEAU DE BORD ÉTUDIANT	
Formation :	Éducation et Motricité
Niveau :	L2
Année :	2026-2027
Moyenne générale :	XX / 20
Crédits obtenus :	XX / XX
Taux présence :	XX %
Solde scolarité :	XX FCFA
Prochain cours :	
XX/XX – XX:XX – Module – Salle	

Les valeurs sont dynamiques et proviennent des modules métiers.

17.7. Inscription étudiant

L'étudiant doit pouvoir consulter :

Année académique
Formation
Parcours
Niveau
Groupe
Statut inscription
Date inscription
Numéro étudiant

Selon les règles définies par l'INJS, certaines démarches peuvent également être réalisables en ligne.

17.8. Formation et pédagogie

L'étudiant consulte :

- sa formation ;
- son parcours ;
- son niveau ;
- ses semestres ;
- ses UE/modules ;

- crédits ;
- coefficients ;
- volumes horaires ;
- enseignants ;
- groupes.

17.9. Emploi du temps étudiant

Le portail doit afficher :

Jour
Heure
Module
Type : CM / TD / TP
Enseignant
Salle
Statut

Fonctions :

- vue jour ;
- semaine ;
- mois ;
- recherche ;
- filtrage ;
- détail de séance ;
- notification de changement ;
- séance annulée ;
- séance reportée.

17.10. Présences étudiant

L'étudiant peut consulter :

Module
Séance
Date
Heure
Statut
Retard
Justification

Il doit pouvoir :

- scanner un QR de séance ;
- consulter ses absences ;

- déposer une justification ;
- suivre son traitement ;
- consulter son taux de présence.

Le QR ne doit jamais être un simple identifiant étudiant permanent.

17.11. Notes étudiant

L'étudiant peut consulter :

Module
Évaluation
Type
Date
Note
Coefficient
Statut

Les notes non publiées ne doivent pas être visibles comme notes officielles.

17.12. Résultats étudiant

Afficher :

Semestre
Moyenne
Crédits
UE validées
UE non validées
Décision

Puis :

Année
Moyenne annuelle
Crédits
Décision
Passage

17.13. Documents étudiant

Accès sécurisé à :

- certificat de scolarité ;
- attestation ;

- relevé de notes ;
- certificat de réussite ;
- convocation ;
- documents administratifs ;
- documents financiers ;
- autres documents autorisés.

Les documents doivent provenir du **Module 14 — Documents, Courriers & Archives**.

17.14. Paiements étudiant

Connexion au Module 11.

L'étudiant consulte :

Frais
Factures
Échéances
Paiements
Reçus
Reste à payer
Historique

Selon les services activés :

Carte bancaire
Orange Money
MTN Money
Moov Money
Virement
Autres moyens autorisés

Les secrets/API de paiement restent exclusivement côté backend.

17.15. PORTAIL ENSEIGNANT

Menu :

Tableau de bord
Mon profil
Mes affectations
Mes formations
Mes modules
Mes groupes
Mon emploi du temps
Mes séances
Présences
Évaluations

Saisie des notes
Mes charges horaires
Documents
Notifications
Demandes

17.16. Tableau de bord enseignant

Modules affectés
Groupes
Séances à venir
Séances réalisées
VH planifiée
VH affectée
VH réalisée
Évaluations à corriger
Notes en attente
Présences à saisir

17.17. Gestion pédagogique enseignant

L'enseignant peut consulter :

- formations auxquelles il est affecté ;
- modules ;
- groupes ;
- maquettes concernées ;
- volumes horaires ;
- CM/TD/TP ;
- étudiants de ses groupes.

Il ne doit voir que son périmètre autorisé.

17.18. Présences enseignant

Fonctions :

Ouvrir séance
Générer QR
Scanner/contrôler
Saisir manuellement
Marquer absent
Marquer retard
Justification
Forçage autorisé
Clôturer séance

Toutes les opérations sensibles sont auditées.

17.19. Notes enseignant

Workflow :

Évaluation
↓
Saisie
↓
Contrôle
↓
Soumission
↓
Validation
↓
Publication

Une note validée ne doit pas pouvoir être silencieusement modifiée.

17.20. PORTAIL PERSONNEL

Destiné aux agents administratifs, techniques et de soutien.

Fonctions :

Tableau de bord
Mon dossier
Mon affectation
Mes documents
Mes congés
Mes absences
Mes missions
Mes formations
Mes demandes
Notifications

Les fonctionnalités RH viennent du Module 13.

17.21. PORTAIL RESPONSABLE

Pour :

- responsables pédagogiques ;
- chefs de service ;

- responsables de formation ;
- responsables administratifs ;
- responsables de groupes.

Dashboard

Effectifs
Activités
Présences
EDT
Notes
Anomalies
Demandes
Rapports
Alertes

Le périmètre est déterminé dynamiquement par les affectations et permissions.

17.22. PORTAIL DIRECTION

Destiné aux directions et responsables habilités.

Accès :

Tableaux de bord
Rapports
Statistiques
Académique
Personnel
Finances
Patrimoine
Documents
Décisions
Alertes

Il doit s'appuyer fortement sur le Module 16.

17.23. PORTAIL ADMINISTRATION

C'est l'espace réservé aux utilisateurs administratifs autorisés.

Il donne accès aux modules selon leurs permissions.

Référentiels
Admissions
Étudiants
Pédagogie
Enseignants

EDT
Campus
Présences
Notes
Jurys
Finances
RH
Documents
Notifications
Rapports
Administration système

Le portail ne doit pas donner automatiquement tous les droits.

17.24. Services publics de vérification

Certaines opérations doivent être accessibles sans compte.

Exemple :

Vérifier un document
↓
QR / numéro de vérification
↓
Serveur INJS
↓
Document valide ?
↓
Oui / Non

La réponse publique doit être minimale :

Document valide
Type
Numéro
Date
Formation éventuellement
Statut

Ne jamais exposer :

- ID Django ;
- données personnelles inutiles ;
- informations internes ;
- informations confidentielles.

17.25. Authentification

Le portail doit utiliser une authentification centralisée.

Utilisateur
↓
Authentification
↓
Identité
↓
Rôles
↓
Permissions
↓
Périmètre
↓
Portail

Il ne faut pas créer un système d'identité indépendant pour chaque portail.

17.26. Compte utilisateur ≠ personne

Principe fondamental :

USER
|
— CANDIDAT
— ÉTUDIANT
— ENSEIGNANT
— PERSONNEL
— RESPONSABLE

Une même personne peut évoluer dans le système sans créer inutilement plusieurs comptes.

Exemple :

Candidat
↓
Admis
↓
Étudiant

Son identité doit rester cohérente.

17.27. Gestion des rôles N0–N4

Le système doit conserver la logique de droits existante du projet.

Exemple de principe :

N0 → accès public / consultation minimale
N1 → utilisateur
N2 → opérationnel

N3 → responsable / validation
N4 → administration avancée

Les droits exacts doivent rester alignés sur le manuel INJS existant et sur les permissions réellement implémentées.

17.28. Navigation dynamique

Le menu Flutter ne doit pas être codé uniquement en dur.

Le backend doit permettre de déterminer :

```
Utilisateur
  ↓
Rôles
  ↓
Permissions
  ↓
Modules autorisés
  ↓
Menus
  ↓
Actions
```

Cela évite qu'un utilisateur voie des menus auxquels il n'a aucun droit.

17.29. Notifications

Tous les portails doivent pouvoir recevoir les notifications du Module 15 :

```
IN_APP
PUSH
EMAIL
SMS
```

Exemple :

⚠ Votre séance de 14h a été déplacée.

17.30. Services en ligne

Le portail peut proposer :

```
Demande d'attestation
Demande de certificat
```

Demande de relevé
Demande de correction
Demande administrative
Dépôt de document
Réclamation
Prise de rendez-vous
Paielement
Téléchargement
Vérification

Chaque demande doit être reliée au module métier concerné.

17.31. Workflow générique des demandes

DEMANDE
↓
SOUSSION
↓
RÉCEPTION
↓
AFFECTATION
↓
TRAITEMENT
↓
VALIDATION
↓
DÉCISION
↓
DOCUMENT
↓
NOTIFICATION
↓
ARCHIVAGE

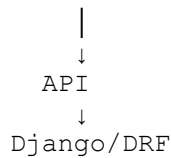
17.32. Responsive / multi-plateforme

Le portail doit fonctionner selon les besoins sur :

- navigateur Web ;
- Flutter Web ;
- Android ;
- iOS ;
- ordinateur ;
- tablette ;
- smartphone.

L'API reste commune.

Flutter Web
Flutter Android
Flutter iOS



17.33. Sécurité

Obligations :

- authentification sécurisée ;
- expiration des sessions ;
- contrôle des permissions côté backend ;
- contrôle du périmètre ;
- protection contre accès horizontal ;
- validation des fichiers ;
- limitation des tentatives ;
- journalisation ;
- audit ;
- protection des données personnelles ;
- aucun secret dans Flutter ;
- aucun accès direct à PostgreSQL depuis le client.

Le frontend ne doit **jamais être considéré comme la barrière de sécurité principale.**

17.34. APIs

Architecture indicative :

```
/api/auth/  
/api/me/  
  
/api/portal/public/  
/api/portal/candidate/  
/api/portal/student/  
/api/portal/teacher/  
/api/portal/staff/  
/api/portal/manager/  
/api/portal/direction/  
  
/api/services/  
/api/requests/  
/api/verification/
```

Les endpoints métiers restent ceux des Modules 01 à 16.

17.35. Architecture Flutter

```
lib/
├── core/
│   ├── auth/
│   ├── permissions/
│   ├── navigation/
│   └── api/
├── portals/
│   ├── public/
│   ├── candidate/
│   ├── student/
│   ├── teacher/
│   ├── staff/
│   ├── manager/
│   └── direction/
├── services/
│   ├── requests/
│   ├── verification/
│   └── notifications/
└── shared/
```

L'architecture existante de l'application doit être réutilisée autant que possible.

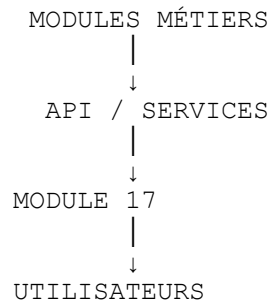
17.36. Tableau de correspondance

Portail	Modules principalement utilisés
Public	01, 02, 14, 15, 16
Candidat	01, 02, 11, 14, 15
Étudiant	03, 04, 06, 08, 09, 10, 11, 14, 15
Enseignant	04, 05, 06, 08, 09, 15
Personnel	13, 14, 15
Responsable	04, 05, 06, 08, 09, 16
Direction	12, 13, 16
Administration	01–16 selon permissions
Vérification	14, éventuellement 10

17.37. Principe d'intégrité

Le Module 17 **ne doit pas devenir un deuxième ERP.**

Il doit fonctionner comme :



et non :

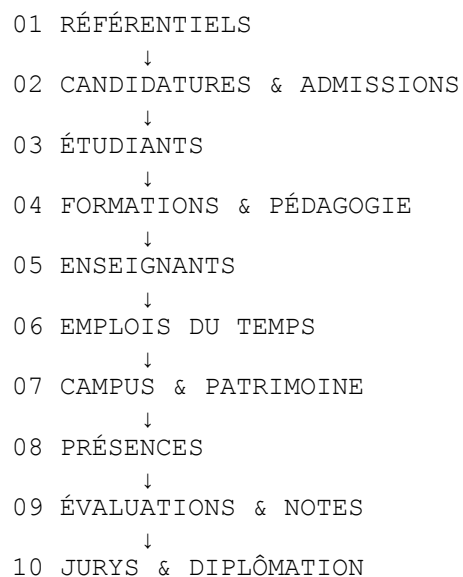
Portail étudiant
└─ copie des étudiants

Portail enseignant
└─ copie des enseignants

Portail finance
└─ copie des finances

Une seule source de vérité par domaine métier.

17.38. Architecture complète après le Module 17



↓
11 FINANCES ÉTUDIANTS
↓
12 FINANCE & COMPTABILITÉ
↓
13 ADMINISTRATION & RH
↓
14 DOCUMENTS, COURRIERS & ARCHIVES
↓
15 NOTIFICATIONS & COMMUNICATIONS
↓
16 RAPPORTS, STATISTIQUES & BI
↓
17 PORTAILS & SERVICES EN LIGNE
↓
18 ADMINISTRATION DU SYSTÈME & SÉCURITÉ

Prochaine étape

Il reste maintenant le **MODULE 18 — ADMINISTRATION DU SYSTÈME & SÉCURITÉ**, qui sera particulièrement important pour ton application : **utilisateurs, rôles N0–N4, permissions, groupes, authentification, sécurité API, journalisation, audit, sauvegardes, paramètres système, sessions, appareils, sécurité des données, administration technique et gouvernance du système.**

Parfait. Nous terminons l’architecture fonctionnelle avec le **MODULE 18 — ADMINISTRATION DU SYSTÈME & SÉCURITÉ**. C’est le module transversal qui garantit que les 17 autres modules restent **sécurisés, traçables, configurables et administrables**.

MODULE 18 — ADMINISTRATION DU SYSTÈME & SÉCURITÉ

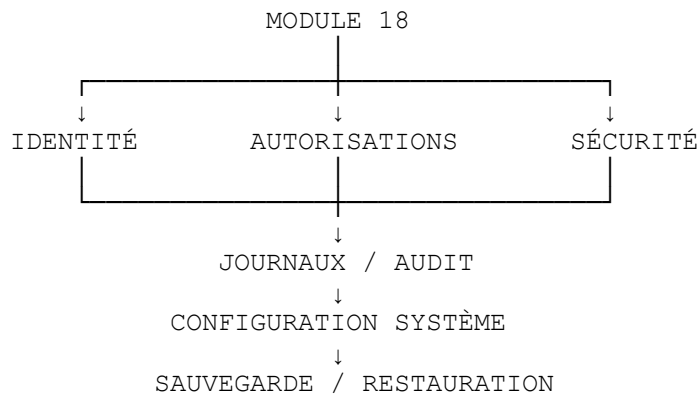
18.1. Objectif

Le Module 18 assure :

- la gestion des utilisateurs ;
- les rôles et permissions ;
- les niveaux N0–N4 ;
- l’authentification ;
- les sessions ;
- les appareils ;
- la sécurité API ;
- l’audit ;
- les journaux ;
- les paramètres système ;
- la configuration ;

- les sauvegardes ;
- la restauration ;
- la surveillance ;
- la gestion des erreurs ;
- la conformité ;
- la gouvernance des données.

Architecture :



18.2. Sous-modules

- 18.1 Utilisateurs
 - 18.2 Personnes et identités
 - 18.3 Rôles
 - 18.4 Permissions
 - 18.5 Niveaux N0-N4
 - 18.6 Groupes
 - 18.7 Authentification
 - 18.8 Sessions
 - 18.9 Appareils
 - 18.10 Sécurité API
 - 18.11 Journalisation
 - 18.12 Audit
 - 18.13 Paramètres système
 - 18.14 Configuration
 - 18.15 Notifications de sécurité
 - 18.16 Sauvegardes
 - 18.17 Restauration
 - 18.18 Surveillance
 - 18.19 Santé du système
 - 18.20 Gestion des erreurs
 - 18.21 Maintenance
 - 18.22 Archivage technique
 - 18.23 Gouvernance des données
 - 18.24 Conformité
-

18.3. Principe fondamental

Le système doit distinguer :

PERSONNE
↓
COMPTE UTILISATEUR
↓
RÔLE
↓
PERMISSIONS
↓
PÉRIMÈTRE
↓
ACTIONS AUTORISÉES

Il ne faut jamais confondre :

- identité ;
- compte ;
- rôle ;
- permission ;
- fonction administrative.

18.4. Gestion des utilisateurs

L'administrateur autorisé peut :

- créer un compte ;
- activer ;
- désactiver ;
- suspendre ;
- réactiver ;
- réinitialiser l'accès ;
- modifier les rôles ;
- consulter les connexions ;
- révoquer les sessions ;
- gérer les appareils.

Statuts :

INVITÉ
ACTIF
SUSPENDU
DÉSACTIVÉ
VERROUILLÉ
EXPIRÉ

18.5. Compte utilisateur

Modèle conceptuel :

User

- username / identifiant
- email
- téléphone
- statut
- date création
- dernière connexion
- dernière activité
- personne liée

Le compte doit être relié à la personne correspondante lorsqu'une identité métier existe.

18.6. Rôles

Créer un référentiel de rôles.

Exemples :

SUPER_ADMIN
ADMIN_SYSTEME
DIRECTION_GENERALE
DIRECTION_ETUDES
RESPONSABLE_FORMATION
SCOLARITE
ENSEIGNANT
ETUDIANT
CANDIDAT
FINANCE
COMPTABILITE
RH
DOCUMENTATION
PATRIMOINE
JURY
AUDITEUR
CONSULTATION

Les noms définitifs doivent être configurables.

18.7. Permissions

Une permission doit être liée à :

Module
Ressource
Action
Périmètre

Actions possibles :

CONSULTER
CRÉER
MODIFIER
SOUMETTRE
VALIDER
REJETER
PUBLIER
ANNULER
EXPORTER
IMPRIMER
ARCHIVER
SUPPRIMER
ADMINISTRER

La suppression physique doit être extrêmement limitée.

18.8. Niveaux N0–N4

Le système conserve la logique du manuel INJS :

N0
N1
N2
N3
N4

Mais il faut éviter de faire dépendre toute la sécurité d'un simple nombre.

La décision d'autorisation doit prendre en compte :

Niveau
+
Rôle
+
Permission
+
Module
+
Périmètre
+
État de l'objet

Exemple :

ENSEIGNANT
+
N2
+
NOTES.MODIFIER
+
SES GROUPES
+
NOTE NON VALIDÉE

→ autorisé.

Mais :

ENSEIGNANT
+
N2
+
NOTES.MODIFIER
+
AUTRE FORMATION

→ refusé.

18.9. RBAC + périmètre

Le système doit utiliser une combinaison :

RBAC :

et **Scope :**

Périmètres possibles :

INJS_ENTIER
DIRECTION
SERVICE
FORMATION
PARCOURS
NIVEAU
GROUPE
MODULE
ETUDIANT
PROPRE_COMPTE

18.10. Authentication

Prévoir :

- connexion ;
- déconnexion ;
- renouvellement de session/token ;
- changement de mot de passe ;
- réinitialisation ;
- vérification email/téléphone si activée ;
- éventuellement MFA.

Architecture :

Flutter/Web
↓
HTTPS
↓
API Auth
↓
Django
↓
Identity
↓
Permissions

18.11. MFA

Pour les comptes sensibles, prévoir une authentification renforcée.

Exemple :

Mot de passe
+
Second facteur

Le MFA peut être activé progressivement pour :

- administrateurs ;
 - direction ;
 - finance ;
 - RH ;
 - utilisateurs ayant accès aux données sensibles.
-

18.12. Gestion des sessions

Le système doit conserver :

Utilisateur
Date connexion
Date dernière activité
Adresse IP
Navigateur / application
Appareil
Date expiration
Statut

Fonctions :

- voir les sessions actives ;
- fermer une session ;
- fermer toutes les sessions ;
- révoquer un appareil ;

- détecter activité inhabituelle.

18.13. Gestion des appareils

Particulièrement importante pour Flutter mobile.

Utilisateur
Device ID
Type
OS
Application
Version
Token push
Dernière activité
Statut

Possibilité :

Révoquer cet appareil

18.14. Sécurité API

Toutes les API sensibles doivent contrôler :

Authentification
↓
Permission
↓
Périmètre
↓
Validation données
↓
Action
↓
Audit

Le frontend ne doit jamais pouvoir contourner ces contrôles.

18.15. Protection contre l'accès horizontal

Exemple critique :

Étudiant A
↓
/api/students/B

Le serveur doit vérifier que l'utilisateur A n'a pas accès à B.

Le simple fait de connaître une URL ou un identifiant ne doit jamais permettre l'accès.

18.16. Identifiants internes

Les identifiants techniques Django/PostgreSQL ne doivent pas être utilisés comme données métier visibles.

Exports :

```
MAT-2026-00125
FORMATION-EM
MODULE-EM101
SALLE-A12
```

et non :

```
id = 1547
formation_id = 12
module_id = 87
```

Cela respecte également le principe déjà retenu pour les imports/exports INJS.

18.17. Journalisation

Le système doit enregistrer les événements importants :

```
Connexion
Déconnexion
Échec connexion
Création
Modification
Validation
Rejet
Publication
Annulation
Export
Téléchargement
Suppression logique
Modification permission
Modification configuration
```

18.18. Audit

L'audit métier est plus détaillé qu'un simple log.

Chaque événement important doit pouvoir enregistrer :

Utilisateur
Date/heure
Action
Module
Objet
Identifiant métier
Ancienne valeur
Nouvelle valeur
Justification
Adresse IP
Session
Résultat

Exemple :

15/09/2026 14:35

Utilisateur : Responsable pédagogique
Action : MODIFICATION_NOTE

Étudiant : MAT-2026-00125
Module : EM101
Ancienne note : 11,50
Nouvelle note : 12,50

Justification : Correction après réclamation

18.19. Audit des opérations sensibles

Doivent obligatoirement être tracées :

- modification d'une note validée ;
 - modification d'un résultat ;
 - décision de jury ;
 - paiement ;
 - annulation financière ;
 - remboursement ;
 - modification d'une permission ;
 - suppression logique ;
 - publication ;
 - signature ;
 - changement de configuration ;
 - restauration de données.
-

18.20. Paramètres système

Le système doit avoir une zone :

Paramètres → Système

Exemples :

Nom établissement
Sigle
Logo
Adresse
Téléphone
Email
Fuseau horaire
Langue
Devise
Format date
Format numéro
Formats documents
Paramètres email
Paramètres SMS
Paramètres push
Paramètres sécurité

18.21. Paramètres académiques ≠ paramètres système

Important :

Module 01

Année académique
Semestres
Crédits
Types d'évaluation
Volumes horaires
Règles académiques

Module 18

Authentification
Sessions
Sécurité
Email
SMS
API
Stockage
Sauvegardes
Journalisation

Il ne faut pas mélanger les deux.

18.22. Configuration sensible

Les secrets ne doivent jamais être stockés dans Flutter.

Exemples :

```
SECRET_KEY  
DATABASE_PASSWORD  
JWT_SECRET  
SMTP_PASSWORD  
PAYMENT_API_KEY  
SMS_API_KEY
```

Ils doivent rester côté serveur, idéalement dans :

- variables d'environnement ;
- gestionnaire de secrets ;
- infrastructure sécurisée.

Ils ne doivent jamais être committés dans Git.

18.23. Sauvegardes

Le système doit prévoir :

```
Sauvegarde complète  
Sauvegarde différentielle  
Sauvegarde incrémentale
```

Selon l'infrastructure réellement retenue.

La sauvegarde doit couvrir notamment :

- base de données ;
 - documents ;
 - fichiers utilisateurs ;
 - configurations nécessaires.
-

18.24. Principe de sauvegarde

Architecture recommandée :

```
PRODUCTION  
↓  
BACKUP
```

↓
STOCKAGE SÉPARÉ
↓
VÉRIFICATION
↓
TEST RESTAURATION

Une sauvegarde qui n'a jamais été testée en restauration ne doit pas être considérée comme pleinement fiable.

18.25. Sauvegardes et Git

Séparer :

CODE
↓
Git

de :

DONNÉES
↓
Dumps / sauvegardes

Les données de démonstration ne doivent pas être dépendantes de l'historique Git.

18.26. Restauration

Prévoir un processus contrôlé :

Sélection sauvegarde
↓
Contrôle intégrité
↓
Validation administrateur
↓
Restauration
↓
Vérification
↓
Journalisation

Une restauration doit être une opération hautement privilégiée.

18.27. Santé du système

Dashboard technique :

Backend	✓
Database	✓
API	✓
Storage	✓
Queue	✓
Notifications	✓
Email	✓
SMS	✓
Backup	✓

18.28. Monitoring

Surveiller :

- disponibilité API ;
 - erreurs HTTP ;
 - temps de réponse ;
 - erreurs DB ;
 - espace disque ;
 - CPU/mémoire si disponible ;
 - files d'attente ;
 - tâches échouées ;
 - sauvegardes ;
 - services externes.
-

18.29. Gestion des erreurs

Une erreur technique ne doit pas exposer :

```
Traceback Django
Chemin serveur
Mot de passe
Variables d'environnement
Requête SQL
```

à l'utilisateur.

L'utilisateur reçoit :

```
Une erreur est survenue.
Code : ERR-2026-000125
```

L'administrateur peut retrouver le détail dans les logs.

18.30. Maintenance

Prévoir un mode maintenance :

```
NORMAL
↓
MAINTENANCE PROGRAMMÉE
↓
MAINTENANCE ACTIVE
↓
RETOUR NORMAL
```

Afficher :

- message ;
- période ;
- services indisponibles ;
- contact éventuel.

18.31. Gestion des versions

Le système doit pouvoir connaître :

```
Version application
Version API
Version base de données
Version Flutter
Version backend
Date déploiement
Commit Git
```

Exemple :

```
INJS-LMD
Version : 2026.09.x
API : v1
Backend : Django 5.1.4
```

18.32. Sécurité des fichiers

Tout fichier envoyé doit être contrôlé :

- extension ;
- MIME type ;
- taille ;

- nom ;
- emplacement ;
- permissions.

Éviter de permettre l'exécution de fichiers uploadés.

18.33. Suppression

Le système doit privilégier :

ACTIF
↓
INACTIF
↓
ARCHIVÉ

plutôt que :

DELETE physique

La suppression physique doit être réservée aux cas techniques ou légaux explicitement autorisés.

18.34. Conservation historique

Les données suivantes doivent conserver un historique :

- étudiants ;
 - inscriptions ;
 - formations ;
 - affectations ;
 - enseignants ;
 - notes ;
 - résultats ;
 - jurys ;
 - paiements ;
 - factures ;
 - contrats ;
 - documents ;
 - permissions.
-

18.35. Gouvernance des données

Chaque donnée importante doit avoir :

Propriétaire métier
Source
Responsable
Statut
Historique
Durée de conservation
Niveau de confidentialité

18.36. Confidentialité

Niveaux possibles :

PUBLIC
INTERNE
CONFIDENTIEL
TRÈS_CONFIDENTIEL

Ils doivent être compatibles avec les droits utilisateur.

Exemple :

Document RH confidentiel
↓
Utilisateur RH autorisé
↓
Accès

mais :

Étudiant
↓
Accès refusé

18.37. Administration des permissions

L'écran d'administration doit permettre de visualiser :

Utilisateur
Rôle
Niveau
Module
Ressource
Action
Périmètre

Exemple :

Rôle	Module	Action	Périmètre
Enseignant	Notes	Saisir	Ses groupes
Enseignant	Présences	Modifier	Ses séances
Scolarité	Étudiants	Modifier	Scolarité
Direction	Rapports	Consulter	INJS
Finance	Paielements	Valider	Finance
Admin	Système	Administrer	INJS

18.38. Séparation des responsabilités

Principe essentiel :

SAISIE
 ≠
 CONTRÔLE
 ≠
 VALIDATION
 ≠
 ADMINISTRATION

Exemple financier :

Caissier
 ↓
 Saisie paiement

 Finance
 ↓
 Contrôle

 Comptabilité
 ↓
 Comptabilisation

 Direction
 ↓
 Supervision

Même logique pour les notes, jurys, documents et RH.

18.39. APIs

/api/auth/

```
/api/users/  
/api/roles/  
/api/permissions/  
/api/groups/  
  
/api/sessions/  
/api/devices/  
  
/api/security/  
/api/security/events/  
/api/security/audit/  
  
/api/system/  
/api/system/settings/  
/api/system/health/  
/api/system/version/  
  
/api/backups/  
/api/backups/{id}/restore/  
  
/api/maintenance/
```

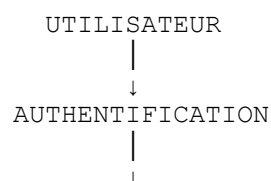
18.40. Modèles Django recommandés

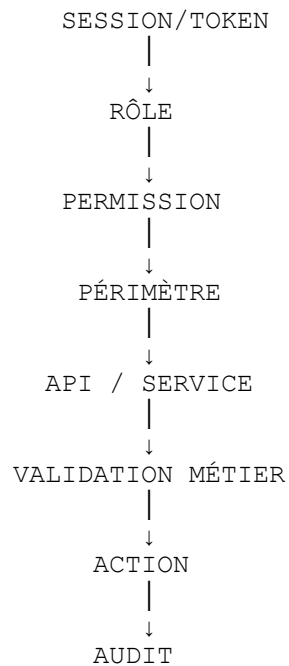
```
apps/  
└─ identity/  
    ├── User  
    ├── Role  
    ├── Permission  
    ├── UserRole  
    ├── UserScope  
    ├── Session  
    └── Device
```

```
apps/  
└─ audit/  
    ├── AuditEvent  
    ├── SecurityEvent  
    └── LoginEvent
```

```
apps/  
└─ core/  
    ├── SystemSetting  
    ├── MaintenanceMode  
    ├── ApplicationVersion  
    └── BackupRecord
```

18.41. Architecture de sécurité globale





18.42. Protection de Cline et des opérations techniques

Pour ton environnement de développement, il faut également conserver une règle importante :

```
Cline
↓
Analyse
↓
Proposition
↓
Confirmation
↓
Modification
↓
Tests
↓
Git diff
↓
Commit
```

Les opérations destructives doivent nécessiter une confirmation explicite :

- `rm -rf ;`
- suppression massive ;
- reset Git ;
- checkout destructif ;
- suppression de migrations ;
- suppression de base ;
- restauration DB ;
- écrasement massif de fichiers ;

- modification de configuration critique.

18.43. Règle Git

Le système de développement doit fonctionner avec :

AVANT MODIFICATION

```
↓  
git status  
↓  
backup/checkpoint  
↓  
Cline  
↓  
tests  
↓  
git diff  
↓  
validation  
↓  
commit
```

C'est particulièrement important dans ton projet puisque plusieurs branches/versions de l'application INJS ont déjà coexisté avec l'ancien projet QR Badge/SYGEP-CPFAE.

18.44. Tests de sécurité

Prévoir des tests pour :

Authentification

- mauvais mot de passe ;
- compte désactivé ;
- session expirée ;
- token invalide.

Autorisation

- accès interdit ;
- accès horizontal ;
- accès hors périmètre ;
- modification non autorisée.

Données

- injection ;

- upload malveillant ;
- données invalides ;
- accès à un document confidentiel.

Métier

- modification d'une note verrouillée ;
- modification d'un paiement validé ;
- modification d'un jury clôturé ;
- modification d'une année académique clôturée.

18.45. Critère majeur de réussite

Le système doit pouvoir répondre à :

Qui a fait quoi, quand, sur quelle donnée, avant/après quelle modification, avec quelle autorisation et dans quel contexte ?

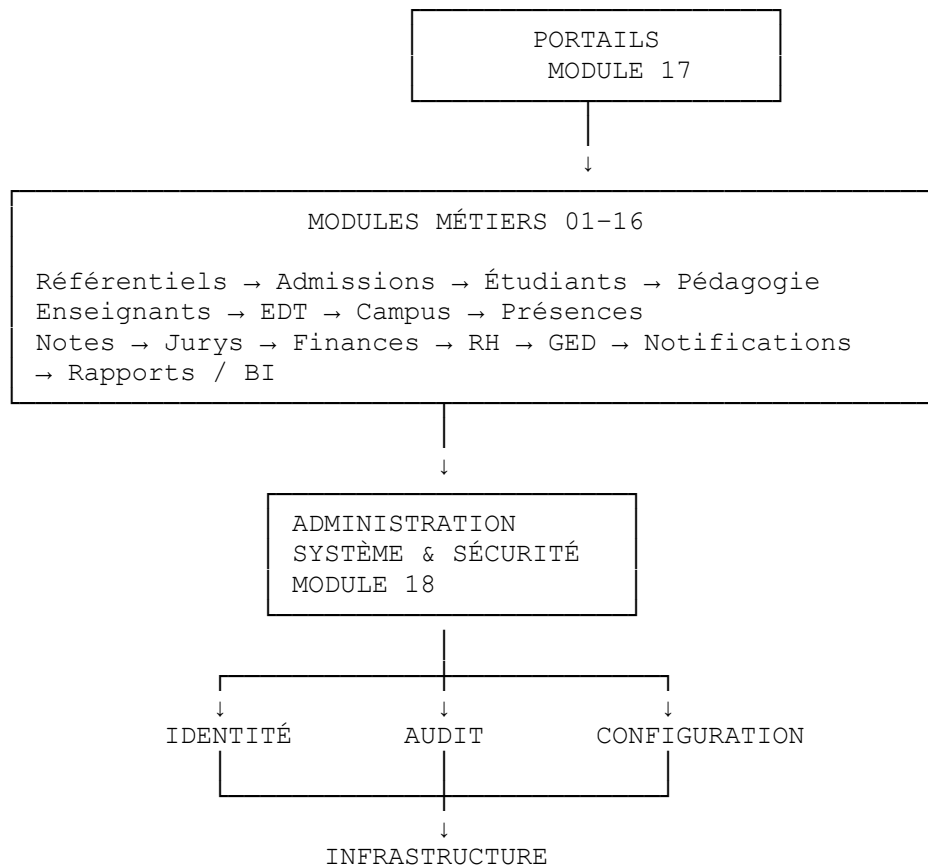
Si cette question ne peut pas être répondue, l'audit du système est insuffisant.

18.46. ARCHITECTURE FINALE DES 18 MODULES

Nous disposons maintenant de l'architecture fonctionnelle complète :

01. RÉFÉRENTIELS
02. CANDIDATURES & ADMISSIONS
03. ÉTUDIANTS
04. FORMATIONS & PÉDAGOGIE
05. ENSEIGNANTS
06. EMPLOIS DU TEMPS
07. CAMPUS & PATRIMOINE
08. PRÉSENCES
09. ÉVALUATIONS & NOTES
10. JURYS & DIPLÔMATION
11. FINANCES ÉTUDIANTS
12. FINANCE & COMPTABILITÉ
13. ADMINISTRATION & RH
14. DOCUMENTS, COURRIERS & ARCHIVES
15. NOTIFICATIONS & COMMUNICATIONS
16. RAPPORTS, STATISTIQUES & BI
17. PORTAILS & SERVICES EN LIGNE
18. ADMINISTRATION DU SYSTÈME & SÉCURITÉ

Architecture fonctionnelle globale



**Les 18 modules constituent maintenant le
référentiel fonctionnel maître
du système INJS-LMD.**

La prochaine étape logique n'est plus d'ajouter des modules : c'est de **transformer ces 18 modules en spécification technique exploitable par Cline**, avec pour chacun : **modèles Django → relations → migrations → serializers → ViewSets/API → permissions N0–N4 → écrans Flutter → workflows → validations → règles métier → tests → données de démonstration → ordre d'implémentation.**

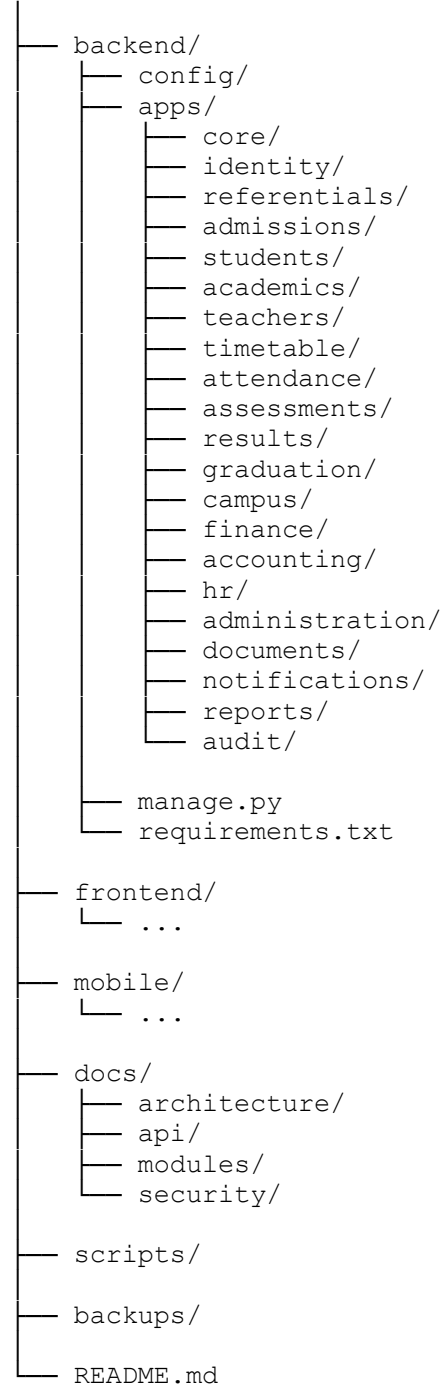
Parfait. Nous passons donc à la **phase d'industrialisation technique** : transformer les 18 modules fonctionnels en une spécification directement exploitable par **VS Code + Cline + Ollama**, sans laisser Cline réinterpréter l'architecture au fur et à mesure.

Je te propose de commencer par le **socle technique transversal**, avant d'implémenter les modules métier.

PHASE 1 — SOCLE TECHNIQUE INJS-LMD

1. Architecture cible

app-injs-lmd-2026/



Important : cette structure est une cible. Cline doit d'abord inspecter l'arborescence réelle avant de créer, déplacer ou supprimer quoi que ce soit.

2. Règle absolue pour Cline

Pour ton projet, je recommande de placer une règle permanente dans les instructions de Cline.

PROMPT — RÈGLES MAÎTRESSES CLINE

RÈGLES MAÎTRESSES — PROJET INJS-LMD 2026

Tu travailles sur l'application INJS-LMD 2026.

1. RÈGLE PRIORITAIRE

NE JAMAIS modifier, supprimer, déplacer ou remplacer massivement du code sans avoir d'abord inspecté le projet existant.

Avant toute intervention :

1. identifier le répertoire courant ;
2. inspecter l'arborescence ;
3. identifier backend, frontend et éventuelle application mobile ;
4. identifier le framework et les versions ;
5. lire les fichiers de configuration concernés ;
6. vérifier Git ;
7. identifier les modifications locales existantes ;
8. comprendre l'architecture déjà présente.

2. NE PAS DÉTRUIRE L'EXISTANT

Ne jamais :

- supprimer une application Django existante sans justification ;
- supprimer des migrations existantes ;
- supprimer une base de données ;
- réinitialiser Git ;
- exécuter `git reset --hard` ;
- exécuter `git clean -fd` ;
- supprimer massivement des fichiers ;
- remplacer l'application existante par une nouvelle application ;
- écraser le frontend existant ;
- modifier le logo ou l'identité visuelle sans demande explicite ;
- réintroduire une ancienne application telle que SYGEP-CPFAE / QR Badge.

3. GIT

Avant une modification importante :

```
git status  
git branch --show-current
```

```
git log -5 --oneline
```

Après modification :

```
git diff  
git status
```

Ne jamais effectuer de commit automatiquement sauf demande explicite.

4. MODIFICATIONS PAR PETITES ÉTAPES

Une tâche doit être découpée en petites étapes.

Pour chaque étape :

1. analyser ;
2. modifier le minimum nécessaire ;
3. tester ;
4. vérifier les erreurs ;
5. présenter le résultat ;
6. attendre la prochaine instruction si l'opération est importante.

5. BASE DE DONNÉES

Ne jamais supprimer ou recréer la base sans confirmation explicite.

Ne jamais supprimer les migrations existantes pour “résoudre” un problème.

Avant une migration :

```
python manage.py makemigrations
```

puis inspecter le résultat.

Ensuite seulement :

```
python manage.py migrate
```

6. DONNÉES

Ne jamais supprimer les données de démonstration existantes sans confirmation.

Les données de démonstration doivent être séparables du code.

Les identifiants internes Django/PostgreSQL ne doivent jamais être utilisés comme identifiants métier visibles.

7. API

Le frontend Flutter/Web communique uniquement avec Django/DRF.

Ne jamais donner au frontend un accès direct à PostgreSQL.

Toutes les permissions doivent être vérifiées côté backend.

8. SÉCURITÉ

Ne jamais placer :

- mot de passe ;
- SECRET_KEY ;
- clé JWT ;
- clé API ;
- clé paiement ;
- identifiant SMTP ;
- secret SMS

dans Flutter, Git ou le code frontend.

Utiliser les variables d'environnement ou un mécanisme sécurisé de secrets.

9. PERMISSIONS

Conserver le système de droits N0–N4 existant.

Ne pas remplacer arbitrairement le système de permissions par un nouveau système.

Toute évolution doit être compatible avec les rôles, permissions et périmètres existants.

10. IDENTIFIANTS MÉTIER

Les interfaces et exports doivent utiliser les codes et libellés métier :

Exemples :

MAT-2026-00125

EM-LIC

EM-L2

EM101

et non :

student_id=125

formation_id=7

module_id=43

11. RÉFÉRENTIELS

Ne jamais coder en dur dans Flutter les formations, modules, niveaux, salles, types, catégories, grades, années ou autres référentiels.

Ces données doivent provenir de l'API.

12. NOTES

Une note validée ou publiée ne doit jamais être modifiée silencieusement.

Toute correction doit être traçable.

13. FINANCES

Une opération financière validée ne doit jamais être supprimée physiquement.

Utiliser annulation, régularisation ou remboursement selon le workflow prévu.

14. DOCUMENTS

Les documents validés/signés doivent être immuables.

Utiliser la gestion de versions.

15. AUDIT

Les opérations sensibles doivent être journalisées :

- utilisateur ;
- date ;
- action ;
- objet ;
- ancienne valeur ;
- nouvelle valeur ;
- justification ;
- résultat.

16. AVANT TOUTE NOUVELLE FONCTIONNALITÉ

Répondre d'abord :

- Où cette fonctionnalité existe-t-elle déjà ?
- Quel module métier est concerné ?
- Quel modèle Django doit être utilisé ?
- Quelle API existe déjà ?
- Quel écran existe déjà ?
- Quel rôle est autorisé ?
- Quel workflow existe déjà ?

Ne pas créer de doublon.

17. PAS DE REFACTORING MASSIF

Si une fonctionnalité peut être ajoutée avec une modification locale, privilégier cette solution.

Ne pas refactoriser toute l'application pour résoudre un problème local.

18. TESTS

Après toute modification :

- lancer les tests concernés ;
- vérifier les imports ;
- vérifier les migrations ;
- vérifier l'API ;
- vérifier le frontend concerné ;
- vérifier la console ;
- vérifier Git diff.

19. RAPPORT FINAL

Après chaque tâche, fournir :

Modifications

Liste précise des fichiers modifiés.

Fonctionnalités

Ce qui a été ajouté ou corrigé.

Tests

Tests effectués et résultats.

Risques

Problèmes éventuels restant à traiter.

Prochaine étape

Une seule prochaine étape recommandée.

20. INTERDICTION DE DEVINER

Si une information fonctionnelle ou technique manque :

NE PAS INVENTER.

Identifier l'information manquante et demander confirmation.

21. PRINCIPE CENTRAL

Préserver l'existant.

Ajouter progressivement.

Tester constamment.

Tracer les modifications.

Ne jamais sacrifier les données ou la stabilité pour aller plus vite.

3. Ordre d'implémentation recommandé

Il ne faut surtout pas demander à Cline :

« Implémente les 18 modules. »

Ce serait beaucoup trop important pour ton environnement actuel.

L'ordre recommandé est :

```
ÉTAPE 0
Audit de l'existant
      ↓
ÉTAPE 1
Socle Core
      ↓
ÉTAPE 2
Identity / Auth / Permissions
      ↓
ÉTAPE 3
Référentiels
      ↓
ÉTAPE 4
Candidatures
      ↓
ÉTAPE 5
Étudiants
      ↓
ÉTAPE 6
Formations & pédagogie
      ↓
ÉTAPE 7
Enseignants
      ↓
ÉTAPE 8
```


EDT

↓
ÉTAPE 9

Campus

↓
ÉTAPE 10

Présences

↓
ÉTAPE 11

Évaluations / Notes

↓
ÉTAPE 12

Jurys / Diplômes

↓
ÉTAPE 13

Finance étudiant

↓
ÉTAPE 14

Finance / Comptabilité

↓
ÉTAPE 15

RH

↓
ÉTAPE 16

Documents

↓
ÉTAPE 17

Notifications

↓
ÉTAPE 18

Rapports / BI

↓
ÉTAPE 19

Portails

↓
ÉTAPE 20

Administration / Sécurité

↓
ÉTAPE 21

Tests globaux

↓
ÉTAPE 22

Préproduction

↓
ÉTAPE 23

Production

4. Première tâche à donner à Cline

Ne lui demande pas encore de coder.

Commence par un audit.

Tu travailles sur le projet INJS-LMD 2026.

IMPORTANT : cette tâche est UNIQUEMENT une tâche d'audit et de lecture.

NE MODIFIE AUCUN FICHIER.
NE CRÉE AUCUN FICHIER.
NE SUPPRIME RIEN.
NE LANCE AUCUNE MIGRATION.
NE MODIFIE PAS LA BASE DE DONNÉES.
NE FAIS AUCUN COMMIT GIT.

Objectif : établir l'état réel de l'application avant toute implémentation.

1. Affiche :

- répertoire courant ;
- arborescence principale ;
- branche Git ;
- git status ;
- derniers commits.

2. Identifie :

- backend ;
- frontend ;
- éventuelle application mobile ;
- Django ;
- DRF ;
- Flutter ;
- PostgreSQL/SQLite ;
- Vite/React si présent.

3. Identifie les applications Django existantes.

4. Identifie les modèles Django existants concernant :

- utilisateurs ;
- rôles ;
- permissions ;
- formations ;
- modules ;
- étudiants ;
- enseignants ;
- emplois du temps ;
- présences ;
- notes ;
- finances ;
- campus ;
- documents.

5. Identifie les routes API existantes.

6. Identifie les écrans frontend déjà existants.

7. Identifie le système de navigation existant.

8. Identifie le système de rôles/permissions existant, notamment N0–N4.

9. Identifie les variables d'environnement utilisées.

10. Vérifie s'il existe des références à :

- SYGEP-CPFAE ;
- QR Badge ;
- anciennes interfaces ;
- anciens logos ;
- anciennes routes ;
- anciennes API.

11. Vérifie les éventuelles migrations non appliquées sans les exécuter.

12. Vérifie les tests existants.

13. Vérifie les scripts npm disponibles.

14. Vérifie les commandes de démarrage backend/frontend.

15. NE CORRIGE RIEN.

À la fin, produis uniquement un rapport structuré :

A. Architecture réelle

B. Backend

C. Frontend

D. Base de données

E. Authentification

F. Permissions N0–N4

G. API

H. Modules déjà présents

I. Écrans déjà présents

J. Problèmes détectés

K. Risques

L. Éléments manquants

M. Recommandation pour la prochaine étape

Ne propose aucune modification automatique.

ATTENDS MA VALIDATION AVANT TOUTE MODIFICATION.

Pourquoi cette première étape est essentielle

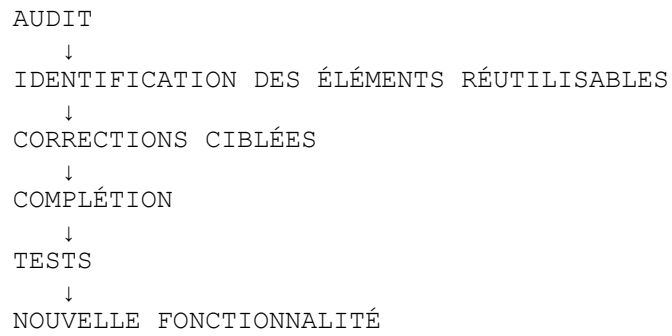
Dans ton cas, **l'audit doit précéder l'implémentation.**

Nous avons maintenant une architecture fonctionnelle très large, mais ton dépôt contient déjà du code, des modèles, des routes, des écrans et probablement des éléments issus de plusieurs versions du projet.

Le danger serait que Cline interprète notre architecture théorique comme une raison de **reconstruire l'application**, alors que l'objectif est plutôt :

EXISTANT

↓



Une fois que Cline aura produit ce rapport d’audit, colle-moi son résultat ici. Je pourrai alors te construire le **PROMPT 2**, adapté à l’état réel de ton dépôt, au lieu de lui faire prendre des décisions à l’aveugle.